

基于信息集分层的微网电力调控策略：因果预测、风险定价与滚动优化

摘要

光伏出力、居民负荷和外网电价都具有明显的时变性，储能又把相邻时段联结起来，微网购电由此形成带状态约束、预测误差和计划偏差结算的连续决策问题。围绕题目给出的四种信息条件，本文始终保持微网能量平衡与储能状态递推不变，仅随问题推进改变计划时可用的信息、允许的调整方式及相应费用，从而形成统一的日前—执行—滚动调度框架。

对于问题一，在典型日电价、负载和光伏预测均已知时建立确定性线性规划，以全天购电费最小为目标，联合决定外网购电与储能充放电，并固定日初、日末储电量为 6000 kWh。求得全天计划购电量为 59482.70 kWh，购电费为 35126.95 元；与禁用储能时的 48052.05 元相比，费用降低 12925.10 元，降幅为 26.8981%。结果表明，储能主要通过低价时段充电、较高电价时段放电以及吸收中午光伏富余实现跨时段能量转移（详见第 12 页表 3）。

对于问题二，每天 0:00 制定的购电计划必须先于当天实际源荷实现。为避免把未来真实数据提前用于计划，主模型只利用此前历史数据预测当天负载和光伏，并将计划购电量在日内锁定。针对紧急购电价格为正常电价 5 倍这一非对称代价，利用单时段边际损失得到光伏低分位风险修正的解析依据，再在历史窗口中滚动选择预测器与安全边际；执行阶段允许储能根据已实现偏差实时缓冲，仍不能覆盖的部分才紧急购电。2 月至 12 月总费用为 14773804.78 元，紧急购电量为 348036.96 kWh。对照实验表明，负荷信息边界、光伏风险修正和储能实时反馈都会显著影响全年成本（详见第 16 页表 5）。

对于问题三，在 0:00 先形成全天基准计划，在 6:00、12:00、18:00 获得新预报后，只对尚未执行的时段重新优化，并以当时实测储电量衔接前后两个滚动阶段。结算时，未被调整的部分按正常电价计费，少购的部分按电价的 50% 支付违约费用，超出计划的部分按电价的 150% 计费；本文据此构造主模型，并将“计划购电费与调整费用并列相加”的另一种理解作为对照。仅在 0:00 决策时，全年费用为 15731164.40 元；依次加入三个日内更新节点后，费用降至 14505850.38、14353879.08、13727998.70 元，累计节省 2003165.70 元。配对对照表明，节点收益由已实现负荷信息与新光伏预报共同形成，本文将其解释为决策更新的综合价值，其中光伏刷新的增量单独报告（详见第 20 页表 9）。

对于问题四，保持问题二、三的预测与滚动结构，将固定日电价替换为附件 4 给出的逐日电价重新优化。按题面直接使用附件 4 价格时，Q4-2 与 Q4-3 的全年费用分别为 15523884.83 元和 14387959.47 元，滚动更新节省 1135925.35 元。以历史价格预测替代未来完整价格作为现实性扩展后，价格信息受限会增加费用，但其影响小于源荷信息误差，且不改变日内滚动更新能够降低总成本的结论（详见第 24 页表 14）。

为保证结果满足物理可行性与时间可执行性，全文分别检验能量平衡、储能状态递推、容量和功率边界、计划持续性、费用回代以及信息因果性，并对终端储电量和效率取值进行敏感性分析。结果说明：储能提供了跨时段调节能力，对预测信息、实际观测和计划结算边界的严格区分，是避免微网调度结果过度乐观的关键。

关键词：微网经济调度；储能优化；因果预测；风险购电；滚动优化

一、问题重述

1.1 问题背景

研究对象为由居民负载、光伏电源、储能设备和外部电网构成的非孤岛微网。小区用电必须持续得到满足，但光伏具有明显的日内波动，外网电价也随交易时段变化。储能可以把低价时段或光伏富余时段的电量保存到后续使用。一天内各时段由储能状态相互联结：当前的充放电动作会改变后续储电量，进而影响以后能够少买多少电、是否需要紧急购电以及全天总费用。

题目以 10 min 为基本调度粒度。储能额定容量为 12000 kWh，允许的储电量范围为 1200 ~ 10800 kWh，最大充、放电功率均为 5000 kW，充放电效率均按 90% 处理。于是，一天 144 个时段通过储能状态递推形成连续的决策链：某一时段多充电，会提高后续可利用的储能；某一时段过早放电，则可能使真正高价或缺电时段失去调节余量。

除上述物理联系外，四个问题还存在一条更重要的递进主线，即**不同决策时刻能够获得的信息并不相同**。问题一给出的信息最完整，可以直接制定确定性全天计划；问题二要求计划先于当天实际源荷实现，并为预测偏差设置高价紧急购电；问题三允许在日内收到新预报后修改尚未执行的计划；问题四再把固定的典型日电价推广为逐日波动价格。由此，四问本质上是在同一微网物理系统上，逐步增加不确定性、信息更新和结算约束。

1.2 问题提出

问题一：在典型日电价、负载和光伏预测均给定的条件下，于每日 0:00 一次性确定全天各 10 min 时段的计划购电量和储能充放电量，使所有时段均满足负载，并保证日初、日末储电量相同。在论文中给出题目指定时段的购电结果、全天购电量与费用，以及各 4 h 时段的储能充放电量；完整方案写入 `result1.xlsx`。

问题二：当全年负载和光伏随日期变化时，仍要求每天 0:00 先制定全天计划；若实际运行后发现供电不足，缺口需按交易时刻正常电价的 5 倍紧急购入。这里存在两个相互制约的问题：计划过少会放大高价紧急购电风险，计划过多又会增加正常购电费用。模型需利用历史数据形成可执行的日前计划，并在实际偏差出现后合理调用储能缓冲。按题目要求给出 2025-03-20、2025-06-21、2025-09-23、2025-12-21 四个指定日期的购电、储能和紧急购电结果，完整方案写入 `result2.xlsx`。

问题三：每天 0:00、6:00、12:00、18:00 均可获得未来 24 h 的光伏预报。0:00 的全天计划是后续调整的基准，新的预报到达后只能修改尚未执行的部分，并对向上、向下调整分别计价。因而需要建立状态连续的滚动优化模型，并通过对照实验回答：新增的日内预报是否值得用于修改购电计划、各更新节点的经济价值有多大。同样对 2025-03-20、2025-06-21、2025-09-23、2025-12-21 给出题目要求的计划购电、调整购电、储能和紧急

购电结果，完整结果写入 result3.xlsx。

问题四：在问题二、三已有调度机制上进一步引入逐日波动电价。根据附件 4 重新决定购电和储能动作，比较固定典型日电价与实时波动价格下的调度差异，并对上述四个指定日期给出相应重算结果，分别形成与问题二、问题三对应的完整结果，写入 result4-2.xlsx 与 result4-3.xlsx。

1.3 研究现状综述

微网经济调度问题在数学上通常被建模为以购电费或综合运行成本最小为目标的线性规划或混合整数规划，外网购电、分布式电源消纳与储能充放电被统一纳入同一组能量平衡约束中 [1, 2]。确定性模型适用于负载、电价与光伏均已知的日前场景，其优点是约束结构清晰、求解稳定可靠。

当光伏出力存在预测误差时，主流处理方式有三类：一是**情景法 / 两阶段随机规划**，用有限个历史或生成情景刻画不确定性，第一阶段确定与情景无关的日前计划，第二阶段按情景确定执行量 [3, 4]；二是**鲁棒优化**，在最坏情景下保证可行性，代价是结果偏保守 [5]；三是**滚动时域优化（模型预测控制）**，在新信息到达后仅重优化剩余时域并只执行近期决策，天然适配“预报分批发布”的决策结构 [6, 7]。含光伏与储能的日前调度研究普遍指出，储能的价值高度依赖预测精度与调度时域长度 [8]，而储能的运行成本又与其循环深度密切相关 [9]。

针对“预测值应偏向哪一侧”的问题，报童模型给出的结论是：当高估与低估的边际损失不对称时，最优点决策由相应分位数确定 [10]。这一思想已被用于电力系统中的发电商投标策略与偏差电量定价 [11]。在预测方法层面，基于数值天气预报的机器学习模型是目前光伏功率预测的重要方法 [12]；在市场机制层面，实时电价与需求响应的耦合会显著改变用户的购电行为 [13]。

现有研究多聚焦于算法本身，对**信息集边界与结算规则**的刻画相对有限。在业务未提供全天计划负荷时，日前优化若直接使用当天真实负载，会混入理想信息优势；若实际业务已提供该信息，则可将其作为信息更充分的对照。比较不同策略时也应尽量保持预测方法、约束和费用规则中的非研究因素一致，才能解释费用变化来自何处。本文据此把信息边界作为建模原则，并用单因素对照与配对控制支持归因；求解则利用线性 / 混合整数线性结构，避免引入不必要的随机搜索误差。

二、 问题分析

2.1 总体思路

四个问题共享同一套微网物理结构。分别为每一问重新建模会重复书写能量平衡、储能容量和功率约束，也容易使效率定义、时间索引和计费方式前后不一致。本文先固

定“电如何在微网中流动”这一物理内核，再随着题目推进改变“决策时能看到什么”和“计划改变后怎样结算”，并把微网能量平衡与储能状态递推作为公共模型，把预测、风险修正、滚动更新和价格变化分别放到对应问题中处理。

在这条主线上，四问形成清晰的递进关系。问题一先回答确定信息下储能与外网购电如何协同；问题二引入计划与实际之间的偏差，重点转向预测误差和高价紧急购电风险；问题三给予日内更新预报，使模型能够比较“继续执行旧计划”和“付出调整代价修改计划”之间的得失；问题四则检验同样的控制逻辑在逐日电价变化后是否仍成立。全文据此按照“统一数据基准—建立公共物理模型—逐问扩展信息和费用规则—解释调度机理—验证结果可靠性”的顺序展开。

2.2 各问题的建模重点

问题一：附件 1 给出了典型日的电价、负载和光伏预测，可直接建立确定性经济调度模型。此时储能的收益来源十分明确：一是把低价时段购买的电转移到高价时段使用，二是把中午暂时超过负载的光伏保存到之后。题目又要求 0:00 与 24:00 储电量相同，因此不能通过消耗初始库存人为降低费用；首末储电状态必须闭合。

问题二：本问与问题一的关键差别在于全天计划必须在源荷真实实现之前确定。主模型严格遵循因果性，只利用 0:00 之前已经发生的数据形成当天负载和光伏预测；若实际业务中能够提前获得全天计划负荷，则把该情形作为信息更充分的对照，不与主模型混用。紧急购电价格达到正常价格的 5 倍，高估光伏会比同量低估产生更大的经济损失，预测也就不能只追求平均误差最小，还需要体现这种非对称风险。全天计划确定后不再修改外网购电量，但储能可根据当前已实现偏差调整当期充放电，使“计划层决策”和“执行层补偿”保持清晰分工。

问题三：本问的核心是判断新预测到达后，重新调整未来购电计划是否划算。每天 0:00 先形成覆盖全天的基准计划，6:00、12:00、18:00 到来时只重算尚未执行的区间，并从当前实测储电量继续递推。为了回答题目关于“其他时刻预报是否需要”的问题，必须保持物理约束、预测器和结算规则不变，只逐步增加允许使用的更新节点，再比较全年费用、紧急购电和调整量。由于节点到来时不仅有新光伏预报，也多了已经实现的负载和状态信息，节点收益应解释为综合决策信息的价值，并进一步用配对对照区分各信息来源。

问题四：题面已给出附件 4 的全年实时电价，正式重算直接以其替换前几问的典型日电价，其他源荷预测、储能反馈和滚动逻辑保持不变。这样，前后结果的差异可以主要归因于价格环境改变。考虑真实市场中未来价格未必在日初全部公开，再设置仅用历史价格预测未来价格的扩展情形，用于衡量价格信息受限的机会成本，而不替代题面主结果。

2.3 求解方法选择

上述能量平衡、状态递推、容量边界以及分段线性费用都可以写成线性约束。问题一、问题二及不要求充放电严格互斥的对照方案属于线性规划；当需要从物理上禁止同一时段发生充、放电重叠时，引入二元变量后形成混合整数线性规划。两类模型均由 HiGHS 求解。相较于遗传算法、粒子群等元启发式方法，线性/混合整数规划可以直接利用本题的结构，在给定规则下获得可验证的最优解，并能逐时回代检查每一项能量和费用，这与本文强调的可解释性和可复现性更一致。

三、模型假设

题目已经给出储能设备的主要参数，但对售电机制、日前信息边界和跨日终端状态等细节未全部规定。以下假设只用于补足这些必要条件，并尽量不额外引入题面之外的市场机制。

1. **统一时间尺度。**以 10 min 为一个调度时段，一天共有 $T = 144$ 个时段， $\Delta t = 1/6$ h；附件中的 0:00+1 视为当天 24:00。附件各行与结果模板按原顺序对应，不再人为平移时间标签。
2. **不考虑反向售电。**题目只规定从外网购电，没有给出向外网售电的价格和交易规则，外网购电量据此保持非负；光伏在满足负载和储能充电后仍无法利用的部分记为弃光。
3. **储能参数全年不变。**主模型取充电效率和放电效率均为 0.9，忽略自放电、容量衰减、循环寿命成本和启停损耗。由于“充放电效率为 90%”存在解释空间，另一种往返效率定义在灵敏度分析中另行检验。
4. **功率上限按时段折算。**最大充、放电功率 5000 kW 在一个 10 min 时段内对应最大充、放电量 $5000\Delta t = 833.33$ kWh，所有充放电约束均按该电量上限执行。
5. **全年储能状态连续。**2025 年 1 月 1 日 0:00 储电量为 6000 kWh，随后每日实际日末储电量直接作为下一日初值。官方结果从 2 月 1 日起填写，1 月既用于历史预测冷启动，也用于把储能状态自然递推到正式评价区间。
6. **问题二只使用决策时刻之前的历史数据。**在 0:00 制定全天计划时，不直接使用当天尚未实现的真实负载和真实光伏，而由历史数据形成因果预测；“日前负荷完全已知”仅作为信息更充分的对照，用来衡量信息价值。
7. **冷启动使用附件 1 先验。**1 月 1 日前不存在历史观测时，以附件 1 的典型日负载和光伏曲线作为一次性初始先验；一旦积累真实历史数据，即转入滚动历史预测，不再长期依赖附件 1 源荷曲线。
8. **允许储能实时缓冲计划偏差。**问题二的全天计划购电量在日内锁定，但当期真实负

载、光伏和储电量实现后，可以调整储能充放电。该调整只使用当前及过去信息，不读取未来真实数据；储能仍无法覆盖的供电缺口才计为紧急购电。

9. 问题二、三计划层采用日循环终端状态。除问题一的题面硬约束外，问题二、三主模型令计划终端储电量回到当日实际初始储电量，以避免有限时域优化在午夜通过透支电池获得虚假的短期收益；取消该约束的情形在敏感性分析中单独比较。
10. 问题四主结果直接使用附件 4 电价。这是题面明确给出的重算数据；仅用历史价格预测未来价格的方案作为现实市场信息受限时的扩展分析，不替代正式结果。

四、符号说明

主要符号列于表 1。为避免功率与电量混淆，负载和光伏仍以 kW 表示，进入每个 10 min 时段的能量平衡前乘以 Δt ；购电、充放电、弃光和紧急购电均直接以 kWh 表示。 $S_{d,t}$ 表示储能内部实际储电量，而非百分比形式的 SOC。

表 1 主要符号说明

符号	单位	含义
d, t	—	日期下标与日内 10 min 时段下标 ($t = 1, \dots, 144$)
Δt	h	单个调度时段长度, $\Delta t = 1/6$
$L_{d,t}$	kW	小区负载功率
$P_{d,t}$	kW	光伏实际发电功率
$\hat{P}_{d,t}$	kW	决策时刻可用的光伏预测功率
$\pi_{d,t}$	元/kWh	外网正常购电单价
$G_{d,t}^p$	kWh	0:00 制定的计划购电量
$G_{d,t}^a$	kWh	执行前最后一次有效调整后的购电量
$C_{d,t}$	kWh	流入储能的充电量
$H_{d,t}$	kWh	储能输出的放电量
$S_{d,t}$	kWh	第 t 时段结束时储能内部储电量
$W_{d,t}$	kWh	未被利用的弃光电量
$E_{d,t}$	kWh	供电不足时的紧急购电量
$u_{d,t}, v_{d,t}$	kWh	调整购电量相对计划的上调量 $(G^a - G^p)^+$ 与下调量 $(G^p - G^a)^+$
$z_{d,t}$	—	充放电互斥变量； $z = 1$ 允许充电， $z = 0$ 允许放电
η_c, η_d	—	充电效率与放电效率，主模型均取 0.9
$S_{\min}, S_{\max}$	kWh	储电量上下限，分别为 1200 与 10800
$\bar{E}$	kWh	单时段充放电量上限， $\bar{E} = 5000\Delta t = 833.33$
$\mathcal{I}_{d,\tau}$	—	第 d 天决策节点 τ 已经获得的信息集合

后文最容易混淆的是 G^p 、 G^a 与 E 。 G^p 是 0:00 对未来各时段作出的基准承诺， G^a 是问题三中的在日内新预报到达后、交易时段执行前最后一次生效的调整量；二者的差额决定调整结算。即使已经进行过调整，真实运行仍可能再次偏离预测，因此还保留紧急购电 E 作为最终供电兜底。三个量分别对应“日初计划—日内修订—实际缺口”，其时

间顺序贯穿问题二、三的全部建模。

五、数据预处理

四个问题虽然调用的附件不同，但所有购电、充放电和状态递推最终都落在同一条 10 min 时间轴上。建模前先统一时间、功率和电量的数据基准。附件 1 给出典型日的 144 个时段，附件 2 给出全年逐日实际源荷，附件 3 给出四个发布时刻的未来 24 h 整点光伏预报，附件 4 则给出全年实时电价。本文按附件原有行序与结果模板逐项对应，保持附件原有时间顺序，从源头避免不同附件之间的时段错配。

负载与光伏的原始单位为 kW，而购电量、充放电量和储电量都以 kWh 计。故在进入能量平衡式前，将每个时段的功率统一乘以 $\Delta t = 1/6 \text{ h}$ 。数据完整性检查表明，负载和光伏均非负、电价均为正，附件 2 与附件 3 的日期能够一一对应。统计得到，全年负载电量约为 $40.52 \times 10^6 \text{ kWh}$ ，光伏电量约为 $20.25 \times 10^6 \text{ kWh}$ ；约 19.27% 的时段光伏功率高于负载，其中 241 个时段的富余功率甚至超过储能最大充电功率 5000 kW。换言之，全年确实存在“光伏已经超过负载且储能也未必来得及吸收”的时段，说明弃光具有实际的能量去向含义，必须在模型中显式表示。

在给出具体约束之前，先观察数据中哪些矛盾会真正驱动调度。图 1 将典型日电价与源荷、全年季节变化以及实时价格日内波动放在同一组图中，该图用于判断储能在什么条件下具有跨时段转移价值。

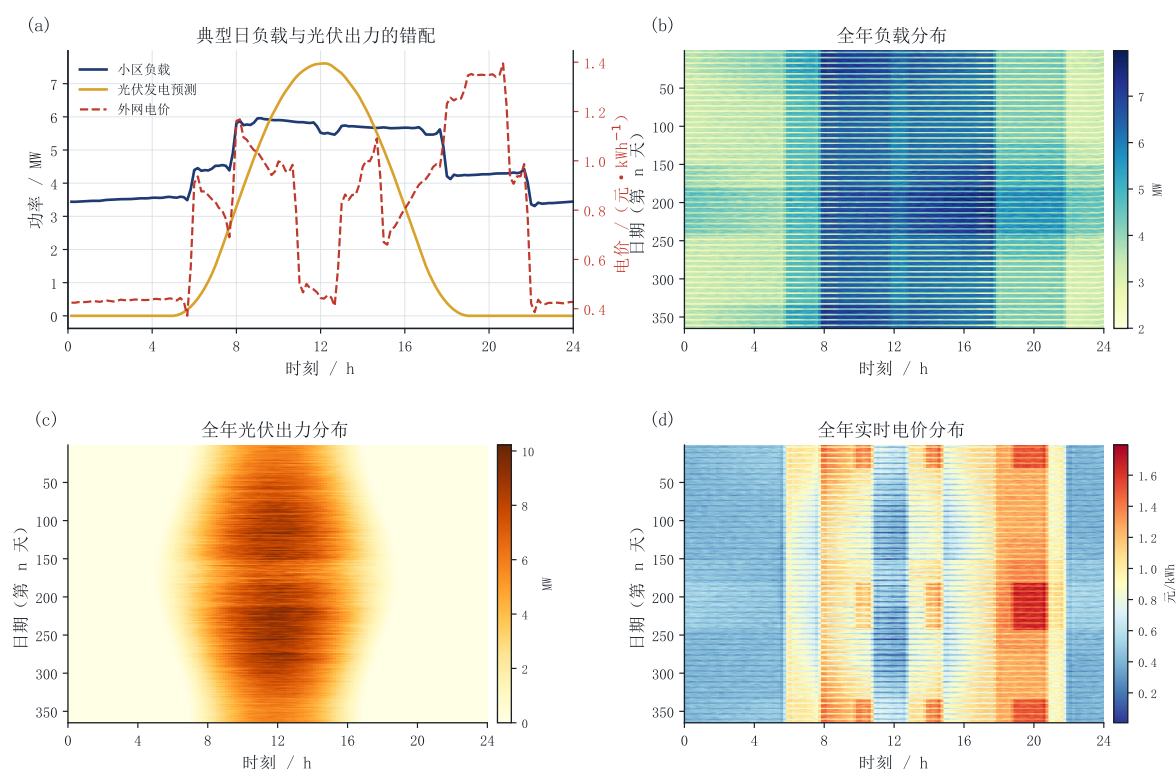


图 1 光伏与高电价峰存在明显错位，储能具备跨时段转移空间

图 1(a) 中，光伏出力主要集中在白天，而较高电价更多出现在早晚负荷峰段，二者

的时间错位正是储能套利的来源：中午富余或低价购入的电可以被保存到晚间再使用。图 1(b)(c) 显示，这种矛盾随季节明显变化。夏季负载与光伏共同升高，冬季则出现负载抬升而光伏衰减，同一调度规则在不同季节面对的是不同程度的源荷失配；附件 4 的逐日电价又叠加了价格环境变化。后续模型既需要统一的物理内核，也必须允许预测和价格随日期更新。

仅比较负载和光伏两条曲线仍不便直接判断系统是否需要外部供电。为此定义净负荷 $L - P$ ：其为正表示本地光伏不足，为负则表示当期光伏超过负载。图 2 用这一量展示季节性的供需失配。

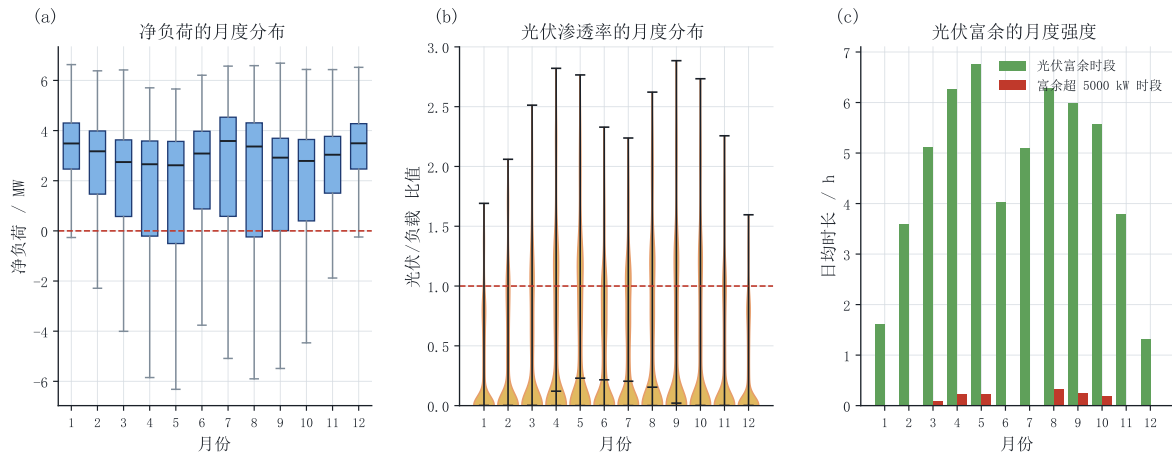


图 2 净负荷与光伏渗透呈显著季节差异，夏季富余最集中

图 2 中，春、秋两季的中午频繁出现负净负荷，说明光伏在这些时段已经能够覆盖全部居民需求；夏季 6 ~ 8 月光伏渗透率中位数接近 1，储能更像是“中午吸收、傍晚释放”的能量缓冲器；冬季净负荷整体抬升后，光伏富余减少，储能的经济价值更多来自电价峰谷转移。净负荷与负载、光伏的相关系数分别为 -0.1140 和 -0.8796 ，全年波动对光伏变化更敏感。由此，后文的三个设计都有了数据依据：对真实富余设置弃光通道，对光伏预测误差进行风险修正，并在新预报到来后允许滚动更新未来计划。

六、模型的建立与求解

6.1 公共物理模型

四问的价格规则和信息条件不断变化，但“电从哪里来、到哪里去”始终相同。为使后续费用差异能够归因于信息和交易规则，先建立一套公共物理内核。之后每一问只替换相应的目标函数、信息集或结算关系。

以第 d 天第 t 个 10 min 时段为基本单元。日初计划购电量记为 $G_{d,t}^p$ ，若问题三在日内对该时段进行了修订，则执行前最后一次生效的购电量记为 $G_{d,t}^a$ ；流入、流出储能的电量分别为 $C_{d,t}$ 、 $H_{d,t}$ ，弃光与紧急购电分别记为 $W_{d,t}$ 、 $E_{d,t}$ ， $S_{d,t}$ 表示时段结束时电池

内部的储电量。下面的约束依次回答三个问题：当期电量怎样守恒、储能状态怎样连续、计划在时间上怎样保持可执行。

6.1.1 微网能量平衡

首先要求每个时段的电量闭合。外网购电、光伏、储能放电和紧急购电是微网中的电量来源；居民负载、储能充电和无法利用的光伏是电量去向。将负载和光伏功率乘以 Δt 后，得到

$$G_{d,t} + P_{d,t}\Delta t + H_{d,t} + E_{d,t} = L_{d,t}\Delta t + C_{d,t} + W_{d,t}, \quad \forall d, t. \quad (1)$$

式 (1) 把题面的“供电不能低于负载”进一步写成完整能量守恒。该式取等号（而非不等式），以免多余电量无解释地消失；写成等式后，任何富余都必须进入储能或弃光变量 $W_{d,t}$ 。 $W_{d,t}$ 用来记录储能受容量或功率限制时真正无法消纳的光伏。

6.1.2 储能状态递推与运行边界

能量平衡只约束当前时段，真正把一天 144 个时段连成一个整体的是储能状态。按上述定义，流入储能的 $C_{d,t}$ 只有 $\eta_c C_{d,t}$ 真正进入电池；储能向微网输出 $H_{d,t}$ 时，电池内部需减少 $H_{d,t}/\eta_d$ 。于是

$$S_{d,t} = S_{d,t-1} + \eta_c C_{d,t} - \frac{H_{d,t}}{\eta_d}, \quad S_{d,0} = S_d^{\text{init}}. \quad (2)$$

式 (2) 决定了“今天早些时候怎么用电池”会怎样影响以后。问题三每到一个新预报节点时，过去已经发生的充放电不能被重新安排，只能把当时实际储电量 S 作为新的起点继续优化。新预报即使十分有利，也必须服从此前真实运行留下的状态，这正是滚动调度与重新计算一张理想全天计划之间的区别。

根据题目给出的安全储电范围和最大充放电功率，有

$$1200 \leq S_{d,t} \leq 10800, \quad \forall d, t, \quad (3)$$

以及

$$0 \leq C_{d,t} \leq \bar{E}, \quad 0 \leq H_{d,t} \leq \bar{E}, \quad \bar{E} = 5000\Delta t = 833.33 \text{ kWh}. \quad (4)$$

式 (3) 防止过充过放，式 (4) 则把 5000 kW 的功率限制正确换算到 10 min 时段。题目没有给出反向售电机理，外网购电和紧急购电均保持非负：

$$G_{d,t}^p \geq 0, \quad G_{d,t}^a \geq 0, \quad E_{d,t} \geq 0. \quad (5)$$

弃光只能来自当期光伏，还需满足

$$0 \leq W_{d,t} \leq P_{d,t} \Delta t. \quad (6)$$

问题一明确要求日初与日末储电量相同，且本题初始值为 6000 kWh，故

$$S_{d,144} = S_{d,0} = 6000 \text{ kWh}. \quad (7)$$

问题二、三没有这一硬性要求。主模型仍在计划层采用“计划终端储电量等于当日实际初始储电量”的日循环策略，以避免有限时域模型在日末通过放空电池获得虚假的短期收益；其影响在灵敏度分析中单独检验。

6.1.3 决策信息边界

物理上可行的低费用方案还可能存在另一类问题：它可能在计划阶段使用了未来才会发生的数据。为了把“什么时候知道什么”也写成模型的一部分，记 $\mathcal{I}_{d,\tau}$ 为第 d 天节点 $\tau \in \{0, 6, 12, 18\}$ 已经获得的信息，则节点 τ 产生的未来决策必须满足

$$X_{d,t}^{(\tau)} \in \sigma(\mathcal{I}_{d,\tau}), \quad t \geq 6\tau + 1. \quad (8)$$

式 (8) 可以理解为一个“不可预知性”要求：若两个情形在某个节点之前完全相同，那么此时形成的计划也应完全相同；只有节点之后真正获得了不同信息，未来方案才允许分化。为把这一抽象条件落实到求解流程，表 2 列出四个节点各自可使用的信息以及下一次更新前真正执行的区间。

表 2 四个决策节点只执行至下一次更新，后续计划保留滚动修订空间

节点	可使用的信息 $\mathcal{I}_{d,\tau}$	当次执行区间
0:00	前一日及更早实际负载、光伏、价格；0:00 发布的光伏预报；当日初始储电量	0:00–6:00
6:00	加入 0:00–6:00 已实现观测、6:00 光伏预报及实测储电量	6:00–12:00
12:00	加入 6:00–12:00 已实现观测、12:00 光伏预报及实测储电量	12:00–18:00
18:00	加入 12:00–18:00 已实现观测、18:00 光伏预报及实测储电量	18:00–24:00

表 2 体现了后文滚动模型最重要的规则：优化时可以看较长的未来，以便决定当前是否应为后续保留储能；执行时却只落实到下一个信息节点为止。下一节点到来后，再从真实状态重新规划。这种“看得远、执行近”的安排既保留了储能的前瞻价值，又不会把尚未发生的数据提前当成事实。

若需要严格禁止同一时段同时充、放电，引入二元变量 $z_{d,t}$ ：

$$0 \leq C_{d,t} \leq \bar{E} z_{d,t}, \quad 0 \leq H_{d,t} \leq \bar{E}(1 - z_{d,t}), \quad z_{d,t} \in \{0, 1\}. \quad (9)$$

由式 (9)，一个时段只能处于充电或放电一侧，从而排除连续模型可能出现的“边充边放”数值退化。采用该约束时模型为混合整数线性规划；不需要整数互斥时则为线性规划。

6.2 问题一：确定性日前调度

6.2.1 模型建立

问题一给出了最清晰的基准场景：日初已经知道当天电价、负载和光伏预测，不存在后续调整价格，也无需为预测偏差单独计费。此时优化的核心只有一个——在保证每个时段供电和储能安全的前提下，决定哪些电应当直接购买、哪些电值得先存后用。全天正常购电费写为

$$\min F_1 = \sum_{t=1}^{144} \pi_t G_t. \quad (10)$$

将式 (10) 与公共物理约束和首末储电量闭环结合，即得到问题一的确定性线性规划。若只以费用为唯一目标，某些时段可能存在多组同价方案，例如在不改变购电费的前提下做无必要的储能循环。为使最终轨迹更符合实际运行，求得最低费用后再依次最小化储能吞吐量和弃光量；这不会改变最优购电费，只在同价解中选取动作更简洁的一条。

6.2.2 求解结果与调度解释

求解得到全天计划购电量 59482.70 kWh，购电费 35126.95 元；储能全天充电 20740.67 kWh、放电 16799.94 kWh，日初与日末均为 6000 kWh，并实现零弃光。作为对照，若完全禁用储能，购电费为 48052.05 元，储能由此使典型日费用降低 12925.10 元，即 26.8981%。

为与题面要求直接对应，表 3 给出六个指定时段及全天购电结果，表 4 汇总六个 4 h 时段的储能充放电量。二者分别回答“从外网买了多少电”和“这些电在日内如何通过储能转移”。

表 3 问题一：储能使全天购电费降至 35126.95 元，高价指定时段购电显著下降

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量/kWh	59482.70	全天购电费/元	35126.95		

表 3 中 10:00、14:00 和 20:00 三个指定时段的外网购电量为零，而其他指定时段仍保留一定购电。仅从购电量本身还无法判断这些负荷由何处补足，接着查看同一天的储能充放电结果。

表 4 问题一：储能在低价与光伏富余时段充电，在高价时段集中放电

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.00	0.00	12:00–16:00	5286.04	91.10
4:00–8:00	833.33	6365.84	16:00–20:00	0.00	5780.13
8:00–12:00	4787.96	1703.00	20:00–24:00	5333.33	2859.87
0:00 储电量/kWh	6000.00		24:00 储电量/kWh	6000.00	

表中若干高价时段购电量显著下降，这些时段的外网购电被此前储存的低价电或光伏电替代。为观察这种“时间搬移”如何发生，图 3 一并给出购电、充放电与储电量曲线。

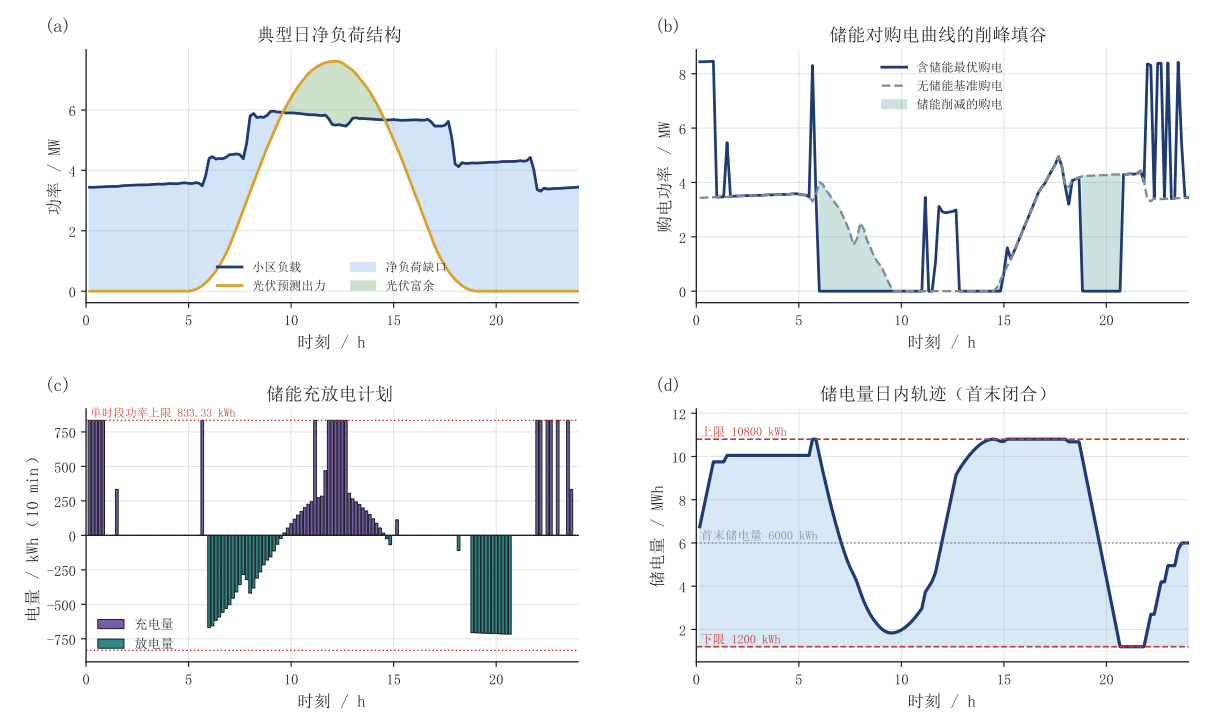


图 3 问题一：低价充电、高价放电形成日内能量转移并保持首末储电量闭合

由图 3 可见，储能在凌晨低价段和中午光伏较多时吸收电量，在早间、午后和傍晚较高电价时段放电；整个过程中储电量始终位于 1200 ~ 10800 kWh 安全范围内，并最终回到 6000 kWh。问题一的节省主要来自“低价充、高价放”和“光伏先存后用”两种机制，居民负载仍得到完整满足。完整 144 时段结果见 result1.xlsx。

6.3 问题二：历史信息下的风险购电与实时储能反馈

6.3.1 信息边界与两阶段决策结构

问题二仍要求在 0:00 给出全天计划，但附件 2 记录全年最终实现的负载和光伏，日初并未给出对应的全天计划预测。若直接把当天完整真实曲线送入计划层，求得的只是信息充分情形下的理想调度。为得到能够按真实时间顺序执行的主策略，本文在日初只使用此前历史形成当天源荷预测；另保留“全天计划负荷已知”的方案作为对照，用来量化信息更充分时最多可以节省多少费用。

这样，问题二自然分成“先承诺、后补救”两个阶段。0:00 需要先决定全天计划购电量 G^p ；进入实际运行后，外网计划不再改变，只能通过储能和紧急购电吸收预测误差。若把尚未实现的源荷视为随机量，费用结构可概括为

$$\min_{G^p} \mathbb{E} \left[\sum_t \pi_t G_t^p + \sum_t 5\pi_t E_t \right]. \quad (11)$$

式 (11) 揭示了本问真正的经济矛盾：计划得越保守，正常购电费越高；计划得越激进，一旦光伏或负荷预测失准，就可能承担 5 倍电价的紧急补购。本文不再为有限历史强行拟合一个高维联合概率分布，而采用逐日的因果近似：先从已有历史产生当天预测，再按预测求一条完整计划，随后用当天真实数据逐时执行和结算。这样既保留了“两阶段”结构，又能清楚追踪每一天计划如何被实际偏差检验。

6.3.2 光伏预测的非对称风险修正

上述计划如果只追求 MAE 或 RMSE 最小，会默认高估和低估光伏同样严重，但题目的 5 倍紧急价格打破了这种对称性。设预测误差为 $\hat{P} - P$ 。高估光伏 δ 时，模型会少安排约 δ 的正常购电，而实际缺口可能不得不从 π 的计划电价升级为 5π 的紧急电价，额外损失约为 $4\pi\delta$ ；低估同样的 δ ，主要代价则是多安排了正常购电。因而，调度应直接区分两类误差方向对应的经济后果。单时段损失可写为

$$\ell(\hat{P} - P) = 4\pi(\hat{P} - P)^+ + \pi(P - \hat{P})^+. \quad (12)$$

式 (12) 给出了 4:1 的非对称代价。若暂时忽略储能跨期耦合，把问题看成单时段报童模型，则最优光伏点预测满足 [10, 11]

$$F(\hat{P}^*) = \frac{1}{4+1} = 0.2, \quad \hat{P}^* = F^{-1}(0.2). \quad (13)$$

式 (13) 只是在忽略储能跨期耦合后的单时段解析结果，0.2 分位在此提供风险修正方向的解析基准：紧急电价越高，调度越应避免高估可用光伏。真正的全年模型还受到

负荷误差、SOC、充放电上限和实时反馈共同影响，本文把 0.2 分位视为构造候选安全边际的理论起点，再用已实现历史选择具体方案。

设第 m 个基础预测器给出的光伏预测为 $B_{d,t}^m$ ，定义历史残差

$$R_{d,t}^m = B_{d,t}^m - P_{d,t}. \quad (14)$$

若要把基础预测向较保守的光伏低分位移动，则从基预测中减去残差的高分位：

$$\hat{P}_{d,t}^{(m,q)} = \max \{0, B_{d,t}^m - Q_q(\mathcal{R}_m)\}, \quad \alpha = 1 - q. \quad (15)$$

由此，光伏输出 0.2 分位对应残差 0.8 分位。为避免把单时段理论机械推广到全年，实际在 $q = 0.80, 0.85, 0.90, 0.95$ 与多个基础预测器之间滚动选择。评价候选时仍采用与经济损失一致的非对称代理：

$$J_m = \sum_{d' \in \mathcal{W}} \sum_t \pi_{d',t} \left[4(\hat{P}_{d',t}^{(m)} - P_{d',t})^+ + (P_{d',t} - \hat{P}_{d',t}^{(m)})^+ \right] \Delta t. \quad (16)$$

其中 $\mathcal{W}$ 为过去 30 天。每天只用已经实现的历史误差选择次日模型，不用次日真实值反选参数；近期历史、持续法与滚动均值等基线的比较方式参考光伏预测方法的常见评估思路 [12]。四个分位候选的累计代理费用和被选天数分别为： $q = 0.80$ 时 1889056.42 元、162 天； $q = 0.85$ 时 1866485.67 元、109 天； $q = 0.90$ 时 1944258.96 元、53 天； $q = 0.95$ 时 2278103.51 元、10 天。负载预测采用持续法、7/14/30 日均值、线性加权均值和同星期均值等候选，并以历史平均绝对误差选择。

6.3.3 实际执行：购电计划锁定，储能实时缓冲

计划层和执行层在问题二中承担不同职责。日初要解决的是“今天各时段向外网承诺买多少电”，所以 G^p 一旦形成便保持不变；实际运行要解决的是“当预测与现实不一致时，微网内部怎样维持供需平衡”，储能不能机械追踪日初充放电轨迹，而应根据当期已经实现的负载、光伏和储电量作反馈修正。这样既不改变外网计划，又能发挥储能作为内部缓冲资源的作用。

当真实供电不足时，首先减少原计划充电；若仍有缺口，在储电量和放电功率允许范围内增加放电；只有储能仍不能补足的部分才计为 E_t ，按 $5\pi_t$ 紧急购买。若真实光伏高于预期，则先减少原计划放电，再增加充电，储能仍无法吸收的部分记为弃光。这个执行规则只使用当前和过去已经发生的信息，不会把问题三的未来预报能力提前带入问题二。

6.3.4 求解结果与原因拆解

按上述“日初计划—逐时执行”流程滚动至全年，正式评价区间为2月1日至12月31日，共334天。主方案总费用为14773804.78元，其中正常计划购电费12877657.23元、紧急购电费1896147.55元；紧急购电量为348036.96 kWh，弃光量为2326222.67 kWh。负载和光伏预测的平均绝对误差分别为188.88 kW和198.14 kW；物理残差、费用回代与求解数值精度见第27页的独立检验。

然而，一个总费用只能说明方案“花了多少钱”，不能说明“为什么花成这样”。为了把不同机制的贡献拆开，表5固定公共物理约束和其他参数，每次只替换一个条件：A组把全天计划负荷视为提前已知，衡量更充分负荷信息的价值；B组是本文主方案；C组取消光伏风险下修，观察仅使用普通点预测的后果；D组关闭储能实时反馈，检验储能是否真正吸收了日内偏差。

表5 问题二：负载信息、光伏风险修正和储能反馈共同决定全年费用

方案	负载信息	光伏预测	实时反馈	总费用/元	紧急购电/kWh	弃光/kWh
A	理想信息	风险边际	开	13275988.13	61482.52	2000031.09
B	因果	风险边际	开	14773804.78	348036.96	2326222.67
C	因果	点预测	开	16038813.56	656904.88	1688048.18
D	因果	风险边际	关	15360709.59	672280.72	2566068.96

表5中，几组方案的费用差来源不同：有的是因为计划时掌握的信息不同，有的是因为对光伏高估风险的态度不同，还有的是因为执行阶段是否允许储能补救。为避免只看总费用而忽略这些机制，图4一并展示计划购电费、紧急购电费和相对主方案的费用增量。

三个关键差额为

$$\begin{aligned}
 \Delta C_{\text{load}} &= C_B - C_A = 1497816.65 \text{ 元}, \\
 \Delta C_{\text{PVrisk}} &= C_C - C_B = 1265008.78 \text{ 元}, \\
 \Delta C_{\text{feedback}} &= C_D - C_B = 586904.81 \text{ 元}.
 \end{aligned} \tag{17}$$

式(17)表明，只使用历史信息的方案相对“全天计划负荷已知”基准增加1497816.65元，占主方案费用的10.14%；这1497816.65元来自两种信息条件之间的价值差。取消光伏风险边际后，费用增加1265008.78元，说明在5倍紧急电价下适度保守能够用较便宜的全天计划电量替代昂贵缺口；关闭实时反馈后费用增加586904.81元，且紧急购电由348036.96 kWh增至672280.72 kWh，说明储能确实承担了日内偏差缓冲功能。题面要求的四个指定日期结果见表6–8，完整逐时结果见result2.xlsx。

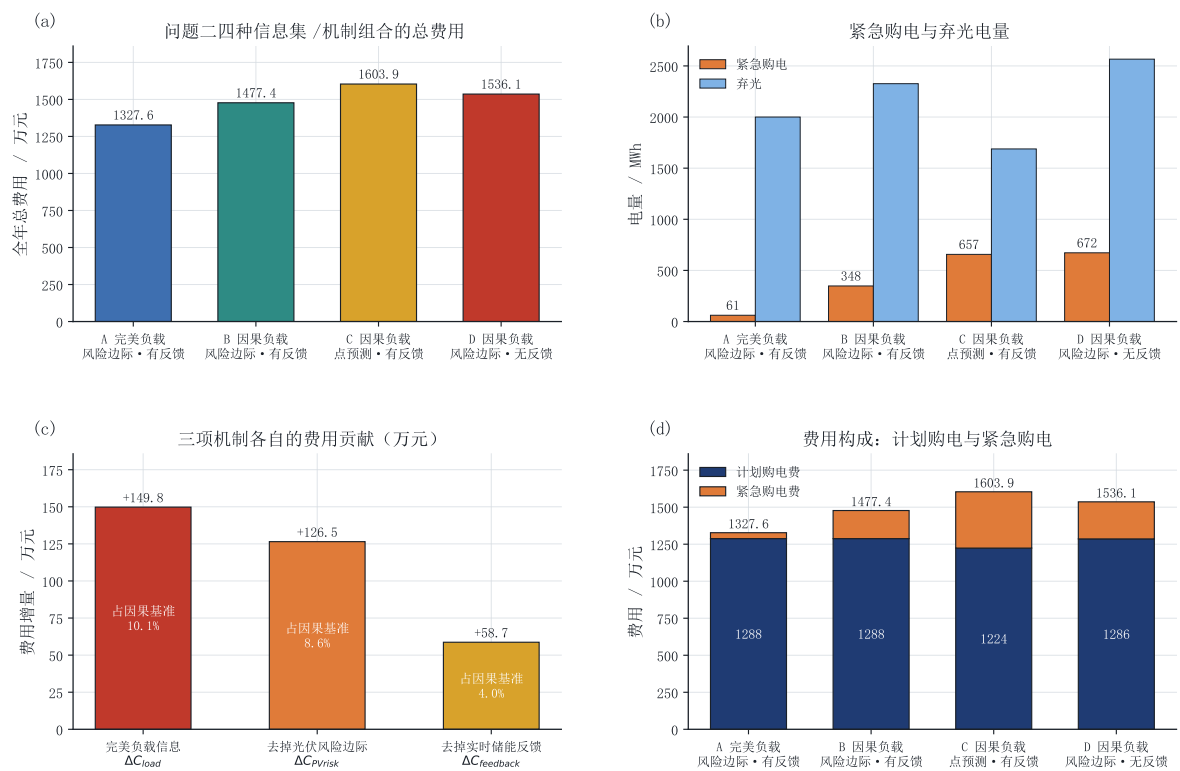


图 4 问题二: 严格历史信息增加计划风险, 风险修正与实时反馈降低高价缺口

表 6 问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	0.00	70508.76	42753.80
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	33673.85	18879.29
2025-09-23	0.00	507.89	0.00	519.64	831.20	5.90	71887.69	45083.22
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92741.53	59280.18

表 7 问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	356.39/185.93	1361.00/4608.26	6269.69/2777.38	2604.97/288.51	112.53/5894.56	9737.25/2860.63	9963.05	9899.29
2025-06-21	2376.47/537.80	1359.38/1738.75	9741.37/0.00	0.00/0.00	0.00/6210.53	158.74/2511.19	1200.00	1252.06
2025-09-23	3.43/2.78	158.81/5967.15	5693.12/1896.62	4388.33/430.85	0.00/5509.97	10293.05/2827.40	10800.00	10800.00
2025-12-21	3646.73/622.21	1638.21/2255.21	5301.47/6488.22	8099.28/2311.29	177.27/5918.77	6323.72/2804.89	6832.77	6833.46

表 8 问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (127.41); 21:40–21:50 (5.16)	132.57	43662.61
2025-06-21	20:30–20:40 (56.42); 21:30–23:40 (555.36)	611.79	20614.91
2025-09-23	20:20–20:30 (6.38)	6.38	45126.21
2025-12-21	7:50–8:00 (19.18); 9:50–10:50 (1391.06); 20:30–20:40 (89.74)	1499.99	66728.16

上述指定日期表把全年费用结论落实到题目要求的具体时段，并保留全天计划、储能状态和紧急购电三类信息。它们与全年统计的关系是：前者用于回答“某一天具体怎么调度”，后者用于判断“这种策略在全年是否值得”。两类结果由同一逐时方案生成，可与 result2.xlsx 逐项核对。

6.4 问题三：多时刻预报下的滚动购电调整

6.4.1 滚动决策结构

问题三新增的能力是：外网购电计划可以随着日内新预报修订；储能实时响应在问题二中已经允许。这里必须区分“当前节点重新规划了哪些未来时段”和“下一节点到来前真正执行哪一段”。0:00 仍先给出覆盖全天的基准计划，后续节点可以重新计算更长的剩余时域，但只执行到下一次预报发布为止。整个过程遵循“看得远、执行近、状态不断链”的滚动原则，与微网能量管理中的滚动时域优化思想一致 [6, 7]。

具体而言，0:00 根据当时预报生成全天计划 G^p 并执行至 6:00；到 6:00 后，以新的光伏预报和实测储电量重新优化尚未执行部分，并执行 6:00–12:00；12:00、18:00 同理。若某个节点在对照实验中被关闭，则继续沿用最近一次已经生效的计划，后续时段沿着该方案执行，直至下一个启用节点到来。这个规则保证不同更新策略之间的唯一差别确实是“是否使用该节点的新信息”。

附件 3 为整点预报，而调度间隔为 10 min，需要先降尺度。为确定插值方法，比较线性插值与阶梯保持在真正会被执行的 6 h 区间上的误差，结果见图 5。

图 5 给出了一个直接结论：线性插值在实际执行的 6 h 区间内 MAE 为 110.22 kW，而阶梯保持为 294.37 kW，因此后续采用线性插值把小时预报映射到 10 min 调度网格。图中还可以看到，不同发布时间的误差并不具有同一分布，这意味着不能用一个全年固定偏差统一校正四个预报节点。为此，0:00、6:00、12:00、18:00 分别维护最近 30 天对应执行区间的历史残差，并沿用问题二的非对称费用代理选择各自的风险边际。于是，每次更新既利用了新的预报中心，也根据该节点过去“偏高还是偏低”的实际表现修正当前决策。

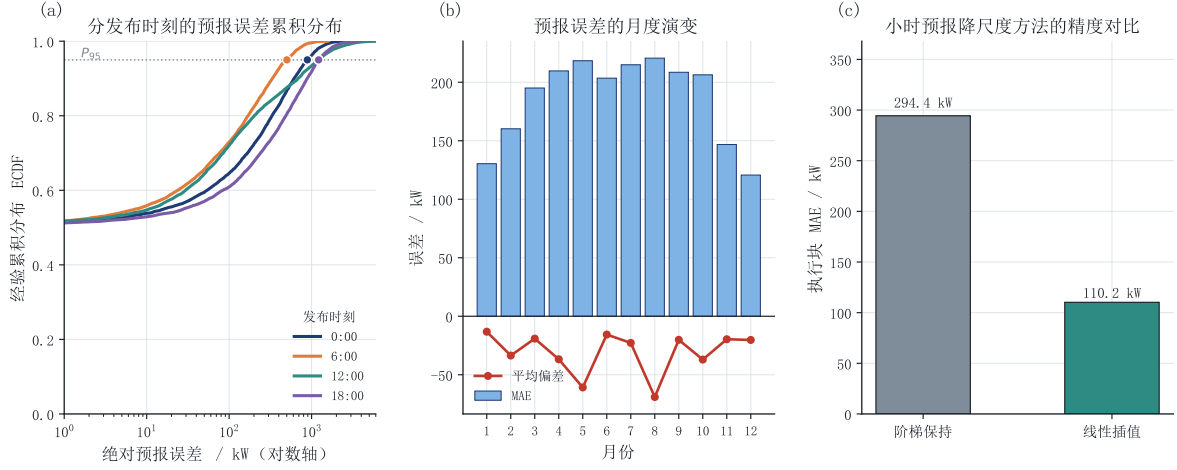


图 5 线性插值较阶梯保持显著降低实际执行区间的预报误差

6.4.2 主结算规则与对照方案

设 G_t^p 为 0:00 基准计划, G_t^a 为该时段执行前最后一次有效调整后的购电量。定义向上调整量 $u_t = (G_t^a - G_t^p)^+$ 、向下调整量 $v_t = (G_t^p - G_t^a)^+$ 。题面对偏差计价使用了“部分”“超出部分”和“违约电价”的表述,即只对计划与调整之间的差额按特殊价格处理,未调整的部分仍按正常电价结算。据此,主模型采用

$$C_t = \pi_t [\min(G_t^p, G_t^a) + 0.5v_t + 1.5u_t]. \quad (18)$$

当 $G_t^a < G_t^p$ 时,式 (18) 化为 $\pi_t(G_t^a + 0.5v_t)$; 当 $G_t^a > G_t^p$ 时,化为 $\pi_t(G_t^p + 1.5u_t)$ 。作为对照,另一种理解把计划购电费与调整费用并列相加:

$$C_t^{\text{sum}} = \pi_t G_t^p + 0.5\pi_t v_t + 1.5\pi_t u_t. \quad (19)$$

两种计费方式只在向下调整时产生分歧。由于结算规则会反过来改变最优购电与储能动作,本文对二者分别从头重新优化,避免把同一条调度曲线套入两套价格公式。

为在线性模型中精确表示调整方向,引入 $u_t, v_t \geq 0$ 并满足

$$G_t^a - G_t^p = u_t - v_t. \quad (20)$$

式 (20) 将调整后的购电量与上下调量直接联系。主模型并采用式 (9) 的充放电互斥约束,防止通过无意义的同时充放电耗散储能状态。

6.4.3 “其他预报是否需要”的对照设计

题目最后要求判断后面三个预报节点是否值得引入。若只把“只用 0 点”和“四节点全用”进行一次总比较,只能证明滚动更新整体有效,却无法知道收益主要来自早晨、

中午还是傍晚。为此，构造逐步嵌套的四组策略：

$$\begin{aligned} S_0 &= \{0:00\}, & S_1 &= \{0:00, 6:00\}, \\ S_2 &= \{0:00, 6:00, 12:00\}, & S_3 &= \{0:00, 6:00, 12:00, 18:00\}. \end{aligned} \quad (21)$$

四组使用相同的预测器、终端策略、储能反馈和结算方式；当某节点关闭时，最近一次有效计划继续执行。于是从 S_{k-1} 到 S_k 的费用下降可以解释为新增该决策节点后的边际收益：

$$V_6 = C_{S_0} - C_{S_1}, \quad V_{12} = C_{S_1} - C_{S_2}, \quad V_{18} = C_{S_2} - C_{S_3}, \quad V_{\text{total}} = C_{S_0} - C_{S_3}. \quad (22)$$

不过，节点更新时不仅获得新的光伏预报，也获得截至当前时刻已经实现的负载信息。故式 (22) 衡量的是整个决策更新节点的综合价值。为单独观察官方光伏预报的贡献，再设置配对控制组 C_k ：允许使用新增负载观测重优化，但暂时冻结光伏预报为上有效节点版本。由此定义

$$V_{\tau}^L = C_{S_{k-1}} - C_{C_k}, \quad V_{\tau}^P = C_{C_k} - C_{S_k}, \quad \tau \in \{6, 12, 18\}. \quad (23)$$

式 (23) 中两项之和等于对应节点的综合价值。需要说明的是，这是一种“先更新负载、再更新光伏”的顺序分解；由于两类信息与储能决策存在交互，若交换顺序，单项贡献可能变化，故这里仅将其视为给定更新顺序下的增量分解。

6.4.4 全年结果与节点价值

表 9 给出四组更新策略的全年费用、紧急购电以及调整电量。该表直接回答“更新一次究竟省了多少钱”，图 6 则观察这些收益在不同季节是否稳定出现。

表 9 问题三：日内更新节点逐步增加时，全年费用与紧急购电持续下降

方案	可用更新节点	总费用/元	增量价值/元	累计价值/元	紧急购电/kWh	调整电量（上/下）/kWh
S_0	0:00	15731164.40	—	0.00	539486.13	0.00/0.00
S_1	+6:00	14505850.38	1225314.02	1225314.02	191492.64	949150.97/476663.37
S_2	+12:00	14353879.08	151971.30	1377285.32	221996.42	707386.98/888581.42
S_3	+18:00	13727998.70	625880.38	2003165.70	78469.06	819067.56/853610.51

表 9 先回答“值不值得更新”：从 S_0 到 S_3 ，每增加一个可用节点，总费用继续下降，紧急购电也总体减少，说明新信息已经实质改变了执行方案。仅看全年合计还无法判断这种收益是否由少数极端月份贡献，因此图 6 再把三个节点的边际价值按季节展开，观察更新价值是否具有稳定的时间分布。

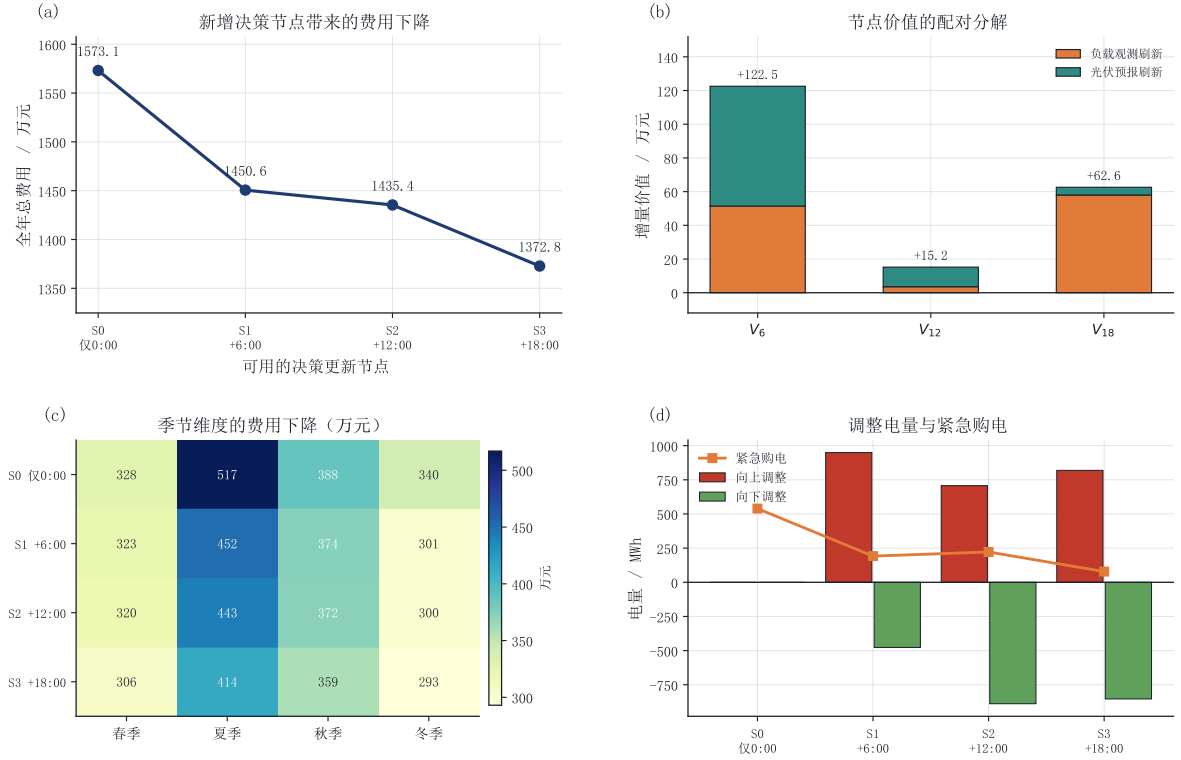


图 6 问题三：6:00 节点价值最大，后续节点仍提供正的边际收益

由表 9 可见，随着日内信息逐步加入，费用从 15731164.40 元依次降至 14505850.38、14353879.08、13727998.70 元，紧急购电也由 539486.13 kWh 大幅下降到 78469.06 kWh。各节点的边际价值为

$$\begin{aligned} V_6 &= 1225314.02 \text{ 元}, & V_{12} &= 151971.30 \text{ 元}, \\ V_{18} &= 625880.38 \text{ 元}, & V_{\text{total}} &= 2003165.70 \text{ 元}. \end{aligned} \quad (24)$$

式 (24) 表明，在题目没有设置预报获取成本和调整操作固定成本的条件下，三个新增节点都带来正收益，应当使用 6:00、12:00、18:00 的更新。其中 6:00 的价值最大，说明越早修正全天计划，能够影响的剩余时段越长；18:00 的综合收益仍为正，但其纯光伏预报配对增量只有 45935.57 元，若实际部署存在固定操作成本，则该节点需要额外进行成本—收益比较。

为避免把上述总收益全部归因于附件 3，表 10 给出“先负载、后光伏”路径下的配对增量分解。

在这一给定顺序下，已实现负载信息贡献 1130133.34 元，新光伏预报贡献 873032.35 元。两者共同说明：日内滚动的价值既来自“预报变新”，也来自“系统状态和已发生负载变得更清楚”。本文将 $S_0 \rightarrow S_3$ 的 2003165.70 元称为决策更新的综合价值，而把 873032.35 元称为该顺序下的光伏刷新增量。

主方案全年总费用可回代为

表 10 问题三：节点收益由已实现负载刷新与新光伏预报共同产生

节点	控制组费用/元	负载刷新贡献/元	光伏刷新贡献/元	两项合计/元
6:00	15216582.00	514582.40	710731.62	1225314.02
12:00	14470244.24	35606.14	116365.17	151971.30
18:00	13773934.27	579944.81	45935.57	625880.38
合计	—	1130133.34	873032.35	2003165.70

$$12638814.57 + 688209.31 + 400974.83 = 13727998.70 \text{ 元}, \quad (25)$$

式 (25) 中三项分别为日初计划购电费、日内调整净额和紧急购电费。相对日初计划费，向下调整中“少购节省扣除违约金”后的净影响为 -281981.80 元，向上增购的费用影响为 970191.10 元。作为对照，另一种计费方式独立重新优化后的 S_3 费用为 13917403.19 元；两种理解的绝对数值不同，但“增加日内决策节点能够降低全年费用”的结论一致。

按题目要求，四个指定日期的计划/调整购电量、储能运行和紧急购电结果列于表 11–13，完整逐时结果见 result3.xlsx。

表 11 问题三指定日期购电结果

日期	类别	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	0.00	69520.52	42031.63
	调整	0.00	537.82	0.00	489.74	716.29	0.00	67158.92	42695.30
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31110.55	17009.78
	调整	0.00	0.00	0.00	75.05	371.70	0.00	32773.16	19510.35
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	5.90	73888.09	46335.87
	调整	0.00	436.86	0.00	481.86	823.63	0.00	68522.54	44621.10
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90602.15	57486.31
	调整	0.00	838.96	0.00	825.73	725.20	0.00	94124.54	61966.73

指定日期结果进一步把“滚动更新”落实为可检查的计划变化：表中一并保留 0:00 基准购电量、最终有效调整量、储能分段动作和紧急购电，从而能够追踪一条全天计划如何在新预报到达后逐段演化。仅凭这些离散时段仍不容易看清源荷与储能之间的连续作用，因此下面选取一个典型日展示完整运行过程。

表 12 问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1968.05/143.94	1278.68/5143.78	5243.62/2777.38	4391.20/272.31	68.72/5875.25	8114.04/2897.33	8450.86	8397.63
2025-06-21	1469.50/642.66	1255.97/1285.96	10322.21/0.00	0.00/0.00	183.90/5772.37	397.48/2685.89	1200.00	1925.18
2025-09-23	3.43/2.78	138.54/6205.13	6293.08/1896.62	4068.83/415.35	119.39/5903.59	10620.55/2829.34	10800.00	10749.67
2025-12-21	3282.85/622.21	1944.62/1464.57	5172.16/7563.28	8130.80/2135.40	47.67/5925.83	6738.41/2804.89	7217.57	7206.67

表 13 问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (127.39); 21:40–21:50 (5.16)	132.55	43604.02
2025-06-21	6:20–7:20 (452.79)	452.79	21420.82
2025-09-23	20:20–20:40 (80.57); 20:50–21:10 (99.53); 21:20–21:50 (62.45)	242.54	45919.52
2025-12-21	7:50–8:00 (29.09); 10:10–10:50 (630.86); 20:30–20:40 (89.74)	749.70	65769.32

6.4.5 典型日运行机理

全年对照回答了“滚动更新是否省钱”，但仍看不出一条具体计划是如何在源荷变化下执行的。为把费用差还原成可观察的运行过程，选取 2025 年 6 月 21 日这一高辐照典型日，图 7 把源荷、日初计划、最终有效计划、储能状态以及弃光/紧急购电放在同一时间轴上。

该日光伏峰值超过 8 MW，而小区负载约为 3.8 MW。图 7(a)(c) 显示，储能在上午快速充电并达到 10.8 MWh 的安全上限；此后即使仍有大量光伏，也无法继续全部吸收，于是图 7(d) 出现超过 4500 kW 的弃光峰值。另一方面，傍晚光伏迅速衰减而负载仍处于较高水平，储能开始持续放电。

图 7(b) 中，0:00 基准计划与最终调整计划总体形态一致，但在傍晚时段出现较明显的上下调。这说明日初预报已经抓住了当天“中午富余、傍晚缺电”的主要结构，后续预报的作用更多是修正剩余量级并重新安排储能释放节奏。该典型日揭示了一个重要限制：即使预测完全准确，储能容量和功率上限仍可能使“正午弃光”和“傍晚高价购电”同时出现。提高预测精度能够减少偏差损失，但不能代替储能容量约束本身。

6.5 问题四：波动电价下的重算

6.5.1 模型迁移

问题四并不需要重新发明一套物理模型。负载、光伏、储能状态以及问题二、三的执行机制全部保留，只把固定典型日电价 π_t 换成附件 4 中的逐日电价 $\pi_{d,t}$ ，再重新优化购电与储能动作。由于附件 4 就是题面指定的重算输入，正式结果直接使用其给定价

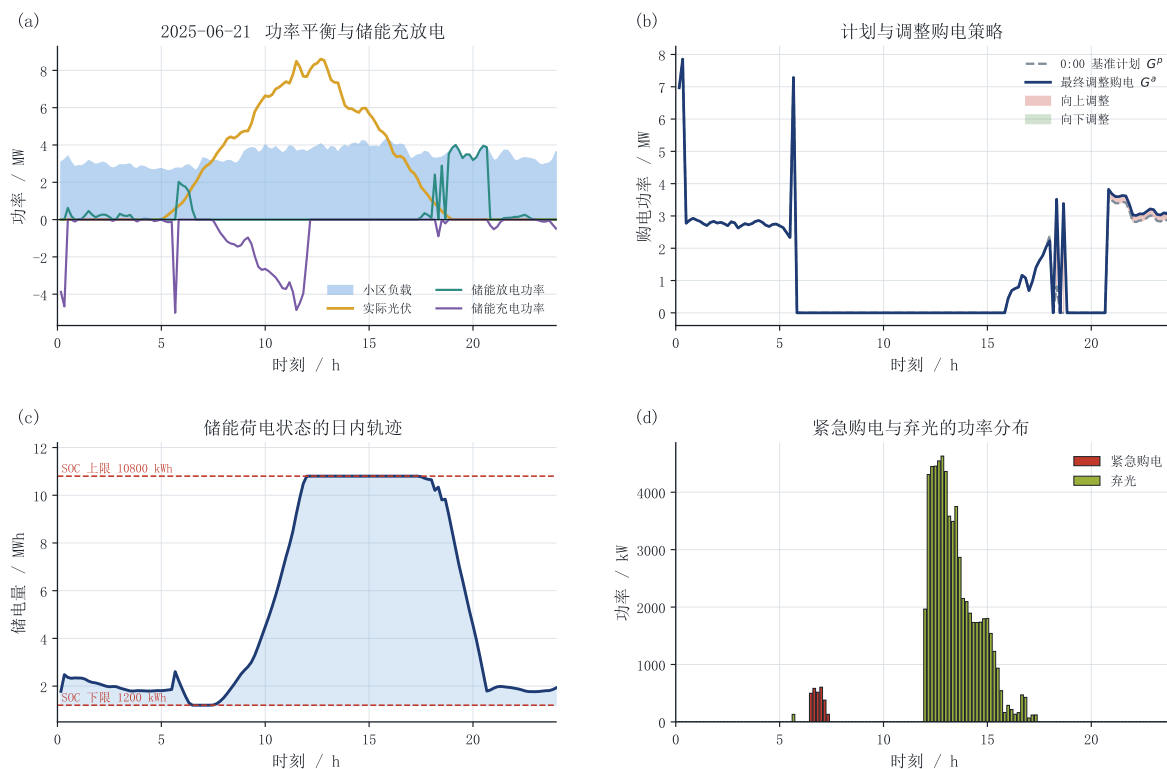


图 7 典型高辐照日中午弃光与傍晚购电并存，储能容量构成主要约束

格；在此之外，再增加“未来价格只能由历史预测”的扩展情形，用于衡量真实市场价格信息受限可能造成的额外成本。

价格变化不会改变电量守恒，却会改变一度电在不同时间的机会价值。谷价更低、峰价更高时，储能更愿意把电量跨时段转移；但这种套利受到三个边界共同限制：电池只能储存有限电量、每个时段的充放电功率有限，而且题目没有设置向外网售电。即使未来价格完全已知，价格信息的价值也不会无限增大。

6.5.2 结果与价格信息价值

为区分“问题二/问题三的决策方式”和“价格是否提前可知”，表 14 给出四组年度结果。

表 14 问题四：波动电价下滚动更新仍降低全年费用，价格预知价值相对有限

方案	价格信息	总费用/元	紧急购电/kWh	弃光/kWh
Q4-2	基准（附件 4）	15523884.83	355425.99	2365390.66
Q4-2	因果预测	15590096.88	354599.54	2322027.45
Q4-3	基准（附件 4）	14387959.47	97333.41	1674534.52
Q4-3	因果预测	14524235.78	95823.19	1666685.67

表 14 给出的四组总费用还不能直接说明“未来价格信息值多少钱”，因为 Q4-2 和 Q4-3 的调度结构本身不同。于是分别在两种结构内部，只比较完整价格与因果价格预测，得到

$$V_{\text{price},Q2} = 66212.05 \text{ 元 (0.42\%)}, \quad V_{\text{price},Q3} = 136276.31 \text{ 元 (0.94\%)}. \quad (26)$$

式 (26) 衡量真实市场中无法提前获得完整价格曲线时增加的费用。两个比例均不足 1%，说明本数据的价格序列具有较稳定的日内结构，历史价格已经能够提供相当有效的预测；另一方面，储能容量、功率上限和禁止售电也限制了即使完全知道价格后还能进一步优化多少。与问题二中负荷信息边界造成的 1497816.65 元费用差相比，价格预知的价值更小，这意味着在本数据条件下，优先改善源荷预测比继续追求更精确的未来价格更有收益。

最后比较波动电价下“只在 0:00 决策”与“允许日内滚动调整”的差异：

$$C_{Q4-2}^{\text{perfect}} - C_{Q4-3}^{\text{perfect}} = 1135925.35 \text{ 元}, \quad C_{Q4-2}^{\text{causal}} - C_{Q4-3}^{\text{causal}} = 1065861.10 \text{ 元}. \quad (27)$$

两项均为正，说明即使电价从固定典型日变为逐日波动，日内更新仍能通过修正未来购电和储能安排降低总费用。因果价格扩展中，0:00、6:00、12:00、18:00 的价格 MAE 分别为 0.053238、0.107166、0.112235、0.064665 元/kWh，误差并不随发布时间单调下降，这也说明价格的可预测性与光伏预报“越临近越准”的机制不同。

题面指定日期的完整重算结果列于表 15–20。

对应的完整结果分别写入文件 result4-2.xlsx 与 result4-3.xlsx。

表 15 问题四对应问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	709.49	70933.40	43543.79
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	34143.61	16105.02
2025-09-23	0.00	530.79	0.00	542.54	831.20	487.35	73783.38	47932.82
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92738.65	69147.78

这些指定日期表说明，问题四需要新的逐时价格下重新决定何时购电、何时充放电以及何时接受调整成本。因而，表中的购电与储能轨迹本身也会随价格曲线改变；这正是问题四需要独立重算而不能沿用固定电价方案的原因。

表 16 问题四对应问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/1399.06	2151.59/5232.95	6269.69/2777.38	2615.35/288.51	138.93/6617.32	9701.81/2172.38	9897.92	9852.79
2025-06-21	1626.39/1304.00	3854.32/1733.02	8935.37/0.00	0.00/0.00	0.00/6210.53	890.45/3117.22	1200.00	1237.25
2025-09-23	945.57/1171.57	771.33/5594.21	5534.21/1896.62	3946.84/407.95	116.98/6713.26	10237.57/1673.93	10800.00	10800.00
2025-12-21	4516.10/563.53	818.23/2242.93	5301.47/6493.23	7698.88/2313.01	579.78/5273.92	6224.28/3480.03	6715.04	6710.31

表 17 问题四对应问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40（113.09）； 21:40–21:50（5.16）	118.25	44311.65
2025-06-21	0:30–0:40（10.28）； 1:10–2:30（532.14）； 3:30–3:40（151.46）； 20:30–20:40（56.42）； 21:30–23:30（527.96）	1278.26	19150.64
2025-09-23	20:20–20:30（6.38）	6.38	47976.90
2025-12-21	7:50–8:00（19.18）； 10:00–10:50（1386.06）； 20:40–20:50（81.48）	1486.71	79176.58

表 18 问题四对应问题三指定日期购电结果

日期	类别	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	709.49	69636.12	42716.40
	调整	0.00	537.82	0.00	489.74	716.29	709.49	67317.87	43190.17
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31580.30	14682.69
	调整	0.00	0.00	0.00	75.05	371.70	0.00	33250.84	16958.76
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	487.35	74177.20	48179.34
	调整	0.00	436.86	0.00	481.86	823.63	374.71	68773.67	46379.59
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90599.28	66885.71
	调整	0.00	838.96	0.00	825.73	725.20	0.00	94415.14	72694.11

表 19 问题四对应问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量（kWh）						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/772.49	2520.12/5234.86	5243.62/2777.38	4412.57/272.31	95.66/6598.00	7825.80/2172.85	8178.13	8164.39
2025-06-21	1626.39/1304.00	2765.24/1733.02	10024.45/0.00	0.00/0.00	218.16/5806.63	1152.74/3286.53	1200.00	1930.30
2025-09-23	945.57/1171.57	623.69/6189.47	5952.05/1896.62	4406.19/415.35	138.37/7135.44	10585.43/1612.86	10800.00	10718.06
2025-12-21	4084.78/609.78	2026.32/2127.30	5172.16/7563.28	7938.66/2298.66	441.37/5280.98	6701.86/3480.03	7144.86	7140.12

表 20 问题四对应问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (112.63); 21:40–21:50 (5.16)	117.78	43954.95
2025-06-21	0:30–0:40 (10.28); 1:10–2:30 (532.14); 3:30–3:40 (151.46)	693.88	18424.25
2025-09-23	18:20–18:40 (3.91); 20:20–20:40 (80.26); 20:50–21:10 (99.53); 21:20–21:50 (62.45)	246.15	47748.48
2025-12-21	7:50–8:00 (29.09); 10:20–10:50 (630.86); 20:40–20:50 (81.48)	741.43	77667.39

七、模型检验与稳健性分析

前述模型连续运行数百天，并在一天内多次更新决策。对于这类时序优化，求解器给出“最优解”只说明数学程序完成了优化，并不能自动保证方案满足真实运行逻辑。一个可信的调度结果至少应满足三点：每个时段的电量能够守恒，储能状态能够连续传递，且任一决策只能使用当时已经获得的信息。基于这一原则，下面不再重复模型求解过程，而是从物理轨迹、时间因果、结算逻辑和关键参数四个方面对最终结果进行独立复核。

7.1 物理可行性与数值一致性

首先把最终输出逐时重新代入能量平衡式和储能状态递推式。若这一层不能通过，即使全年费用看起来合理，也可能只是单位换算、储能递推或约束遗漏造成的“数值低成本”，并检查储电量是否始终落在允许区间、主方案是否出现同时充放电，以及由逐时交易重新汇总出的费用能否回到论文报告值。结果列于表 21。

表 21 物理残差与费用回代误差均低于设定校验阈值

检验项目	最大偏差或范围	判定标准	结论
能量平衡残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量递推残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量取值范围	[1200, 10800] kWh	题设区间	通过
主方案充放电重叠量	$< 10^{-8}$ kWh	10^{-5} kWh	通过
问题三费用回代差	$< 10^{-8}$ 元	10^{-7} 元	通过
结果工作簿回读差异	$< 10^{-10}$	数值一致	通过

表 21 中两类状态方程的残差均远低于数值容差，表中也给出求解结果的误差量级与费用回代精度。说明每一度电在模型中都有明确来源和去向，储能也没有通过越界或异常耗散获得额外收益。加入充放电互斥约束后，充、放电重叠量只剩浮点舍入量级。

按逐时计划购电、调整交易和紧急购电重新结算，所得总费用与目标函数一致；五个结果工作簿重新读入后，也与求解明细逐项对应。后文讨论的费用差异可以解释为信息和策略差异，基本物理约束与文件写入均已通过独立校验。

7.2 时间因果性的反事实扰动检验

物理可行仍不足以证明计划真实可执行。最容易被忽略的风险是“未来信息穿越”：如果 0:00 的全天计划会随着当天尚未发生的真实光伏或负载而变化，那么模型实际上提前知道了未来，即使费用更低也失去实际意义。为直接检查这一点，本文采用反事实扰动：固定某个决策节点之前的全部历史信息，只改变节点之后才会实现的数据，再观察当前计划是否随之改变。

在问题二中，分别修改当天未来真实负荷与真实光伏，0:00 计划保持不变；在问题三中，改变 6:00 之后的实际源荷，不会反向改变 6:00 已经形成的计划；在问题四的因果价格扩展中，未来实际电价的扰动同样不能影响扰动发生前的决策。所有非预见性检验的最大计划差异均低于 10^{-10} 。作为对照，在直接使用附件 4 完整价格的题面主情形中，改变未来价格会相应改变计划，说明前述“不变”确实来自信息隔离；对照情形也证明扰动本身能够影响计划。

滚动模型还需要保证计划能够正确延续。仅启用 0:00 和 6:00 两个更新节点时，12:00 以后应继续沿用 6:00 已经生效的未来方案，而不能因为 12:00 节点被关闭就退回日初计划。逐段追踪表明，关闭节点后最近一次有效方案得到保留，且每个执行块都从上一阶段的实测储电量继续递推。由此，问题三各更新节点之间的费用差才真正对应新增信息，而不会混入错误的计划回退效应。

7.3 调整结算与滚动计划的一致性检验

问题三中，调整费用会改变模型愿意上调还是下调购电，因此费用公式必须和优化过程保持一致。先用单时段手算核对主模型与对照方案。设正常电价为 p ，当计划购电量 $G^p = 5000 \text{ kWh}$ 、执行前调整为 $G^a = 4000 \text{ kWh}$ 时，主模型按“未调整部分正常结算、少购部分支付违约费用”计算为 $4000p + 0.5 \times 1000p = 4500p$ ；对照方案把原计划费与违约费用并列相加，得到 $5000p + 0.5 \times 1000p = 5500p$ 。当计划由 4000 kWh 上调至 5000 kWh 时，两种计费方式均为 $4000p + 1.5 \times 1000p = 5500p$ 。两种计费方式只在实际购电量低于计划时产生分歧。对照方案下，少买 1000 kWh 会比不调整多付 $500p$ ，从而抑制向下修正；主模型下少买能够降低支出，但要为少买部分支付 50% 的违约费用，这与题面“违约电价”的措辞和经济含义一致。

不同结算规则会反过来改变最优调度。两种计费方式均从计划层重新求解，避免在同一条购电和储能轨迹上事后更换计价公式。随机源荷与价格序列上的交叉复算显示，每种计费方式下重新优化得到的方案，都不会被另一种方式的最优轨迹直接替代而获得

更低费用。这说明费用规则已经真正进入决策。

对于多次滚动更新，同一未来时段可能先后出现在 0:00、6:00、12:00 和 18:00 的规划中，但实际执行时只采用交易前最后一次生效的方案。该处理与前述计划持续性检验共同保证：模型可以反复“看未来”，但不会重复执行多份计划，也不会因节点缺失时凭空改变既有安排。

7.4 历史配置的复算一致性

为确认正文调整未改变既有计算链，本文保留历史配置进行独立复算，并逐项比较旧计算与当前程序在相同参数下的输出。表 22 只报告一致性结论，不在正文重复列出旧版本费用数值，以避免历史结果与当前主方案混淆。

表 22 相同参数下五组历史配置均可复算到一致结果

复算对象	旧计算与当前复算	最大绝对偏差/元
问题一	一致	$< 10^{-3}$
问题二	一致	$< 10^{-3}$
问题三	一致	$< 10^{-3}$
问题四-2	一致	$< 10^{-3}$
问题四-3	一致	$< 10^{-3}$

该复算说明，新增的信息条件、计费方式与储能约束并未破坏既有配置下的计算行为；历史结果可作为信息更充分或规则不同条件下的基准，而不与当前主方案混合比较。

7.5 终端储电量策略的敏感性

问题一明确要求日初与日末储电量相同，而问题二、三没有给出同样的硬约束。如果完全取消终端要求，单日优化会倾向于在午夜前尽量放空电池，把本应留给下一日的储能价值转化为当前日的低费用。这种做法在单日目标函数上看似更优，却可能只是把成本推迟到下一日。为判断日循环假设是否过强，本文保留无终端约束方案作为对照。

两种终端策略的费用与储电量变化如图 8 所示。日循环策略全年费用为 14773804.78 元；取消终端约束后费用降至 14728422.40 元，变化幅度仅为 0.307%，但全年最终储电量下降到 1235.46 kWh，日末平均储电量也只有 1660 kWh，已经接近 1200 kWh 的安全下限。

图 8 说明，无终端约束获得的少量费用优势主要来自持续压低日末储电量——日末状态大量集中在下限附近，并不意味着光伏或电价利用效率真正提高。其日末 SOC 标准差虽然为 896 kWh，小于日循环策略的 3798 kWh，但这是因为大量状态集中在下限

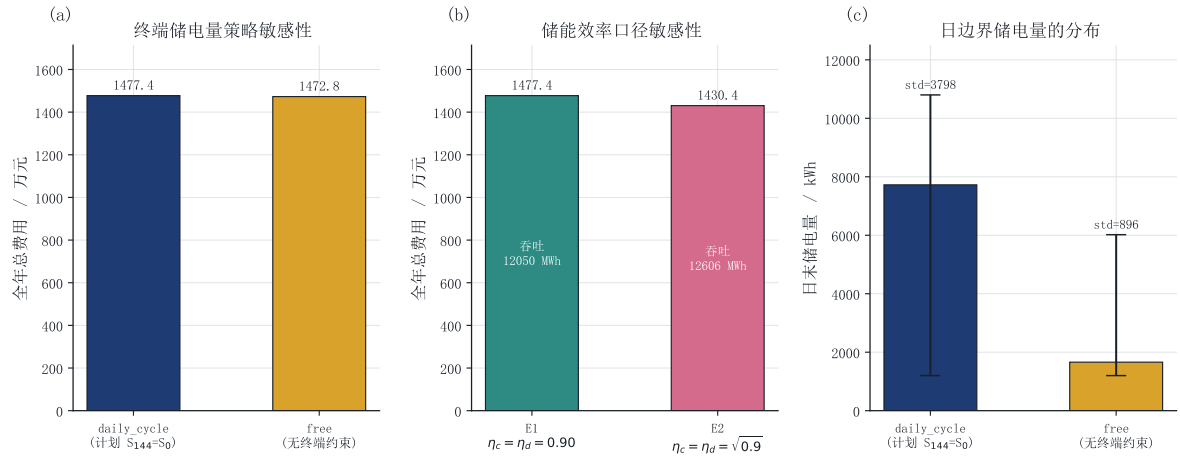


图 8 取消终端约束仅小幅降费，却使日末储电量逼近安全下限

附近，不能解释为运行更稳健。考虑全年连续运行时下一日仍需要可比的储能起点，本文保留日循环作为主策略，并把无终端约束结果用于说明该假设对总费用的影响有限。

7.6 储能效率取值的敏感性

题目给出的“充放电效率为 90%”可以有两种理解：其一是充电和放电效率分别为 0.9，对应往返效率 0.81；其二是整个充放电过程的往返效率为 0.9，若两侧对称，则可取 $\eta_c = \eta_d = \sqrt{0.9}$ 。由于这一解释会直接影响每次跨时段搬移电量的损耗，有必要在保持其他设置不变的条件下重新求解。结果见表 23。

表 23 提高往返效率会降低全年费用，但不改变储能与滚动更新的作用方向

效率定义	全年总费用/元	储能吞吐量/kWh	紧急购电量/kWh
定义 A: $\eta_c = \eta_d = 0.90$ (往返 0.81)	14773804.78	12049920.76	348036.96
定义 B: $\eta_c = \eta_d = \sqrt{0.9}$ (往返 0.90)	14304371.70	12606462.13	349968.83

提高往返效率后，单位充入电量可以在后续释放更多有效电能，储能因而更愿意承担峰谷转移和偏差缓冲，全年费用随之下降。但两种效率定义下，储能的作用方向、风险购电的必要性以及滚动更新的相对优势均未改变。也就是说，效率解释会影响费用的绝对数值，却不会推翻本文关于“储能降低高价购电、更新信息降低偏差成本”的主要判断。主模型取 $\eta_c = \eta_d = 0.9$ ，相当于采用较保守的效率解释。

7.7 误差来源与适用边界

模型剩余误差主要来自三类信息近似。其一，问题二的负载和光伏只根据历史序列预测，突发天气、节假日或异常用电会使近期历史失去代表性；其二，附件 3 只给出整

点预报，将其插值到 10 min 能恢复调度时间尺度，却不能生成原预报中不存在的短时波动；其三，问题四主结果按题意直接使用附件 4 完整价格，而真实市场中未来价格的发布方式可能更受限制。本文分别通过滚动误差更新、储能实时反馈、节点对照和因果价格扩展量化了这些影响。

本文结果首先回答的是题目给定数据和交易规则下的调度问题。若向真实长期运行推广，还应引入天气预报、工作日特征、需求侧响应、储能衰减和价格发布机制等外生信息。上述限制不会破坏当前结果的内部一致性，但决定了模型从竞赛数据走向实际系统时还需要哪些新增信息与设备模型。

八、模型评价与推广

8.1 模型评价

本文在同一套微网物理约束上逐步增加信息和交易规则，使四问的费用变化能够追溯到信息边界、结算方式和价格条件。统一物理内核也使各问可以共享能量与状态校验，减少因重复建模带来的定义漂移。

统一结构带来三方面优势。其一，调度结果能够直接还原为低价充电、高价放电、光伏先存后用和滚动修正等可解释动作；其二，计划信息与执行信息严格分离，并由反事实扰动检验非预见性，使费用结果对应现实时间顺序下可实施的策略；其三，线性/混合整数规划允许逐时回代能量、状态和费用，便于发现单位错误、状态越界和充放电重叠退化。

需要看到的是，模型的“严格可执行”仍建立在若干简化之上。问题二、三采用日循环终端策略，以抑制午夜前透支电池的有限时域效应，但日循环约束也限制了跨日价格和光伏差异的主动利用；历史预测器只使用已有序列，没有气象、温度、节假日等外生特征，对极端天气和异常负荷的响应有限；光伏风险边际目前按预测器使用统一残差分位，尚未刻画不同季节和日内时刻的异方差；储能也未计入自放电、容量衰减和循环寿命成本。因而，本文给出的全年费用应理解为既定储能参数和题面交易规则下的运行费用，尚未包含完整资产寿命成本。

8.2 模型改进

模型改进应优先针对上述简化。第一，可把计划时域由单日扩展到 3~7 天，并只执行首日方案，在窗口末端加入储能残值，使模型能够利用跨日差异，并避免最后一天放空电池。第二，可引入数值天气预报、辐照度、温度、工作日等特征，建立负载与光伏的联合概率场景，再通过情景抽样或 CVaR 直接优化跨时段风险，使问题二中的单时段分位数启发转化为完整的随机调度。第三，可按季节和日内时刻分别估计光伏残差分布，使安全边际随实际预测不确定性变化，避免全天采用同一绝对修正量。第四，可加

入电池循环衰减、自放电、功率相关效率和维护费用，在“少买电”和“延长储能寿命”之间建立长期经济权衡。第五，当前费用函数未计入获取日内新预报、修改交易计划所需的固定操作成本，也未考虑需求侧响应；若面向真实系统，应把这些成本与可调负荷一并纳入。

8.3 模型推广

本文“公共物理约束—信息分层—滚动修正”的结构具有一定的可迁移性。在园区综合能源系统中，可把单一电力平衡扩展为电、热、气多能流平衡，并增加储热、储气状态 [14]；在电动汽车充电站中，可把居民负载替换为车辆到站后的充电需求，再加入离站时间和最低荷电状态约束；在工业用户侧储能中，则可以在现有目标函数中加入最大需量电费和生产负荷约束。更一般地，只要系统具有“先做计划、后观察偏差、状态逐期传递、未来计划允许被新信息修正”的特征，例如库存补货、水库调度和供应链运输，都可以沿用本文的信息边界与滚动决策框架，再根据具体业务替换物理状态和费用规则。

AI 工具使用声明

本参赛队使用了 MathModel 内置 AI 助手（OpenAI GPT-5），用于审阅既有模型、协助修改 Python 求解与测试代码、分析报错和数值矛盾、生成绘图与 LaTeX 排版草稿，以及语言校订。团队向工具提供赛题、既有工程和阶段性审阅意见，逐项要求修正结算口径、滚动计划持续性、指定日期表格及提交材料。所有采纳内容均由人工复核；数值结论由本地脚本实际运行，约束与费用逐时回代，并经自动测试和 Excel 回读核验。详细提示过程、采纳范围与人工核验见支撑材料 AI 工具使用详情.pdf。

参考文献

- [1] Wu Hongbin, Liu Xingyue, Ding Ming. Dynamic economic dispatch of a microgrid: mathematical models and solution algorithm[J/OL]. International Journal of Electrical Power & Energy Systems, 2014, 63: 336-346. <http://dx.doi.org/10.1016/j.ijepes.2014.06.002>.
- [2] Nemati M, Braun M, Tenbohlen S. Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming[J/OL]. Applied Energy, 2018, 210: 944-963. <http://dx.doi.org/10.1016/j.apenergy.2017.07.007>.
- [3] Wu Zhi, Zeng Pingliang, Zhang X-P, et al. A solution to the chance-constrained two-stage stochastic program for unit commitment with wind energy integration[J/OL]. IEEE Transactions on Power Systems, 2016, 31 (6): 4185-4196. <http://dx.doi.org/10.1109/tpwrs.2015.2513395>.
- [4] 孙硕, 杨仕友. 考虑光伏出力不确定性的分布鲁棒优化调度方法[J/OL]. 激光与光电子学进展, 2025, 62 (9): 0925001. <http://dx.doi.org/10.3788/lop241935>.
- [5] Zhang Yan, Fu Lijun, Zhu Wanlu, et al. Robust model predictive control for optimal energy management of island microgrids with uncertainties[J/OL]. Energy, 2018, 164: 1229-1241. <http://dx.doi.org/10.1016/j.energy.2018.08.200>.

- [6] Elkazaz M, Sumner M, Thomas D. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J/OL]. *International Journal of Electrical Power & Energy Systems*, 2020, 115: 105483. <http://dx.doi.org/10.1016/j.ijepes.2019.105483>.
- [7] Silva V A, Aoki A R, Lambert-Torres G. Optimal day-ahead scheduling of microgrids with battery energy storage system[J/OL]. *Energies*, 2020, 13 (19): 5188. <http://dx.doi.org/10.3390/en13195188>.
- [8] Luo Liang, Abdulkareem S S, Rezvani A, et al. Optimal scheduling of a renewable based microgrid considering photovoltaic system and battery energy storage under uncertainty[J/OL]. *Journal of Energy Storage*, 2020, 28: 101306. <http://dx.doi.org/10.1016/j.est.2020.101306>.
- [9] Maheshwari A, Paterakis N G, Santarelli M, et al. Optimizing the operation of energy storage using a non-linear lithium-ion battery degradation model[J/OL]. *Applied Energy*, 2020, 261: 114360. <http://dx.doi.org/10.1016/j.apenergy.2019.114360>.
- [10] Qin Yan, Wang Ruoxuan, Vakharia A J, et al. The newsvendor problem: review and directions for future research[J/OL]. *European Journal of Operational Research*, 2011, 213 (2): 361-374. <http://dx.doi.org/10.1016/j.ejor.2010.11.024>.
- [11] Polat E, Güler M G, Ulukuş M Y. Renewable GenCo bidding strategy using newsvendor-based neural networks: an example from turkish electricity market[J/OL]. *Electric Power Systems Research*, 2024, 231: 110301. <http://dx.doi.org/10.1016/j.epsr.2024.110301>.
- [12] Markovics D, Mayer M J. Comparison of machine learning methods for photovoltaic power forecasting based on numerical weather prediction[J/OL]. *Renewable and Sustainable Energy Reviews*, 2022, 161: 112364. <http://dx.doi.org/10.1016/j.rser.2022.112364>.
- [13] Wang Qi, Zhang Chunyu, Ding Yi, et al. Review of real-time electricity markets for integrating distributed energy resources and demand response[J/OL]. *Applied Energy*, 2015, 138: 695-706. <http://dx.doi.org/10.1016/j.apenergy.2014.10.048>.
- [14] 刘继春, 周春燕, 高红均, et al. 考虑氢能-天然气混合储能的气-电综合能源微网日前经济调度优化[J/OL]. *电网技术*, 2018, 42 (1): 170-178. <https://doi.org/10.13335/j.1000-3673.pst.2017.0966>.

附录

附录 A 支撑材料文件列表

表 24 支撑材料文件清单

文件	说明
README.md	支撑材料目录、运行环境与唯一复现命令
__init__.py	Python 包标记, 保证从项目根目录稳定导入
AI 工具使用详情.pdf	AI 工具名称、版本、使用环节与人工核验说明
results/	题面要求的五个最终结果工作簿, 文件名与附件 5 一致
solve/	求解、报告生成、结果验证、制表与绘图 Python 源码
tests/	信息集因果性、计划持续性、结算、主结果和历史配置回归测试
tools/	代码同步、LaTeX 检查、论文编译、支撑材料打包与交付脚本
paper/	LaTeX 源码、参考文献、数值宏、矢量图、附录代码副本及编译后的论文
reports/	MODEL_AUDIT.md、model_audit_raw.json 与文献核验记录

附录 B 源程序代码

全部代码使用 Python 3.12 编写, 线性规划和混合整数线性规划均由 SciPy 调用 HiGHS 求解。项目根目录的唯一测试命令为 `python -m pytest MathModel/tests -q --import-mode=importlib`; 各问可用 `python -m MathModel.solve.< 模块名 >` 复现。以下按调用关系列出建模求解、验证与绘图的核心源码。为压缩篇幅, 仅论文代码副本删除注释和文档字符串, 本地源码不变。审计报告生成脚本 `report.py` 位于支撑材料的 `solve/` 目录; 编译、检查和打包脚本位于 `tools/` 目录, 均不改变模型求解逻辑。

B.1 物理参数与口径定义 (`params.py`)

Listing 1:

```
1 from __future__ import annotations
2 import math
3 from dataclasses import dataclass
4 from .io_data import DT_HOURS, SLOTS_PER_DAY
5 POWER_LIMIT_KW = 5000.0
6 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS
7 @dataclass(frozen=True)
8 class StorageParams:
9     eta_ch: float = 0.9
10     eta_dis: float = 0.9
11     soc_min: float = 1200.0
```

```

12     soc_max: float = 10800.0
13     power_limit_kw: float = POWER_LIMIT_KW
14     @property
15     def energy_limit(self) -> float:
16         return self.power_limit_kw * DT_HOURS
17     @property
18     def round_trip(self) -> float:
19         return self.eta_ch * self.eta_dis
20     def as_dict(self) -> dict:
21         return {
22             "eta_ch": self.eta_ch,
23             "eta_dis": self.eta_dis,
24             "round_trip_efficiency": self.round_trip,
25             "soc_min": self.soc_min,
26             "soc_max": self.soc_max,
27             "power_limit_kw": self.power_limit_kw,
28             "energy_limit_per_slot_kwh": self.energy_limit,
29         }
30     DEFAULT_STORAGE = StorageParams()
31     SQRT_HALF = math.sqrt(0.9)
32     EFFICIENCY_VARIANTS: dict[str, StorageParams] = {
33         "E1_eta_ch_0.90_eta_dis_0.90": StorageParams(0.9, 0.9),
34         "E2_round_trip_0.90": StorageParams(SQRT_HALF, SQRT_HALF),
35     }
36     TERMINAL_POLICIES = ("daily_cycle", "free")
37     LOAD_MODES = ("perfect", "causal")
38     PRICE_MODES = ("perfect", "causal")
39     PV_RISK_MODES = ("asymmetric", "point")
40     SETTLEMENT_MODES = ("replacement", "additive")

```

B.2 数据读取与时间对齐 (io_data.py)

Listing 2:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  from datetime import date, datetime
4  from pathlib import Path
5  import numpy as np
6  import openpyxl
7  REPO_ROOT=Path(__file__).resolve().parents[2]
8  DEFAULT_DATA_ROOT=REPO_ROOT/"CUMCM2026Problems"/"C题"/"附件"
9  DT_HOURS=1.0/6.0; SLOTS_PER_DAY=144
10 @dataclass(frozen=True)
11 class DayData:
12     labels:list[str]; price:np.ndarray; load_kw:np.ndarray; pv_kw:np.ndarray

```

```

13 @dataclass(frozen=True)
14 class AnnualData:
15     dates:list[date]; load_kw:np.ndarray; pv_kw:np.ndarray
16 @dataclass(frozen=True)
17 class RollingForecastData:
18     dates:list[date]; hourly_kw:np.ndarray
19 @dataclass(frozen=True)
20 class AnnualPriceData:
21     dates:list[date]; price:np.ndarray
22 def _as_float_array(values:list[object],name:str)->np.ndarray:
23     array=np.asarray(values, dtype=float)
24     if array.shape!=(144,) or not np.isfinite(array).all(): raise ValueError(f"{name}
        invalid: {array.shape}")
25     return array
26 def _coerce_date(value:object)->date:
27     if isinstance(value,datetime): return value.date()
28     if isinstance(value,date): return value
29     text=str(value).strip()
30     for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%m/%d/%Y"):
31         try: return datetime.strptime(text,fmt).date()
32         except ValueError: pass
33     raise ValueError(f"无法解析日期: {value!r}")
34 def read_attachment1(data_root:Path=DEFAULT_DATA_ROOT)->DayData:
35     wb=openpyxl.load_workbook(Path(data_root)/"附件1.xlsx",data_only=True,read_only=True)
36     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=145,min_col=1,max_col=4,
        values_only=True))
37     labels=[str(r[0]) for r in rows]; price=_as_float_array([r[1] for r in rows],"price")
38     load=_as_float_array([r[2] for r in rows],"load"); pv=_as_float_array([r[3] for r in
        rows],"pv")
39     if (price<=0).any() or (load<0).any() or (pv<0).any(): raise ValueError("attachment1
        bounds")
40     return DayData(labels,price,load,pv)
41 def read_attachment2(data_root:Path=DEFAULT_DATA_ROOT)->AnnualData:
42     wb=openpyxl.load_workbook(Path(data_root)/"附件2.xlsx",data_only=True,read_only=True)
43     arrays=[]; date_lists=[]
44     for sheet_name in wb.sheetnames[:2]:
45         rows=list(wb[sheet_name].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
        values_only=True))
46         date_lists.append([_coerce_date(r[0]) for r in rows]); values=np.asarray([r[1:] for
        r in rows], dtype=float)
47         if values.shape!=(365,144) or not np.isfinite(values).all() or (values<0).any():
            raise ValueError(f"{sheet_name} invalid")
48         arrays.append(values)
49     if date_lists[0]!=date_lists[1]: raise ValueError("attachment2 date mismatch")
50     return AnnualData(date_lists[0],arrays[0],arrays[1])
51 def read_attachment3(data_root:Path=DEFAULT_DATA_ROOT)->RollingForecastData:
52     wb=openpyxl.load_workbook(Path(data_root)/"附件3.xlsx",data_only=True,read_only=True)

```

```

53 rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=1461,min_col=1,max_col=26,
        values_only=True))
54 if len(rows)!=1460: raise ValueError("attachment3 must have 1460 forecast rows")
55 dates=[]; blocks=[]; current_date=None
56 expected=("0:00","6:00","12:00","18:00")
57 for day in range(365):
58     chunk=rows[day*4:(day+1)*4]
59     current_date=_coerce_date(chunk[0][0])
60     dates.append(current_date)
61     issues=tuple(str(r[1]) for r in chunk)
62     if issues!=expected: raise ValueError(f"attachment3 issue times invalid on {
        current_date}: {issues}")
63     values=np.asarray([r[2:] for r in chunk],dtype=float)
64     if values.shape!=(4,24) or not np.isfinite(values).all() or (values<0).any(): raise
        ValueError(f"attachment3 invalid {current_date}")
65     blocks.append(values)
66     return RollingForecastData(dates,np.stack(blocks))
67 def hourly_to_ten_min(hourly_kw:np.ndarray,method:str="step")->np.ndarray:
68     hourly=np.asarray(hourly_kw,dtype=float)
69     if hourly.shape!=(24,): raise ValueError("hourly forecast must have 24 values")
70     if method=="step": return np.repeat(hourly,6)
71     if method=="linear":
72         return np.interp(np.arange(1,145)/6.0,np.arange(1,25),hourly,left=hourly[0],right=
            hourly[-1])
73     raise ValueError(f"unknown downscale method: {method}")
74 def read_attachment4(data_root:Path=DEFAULT_DATA_ROOT)->AnnualPriceData:
75     wb=openpyxl.load_workbook(Path(data_root)/"附件4.xlsx",data_only=True,read_only=True)
76     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
        values_only=True))
77     dates=[_coerce_date(r[0]) for r in rows]; price=np.asarray([r[1:] for r in rows],dtype=
        float)
78     if price.shape!=(365,144) or not np.isfinite(price).all() or (price<=0).any(): raise
        ValueError("attachment4 invalid")
79     return AnnualPriceData(dates,price)
80 def template_path(name:str,data_root:Path=DEFAULT_DATA_ROOT)->Path:
81     path=Path(data_root)/"附件5"/name
82     if not path.exists(): raise FileNotFoundError(path)
83     return path

```

B.3 公共线性规划内核 (lp_core.py)

Listing 3:

```

1 from __future__ import annotations
2 from dataclasses import dataclass
3 from time import perf_counter

```

```

4 import numpy as np
5 from scipy.optimize import Bounds, LinearConstraint, linprog, milp
6 from .io_data import DT_HOURS
7 from .params import DEFAULT_STORAGE, ENERGY_LIMIT, StorageParams
8 ETA_CH = DEFAULT_STORAGE.eta_ch
9 ETA_DIS = DEFAULT_STORAGE.eta_dis
10 SOC_MIN = DEFAULT_STORAGE.soc_min
11 SOC_MAX = DEFAULT_STORAGE.soc_max
12 POWER_LIMIT_KW = DEFAULT_STORAGE.power_limit_kw
13 @dataclass
14 class DispatchResult:
15     grid: np.ndarray
16     charge: np.ndarray
17     discharge: np.ndarray
18     soc: np.ndarray
19     curtail: np.ndarray
20     objective: float
21     primary_objective: float
22     curtail_total: float
23     throughput_total: float
24     status: int
25     message: str
26     nit: int
27     solve_seconds: float
28     equality_marginals: np.ndarray
29 def _slices(t: int):
30     return slice(0, t), slice(t, 2 * t), slice(2 * t, 3 * t), slice(3 * t, 4 * t), slice(4 *
        t, 5 * t)
31 def _solve(c, a_eq, b_eq, bounds, a_ub=None, b_ub=None):
32     return linprog(
33         c,
34         A_ub=a_ub,
35         b_ub=b_ub,
36         A_eq=a_eq,
37         b_eq=b_eq,
38         bounds=bounds,
39         method="highs",
40         options={"primal_feasibility_tolerance": 1e-9, "dual_feasibility_tolerance": 1e-9},
41     )
42 def _solve_milp(c, a_eq, b_eq, bounds, integrality, a_ub=None, b_ub=None):
43     lower = np.asarray([( -np.inf if lo is None else lo) for lo, _ in bounds], dtype=float)
44     upper = np.asarray([( np.inf if hi is None else hi) for _, hi in bounds], dtype=float)
45     constraints = [LinearConstraint(np.asarray(a_eq), np.asarray(b_eq), np.asarray(b_eq))]
46     if a_ub is not None and len(a_ub):
47         constraints.append(
48             LinearConstraint(
49                 np.asarray(a_ub, dtype=float),

```

```

50         np.full(len(b_ub), -np.inf, dtype=float),
51         np.asarray(b_ub, dtype=float),
52     )
53 )
54 return milp(
55     np.asarray(c, dtype=float),
56     integrality=np.asarray(integrality, dtype=int),
57     bounds=Bounds(lower, upper),
58     constraints=constraints,
59     options={"presolve": True, "mip_rel_gap": 1e-10},
60 )
61 SLACK_ABS = 1e-5
62 SLACK_REL = 1e-10
63 def _lexicographic_slack(value: float, factor: float = 1.0) -> float:
64     return factor * max(SLACK_ABS, SLACK_REL * max(1.0, abs(value)))
65 def _lexicographic(objectives, a_eq, b_eq, bounds, extra_rows=None, extra_rhs=None,
66     slack_factor=1.0):
67     base_rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
68     base_rhs = [float(v) for v in (extra_rhs or [])]
69     rows = list(base_rows)
70     rhs = list(base_rhs)
71     x = None
72     result = None
73     degraded = False
74     values: list[float] = []
75     for index, c in enumerate(objectives):
76         solution = None
77         for factor, keep in ((1.0, None), (100.0, None), (1.0, 1)):
78             if keep is None:
79                 sel_rows, sel_rhs = rows, rhs
80             else:
81                 sel_rows, sel_rhs = rows[-keep:], rhs[-keep:]
82             a_ub = np.asarray(sel_rows) if sel_rows else None
83             b_ub = np.asarray(sel_rhs) if sel_rhs else None
84             solution = _solve(c, a_eq, b_eq, bounds, a_ub, b_ub)
85             if solution.success:
86                 if factor != 1.0 or (keep is not None and len(rows) > keep):
87                     degraded = True
88                     break
89             if solution is None or not solution.success:
90                 if x is None:
91                     raise RuntimeError(f"lexicographic LP stage {index} failed: {solution.
92                         message}")
93                 degraded = True
94                 break
95     x = solution.x
96     result = solution

```

```

95         value = float(solution.fun)
96         values.append(value)
97         rows.append(np.asarray(c, dtype=float))
98         rhs.append(value + _lexicographic_slack(value, slack_factor))
99     return x, result, degraded, values
100 def _milp_lexicographic(objectives, a_eq, b_eq, bounds, integrality, extra_rows=None,
101                          extra_rhs=None):
102     rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
103     rhs = [float(v) for v in (extra_rhs or [])]
104     values: list[float] = []
105     result = None
106     for index, objective in enumerate(objectives):
107         result = _solve_milp(
108             objective,
109             a_eq,
110             b_eq,
111             bounds,
112             integrality,
113             np.asarray(rows) if rows else None,
114             np.asarray(rhs) if rhs else None,
115         )
116         if not result.success:
117             raise RuntimeError(f"lexicographic MILP stage {index} failed: {result.message}")
118         value = float(np.dot(objective, result.x))
119         values.append(value)
120         rows.append(np.asarray(objective, dtype=float))
121         rhs.append(value + _lexicographic_slack(value, 10.0))
122     return result.x, result, values
123 def _validate(load_kw, pv_kw, price, storage: StorageParams, soc_initial, soc_terminal):
124     load_kw = np.asarray(load_kw, dtype=float)
125     pv_kw = np.asarray(pv_kw, dtype=float)
126     price = np.asarray(price, dtype=float)
127     if not (load_kw.shape == pv_kw.shape == price.shape):
128         raise ValueError("load/pv/price shape mismatch")
129     t = len(price)
130     if t == 0 or not np.isfinite(np.r_[load_kw, pv_kw, price]).all():
131         raise ValueError("invalid input")
132     if (load_kw < 0).any() or (pv_kw < 0).any() or (price <= 0).any():
133         raise ValueError("physical bounds violated")
134     if not storage.soc_min <= soc_initial <= storage.soc_max:
135         raise ValueError("initial SOC invalid")
136     if soc_terminal is not None and not storage.soc_min <= soc_terminal <= storage.soc_max:
137         raise ValueError("terminal SOC invalid")
138     return load_kw, pv_kw, price, t
139 def _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage: StorageParams, soc_initial)
140     :
141     g, ch, dis, soc, spill = _slices(t)

```



```

140     for i in range(t):
141         a_eq[i, g.start + i] = 1
142         a_eq[i, ch.start + i] = -1
143         a_eq[i, dis.start + i] = 1
144         a_eq[i, spill.start + i] = -1
145         b_eq[i] = (load_kw[i] - pv_kw[i]) * DT_HOURS
146         row = t + i
147         a_eq[row, soc.start + i] = 1
148         a_eq[row, ch.start + i] = -storage.eta_ch
149         a_eq[row, dis.start + i] = 1.0 / storage.eta_dis
150         if i == 0:
151             b_eq[row] = soc_initial
152         else:
153             a_eq[row, soc.start + i - 1] = -1
154 def solve_dispatch(
155     load_kw: np.ndarray,
156     pv_kw: np.ndarray,
157     price: np.ndarray,
158     *,
159     soc_initial: float,
160     soc_terminal: float | None,
161     storage: StorageParams = DEFAULT_STORAGE,
162 ) -> DispatchResult:
163     load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
164                                         soc_terminal)
165     g, ch, dis, soc, spill = _slices(t)
166     n = 5 * t
167     a_eq = np.zeros((2 * t + (soc_terminal is not None), n))
168     b_eq = np.zeros(a_eq.shape[0])
169     _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
170     if soc_terminal is not None:
171         a_eq[-1, soc.stop - 1] = 1
172         b_eq[-1] = soc_terminal
173     limit = storage.energy_limit
174     bounds = (
175         [(0, None)] * t
176         + [(0, limit)] * t
177         + [(0, limit)] * t
178         + [(storage.soc_min, storage.soc_max)] * t
179         + [(0, float(x) * DT_HOURS) for x in pv_kw]
180     )
181     primary_c = np.zeros(n)
182     primary_c[g] = price
183     throughput_c = np.zeros(n)
184     throughput_c[ch] = 1
185     throughput_c[dis] = 1
186     spill_c = np.zeros(n)

```

```

186     spill_c[spill] = 1
187     started = perf_counter()
188     x, tertiary, fell_back, values = _lexicographic(
189         [primary_c, throughput_c, spill_c], a_eq, b_eq, bounds
190     )
191     primary_value = values[0]
192     return DispatchResult(
193         x[g].copy(),
194         x[ch].copy(),
195         x[dis].copy(),
196         x[soc].copy(),
197         x[spill].copy(),
198         float(np.dot(price, x[g])),
199         primary_value,
200         float(x[spill].sum()),
201         float(x[ch].sum() + x[dis].sum()),
202         int(tertiary.status),
203         str(tertiary.message),
204         int(getattr(tertiary, "nit", -1)),
205         perf_counter() - started,
206         np.asarray(tertiary.eqlin.marginals, dtype=float),
207     )
208 @dataclass
209 class AdjustmentDispatchResult:
210     dispatch: DispatchResult
211     increase: np.ndarray
212     decrease: np.ndarray
213     settlement_objective: float
214     variable_objective: float
215     adjustment_mask: np.ndarray
216     settlement_mode: str
217     plan_purchase_constant: float
218     lexicographic_fallback: bool = False
219     throughput_objective: float = 0.0
220 def settlement_cost(plan_grid: np.ndarray, adjusted_grid: np.ndarray, price: np.ndarray,
221     mode: str) -> float:
222     plan = np.asarray(plan_grid, dtype=float)
223     adjusted = np.asarray(adjusted_grid, dtype=float)
224     p = np.asarray(price, dtype=float)
225     increase = np.maximum(adjusted - plan, 0.0)
226     decrease = np.maximum(plan - adjusted, 0.0)
227     if mode == "replacement":
228         return float(np.dot(p, np.minimum(plan, adjusted) + 0.5 * decrease + 1.5 * increase)
229     )
230     if mode == "additive":
231         return float(np.dot(p, plan) + np.dot(p, 0.5 * decrease + 1.5 * increase))
232     raise ValueError(f"unknown settlement mode: {mode}")

```

```

231 def solve_adjustment_dispatch(
232     load_kw: np.ndarray,
233     pv_kw: np.ndarray,
234     price: np.ndarray,
235     baseline_grid: np.ndarray,
236     adjustment_mask: np.ndarray,
237     *,
238     soc_initial: float,
239     soc_terminal: float | None,
240     settlement_mode: str = "replacement",
241     storage: StorageParams = DEFAULT_STORAGE,
242 ) -> AdjustmentDispatchResult:
243     if settlement_mode not in ("replacement", "additive"):
244         raise ValueError(f"unknown settlement mode: {settlement_mode}")
245     load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
246                                         soc_terminal)
247     baseline = np.asarray(baseline_grid, dtype=float)
248     mask = np.asarray(adjustment_mask, dtype=bool)
249     if not (baseline.shape == mask.shape == (t,)):
250         raise ValueError("adjustment shape mismatch")
251     g, ch, dis, soc, spill = _slices(t)
252     inc = slice(5 * t, 6 * t)
253     dec = slice(6 * t, 7 * t)
254     use_milp = True
255     z = slice(7 * t, 8 * t)
256     n = 8 * t
257     masked = np.flatnonzero(mask)
258     a_eq = np.zeros((2 * t + (soc_terminal is not None) + len(masked), n))
259     b_eq = np.zeros(a_eq.shape[0])
260     _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
261     if soc_terminal is not None:
262         last = a_eq.shape[0] - 1 - len(masked)
263         a_eq[last, soc.stop - 1] = 1
264         b_eq[last] = soc_terminal
265     limit = storage.energy_limit
266     bounds = (
267         [(0, None)] * t
268         + [(0, limit)] * t
269         + [(0, limit)] * t
270         + [(storage.soc_min, storage.soc_max)] * t
271         + [(0, float(x) * DT_HOURS) for x in pv_kw]
272         + [((0, None) if flag else (0, 0)) for flag in mask]
273         + [((0, None) if flag else (0, 0)) for flag in mask]
274         + [(0, 1)] * t if use_milp else []
275     )
276     for k, i in enumerate(masked):
277         row = 2 * t + (1 if soc_terminal is not None else 0) + k

```

```

277     a_eq[row, g.start + i] = 1.0
278     a_eq[row, inc.start + i] = -1.0
279     a_eq[row, dec.start + i] = 1.0
280     b_eq[row] = baseline[i]
281 adjustment_rows: list[np.ndarray] = []
282 adjustment_rhs: list[float] = []
283 primary_c = np.zeros(n)
284 if settlement_mode == "replacement":
285     primary_c[g] = price
286     primary_c[inc] = np.where(mask, 0.5 * price, 0.0)
287     primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
288     constant = 0.0
289 else:
290     primary_c[inc] = np.where(mask, 1.5 * price, 0.0)
291     primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
292     constant = float(np.sum(price[mask] * baseline[mask]))
293 throughput_c = np.zeros(n)
294 throughput_c[ch] = 1
295 throughput_c[dis] = 1
296 throughput_c[inc] = 1
297 throughput_c[dec] = 1
298 spill_c = np.zeros(n)
299 spill_c[spill] = 1
300 started = perf_counter()
301 if use_milp:
302     mutual_rows = np.zeros((2 * t, n))
303     mutual_rhs = np.zeros(2 * t)
304     for i in range(t):
305         mutual_rows[2 * i, ch.start + i] = 1.0
306         mutual_rows[2 * i, z.start + i] = -limit
307         mutual_rows[2 * i + 1, dis.start + i] = 1.0
308         mutual_rows[2 * i + 1, z.start + i] = limit
309         mutual_rhs[2 * i + 1] = limit
310     integrality = np.zeros(n, dtype=int)
311     integrality[z] = 1
312     tertiary = _solve_milp(
313         primary_c,
314         a_eq,
315         b_eq,
316         bounds,
317         integrality,
318         mutual_rows,
319         mutual_rhs,
320     )
321     if not tertiary.success:
322         raise RuntimeError(f"adjustment MILP failed: {tertiary.message}")
323     x = tertiary.x

```

```

324     values = [
325         float(np.dot(primary_c, x)),
326         float(np.dot(throughput_c, x)),
327         float(np.dot(spill_c, x)),
328     ]
329     fell_back = False
330     else:
331         x, tertiary, fell_back, values = _lexicographic(
332             [primary_c, throughput_c, spill_c],
333             a_eq,
334             b_eq,
335             bounds,
336             adjustment_rows,
337             adjustment_rhs,
338         )
339     primary_value = values[0]
340     adjusted = x[g].copy()
341     increase_vec = x[inc].copy()
342     decrease_vec = x[dec].copy()
343     if settlement_mode == "replacement":
344         settlement = float(np.dot(primary_c, x))
345     else:
346         settlement = constant + float(np.dot(primary_c, x))
347     dispatch = DispatchResult(
348         adjusted,
349         x[ch].copy(),
350         x[dis].copy(),
351         x[soc].copy(),
352         x[spill].copy(),
353         settlement,
354         float(primary_value + constant),
355         float(x[spill].sum()),
356         float(x[ch].sum() + x[dis].sum()),
357         int(tertiary.status),
358         str(tertiary.message),
359         int(getattr(tertiary, "nit", getattr(tertiary, "mip_node_count", -1)) or 0),
360         perf_counter() - started,
361         (
362             np.zeros(a_eq.shape[0], dtype=float)
363             if use_milp
364             else np.asarray(tertiary.eqlin.marginals, dtype=float)
365         ),
366     )
367     return AdjustmentDispatchResult(
368         dispatch,
369         increase_vec,
370         decrease_vec,

```

```

371         settlement,
372         float(np.dot(primary_c, x)),
373         mask.copy(),
374         settlement_mode,
375         constant,
376         bool(fell_back),
377         float(values[1]) if len(values) > 1 else float("nan"),
378     )

```

B.4 光伏日前预测与风险定价 (forecast.py)

Listing 4:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS, SLOTS_PER_DAY
5  METHOD_WINDOWS = {"persistence": 1, "mean7": 7, "mean14": 14, "mean30": 30, "weighted14":
6      14, "weighted30": 30}
7  QUANTILES = (0.80, 0.85, 0.90, 0.95)
8  @dataclass(frozen=True)
9  class ForecastDecision:
10     day_index: int
11     name: str
12     method: str
13     quantile: float
14     margin_kw: float
15     historical_score: float | None
16     history_days_used: int
17     forecast_kw: np.ndarray
18     candidates: dict[str, np.ndarray]
19     bases: dict[str, np.ndarray]
20     margins: dict[str, float]
21 class OnlinePVForecaster:
22     def __init__(
23         self,
24         price: np.ndarray,
25         score_window: int = 30,
26         residual_window: int = 30,
27         risk_mode: str = "asymmetric",
28         prior_pv: np.ndarray | None = None,
29     ) -> None:
30         if risk_mode not in ("asymmetric", "point"):
31             raise ValueError(f"unknown risk_mode: {risk_mode}")
32         self.price = np.asarray(price, dtype=float)
33         if self.price.shape != (SLOTS_PER_DAY,):

```

```

33         raise ValueError("price must contain 144 slots")
34     self.prior_pv = None if prior_pv is None else np.asarray(prior_pv, dtype=float)
35     self.score_window = score_window
36     self.residual_window = residual_window
37     self.risk_mode = risk_mode
38     self.actual_history: list[np.ndarray] = []
39     self.residual_days: dict[str, list[np.ndarray]] = {m: [] for m in METHOD_WINDOWS}
40     self.score_days: dict[str, list[float]] = {self._name(m, q): [] for m in
        METHOD_WINDOWS for q in QUANTILES}
41     if risk_mode == "point":
42         self.score_days = {m: [] for m in METHOD_WINDOWS}
43     @staticmethod
44     def _name(method: str, q: float) -> str:
45         return f"{method}_q{int(round(q * 100)):02d}"
46     def _base(self, method: str) -> np.ndarray:
47         if not self.actual_history:
48             return np.zeros(SLOTS_PER_DAY) if self.prior_pv is None else np.clip(self.
                prior_pv, 0.0, None)
49         n = min(METHOD_WINDOWS[method], len(self.actual_history))
50         recent = np.stack(self.actual_history[-n:])
51         if method == "persistence":
52             return recent[-1].copy()
53         if method.startswith("weighted"):
54             return np.average(recent, axis=0, weights=np.arange(1, n + 1, dtype=float))
55         return recent.mean(axis=0)
56     def _margin(self, method: str, q: float) -> float:
57         days = self.residual_days[method][-self.residual_window :]
58         if not days:
59             return 0.0
60         residual = np.concatenate(days)
61         return 0.0 if residual.size == 0 else max(0.0, float(np.quantile(residual, q)))
62     def predict(self) -> ForecastDecision:
63         d = len(self.actual_history)
64         bases = {m: self._base(m) for m in METHOD_WINDOWS}
65         candidates: dict[str, np.ndarray] = {}
66         margins: dict[str, float] = {}
67         if self.risk_mode == "asymmetric":
68             for method, base in bases.items():
69                 for q in QUANTILES:
70                     name = self._name(method, q)
71                     margins[name] = self._margin(method, q)
72                     candidates[name] = np.clip(base - margins[name], 0.0, None)
73                 default = self._name("persistence", 0.80)
74                 quantile = 0.80
75         else:
76             for method, base in bases.items():
77                 margins[method] = 0.0

```

```

78         candidates[method] = np.clip(base, 0.0, None)
79         default = "persistence"
80         quantile = 0.0
81         eligible = [n for n, s in self.score_days.items() if s]
82         if d < 7 or not eligible:
83             selected = default
84             hist = None
85         else:
86             scores = {n: float(np.mean(self.score_days[n][-self.score_window :])) for n in
                        eligible}
87             selected = min(scores, key=lambda n: (scores[n], n))
88             hist = scores[selected]
89         if self.risk_mode == "asymmetric":
90             method, q_text = selected.rsplit("_q", 1)
91             q = int(q_text) / 100.0
92         else:
93             method = selected
94             q = 0.0
95         return ForecastDecision(
96             d,
97             selected,
98             method,
99             q if self.risk_mode == "asymmetric" else 0.0,
100             margins[selected],
101             hist,
102             d,
103             candidates[selected].copy(),
104             candidates,
105             bases,
106             margins,
107         )
108     def observe(self, decision: ForecastDecision, actual_kw: np.ndarray, price: np.ndarray |
        None = None) -> dict[str, float]:
109         actual = np.asarray(actual_kw, dtype=float)
110         if actual.shape != (SLOTS_PER_DAY,) or decision.day_index != len(self.actual_history
            ):
111             raise ValueError("forecast observation out of sequence")
112         score_price = self.price if price is None else np.asarray(price, dtype=float)
113         if score_price.shape != (SLOTS_PER_DAY,):
114             raise ValueError("score price must contain 144 slots")
115         result: dict[str, float] = {}
116         for name, forecast in decision.candidates.items():
117             error = forecast - actual
118             if self.risk_mode == "asymmetric":
119                 score = float(np.sum(score_price * (4 * np.maximum(error, 0) + np.maximum(-
                    error, 0))) * DT_HOURS)
120             else:

```



```

121         score = float(np.mean(np.abs(error)))
122         self.score_days[name].append(score)
123         result[name] = score
124         for method, base in decision.bases.items():
125             mask = (actual > 1e-9) | (base > 1e-9)
126             self.residual_days[method].append((base - actual)[mask].copy())
127         self.actual_history.append(actual.copy())
128         return result

```

B.5 因果负载预测 (load_forecast.py)

Listing 5:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS, SLOTS_PER_DAY
5  METHOD_WINDOWS: dict[str, int] = {
6      "persistence": 1,
7      "mean7": 7,
8      "mean14": 14,
9      "mean30": 30,
10     "weighted14": 14,
11     "weighted30": 30,
12     "weekday_mean": 28,
13 }
14 BASE_METHODS = tuple(METHOD_WINDOWS)
15 NODE_SLOTS = (0, 36, 72, 108)
16 @dataclass(frozen=True)
17 class LoadForecastDecision:
18     day_index: int
19     node: int
20     name: str
21     method: str
22     corrected: bool
23     offset_kw: float
24     historical_score: float | None
25     history_days_used: int
26     horizon: int
27     forecast_kw: np.ndarray
28     candidates: dict[str, np.ndarray]
29     weekday_of_target: int
30 def _daily_window_shape(history: list[np.ndarray], method: str, weekday: int) -> np.ndarray:
31     if not history:
32         return np.zeros(SLOTS_PER_DAY, dtype=float)
33     if method == "weekday_mean":

```

```

34     same = [h for h, w in history if w == weekday]
35     if len(same) >= 2:
36         recent = np.stack(same[-4:])
37         return recent.mean(axis=0)
38     method = "mean7"
39     n = min(METHOD_WINDOWS[method], len(history))
40     recent = np.stack([h for h, _ in history[-n:]])
41     if method == "persistence":
42         return recent[-1].copy()
43     if method.startswith("weighted"):
44         weights = np.arange(1, n + 1, dtype=float)
45         return np.average(recent, axis=0, weights=weights)
46     return recent.mean(axis=0)
47 class OnlineLoadForecaster:
48     def __init__(self, score_window: int = 30, prior_kw: np.ndarray | None = None) -> None:
49         self.score_window = score_window
50         self.history: list[tuple[np.ndarray, int]] = []
51         self.prior_days = 0
52         if prior_kw is not None:
53             prior = np.asarray(prior_kw, dtype=float)
54             if prior.shape != (SLOTS_PER_DAY,):
55                 raise ValueError("prior load must have 144 slots")
56             self.history.append((prior.copy(), -1))
57             self.prior_days = 1
58         self.partial: np.ndarray = np.zeros(SLOTS_PER_DAY, dtype=float)
59         self.partial_len: int = 0
60         self.score_days: dict[tuple[int, str], list[float]] = {}
61     @property
62     def observed_days(self) -> int:
63         return len(self.history) - self.prior_days
64     def _target_weekday(self, day_index: int, day_offset: int) -> int:
65         return (day_index + day_offset) % 7
66     def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
67         wd_today = self._target_weekday(day_index, 0)
68         shape_today = _daily_window_shape(self.history, method, wd_today)
69         head = shape_today[node:]
70         if len(head) >= horizon:
71             return head[:horizon].copy()
72         wd_next = self._target_weekday(day_index, 1)
73         shape_next = _daily_window_shape(self.history, method, wd_next)
74         tail = horizon - len(head)
75         while len(shape_next) < tail:
76             shape_next = np.concatenate([shape_next, shape_next])
77         return np.concatenate([head, shape_next[:tail]])
78     def _offset(self, method: str, day_index: int, node: int) -> float:
79         if node <= 0 or self.partial_len <= 0:

```

```

80         return 0.0
81     k = min(36, node, self.partial_len)
82     if k < 6:
83         return 0.0
84     observed = self.partial[node - k : node]
85     base = _daily_window_shape(self.history, method, self._target_weekday(day_index, 0))
86         [node - k : node]
87     return float(np.mean(observed - base))
88
89 def _candidate_names(self, node: int) -> tuple[str, ...]:
90     if node == 0:
91         return BASE_METHODS
92     return tuple(f"{m}_{suffix}" for m in BASE_METHODS for suffix in ("raw", "offset"))
93
94 def _build(self, name: str, day_index: int, node: int, horizon: int) -> tuple[np.ndarray
95     , str, bool, float]:
96     if name.endswith("_offset"):
97         method, corrected = name[: -len("_offset")], True
98     elif name.endswith("_raw"):
99         method, corrected = name[: -len("_raw")], False
100     else:
101         method, corrected = name, False
102     base = self._base_horizon(method, day_index, node, horizon)
103     offset = self._offset(method, day_index, node) if corrected else 0.0
104     return np.clip(base + offset, 0.0, None), method, corrected, offset
105
106 def predict(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY) ->
107     LoadForecastDecision:
108     if node not in NODE_SLOTS:
109         raise ValueError(f"node must be one of {NODE_SLOTS}")
110     names = self._candidate_names(node)
111     candidates: dict[str, np.ndarray] = {}
112     meta: dict[str, tuple[str, bool, float]] = {}
113     for name in names:
114         forecast, method, corrected, offset = self._build(name, day_index, node, horizon
115             )
116         candidates[name] = forecast
117         meta[name] = (method, corrected, offset)
118     eligible = [n for n in names if self.score_days.get((node, n))]
119     history_days = self._score_history_days(node)
120     default = "persistence" if node == 0 else "persistence_raw"
121     if history_days < 7 or not eligible:
122         selected, hist = default, None
123     else:
124         scores = {
125             n: float(np.mean(self.score_days[(node, n)][-self.score_window :])) for n in
126                 eligible
127         }
128         selected = min(scores, key=lambda n: (scores[n], n))
129         hist = scores[selected]

```

```

122     method, corrected, offset = meta[selected]
123     return LoadForecastDecision(
124         day_index=day_index,
125         node=node,
126         name=selected,
127         method=method,
128         corrected=corrected,
129         offset_kw=offset,
130         historical_score=hist,
131         history_days_used=self.observed_days,
132         horizon=horizon,
133         forecast_kw=candidates[selected].copy(),
134         candidates=candidates,
135         weekday_of_target=self._target_weekday(day_index, 0),
136     )
137     def _score_history_days(self, node: int) -> int:
138         return max((len(v) for (n, _), v in self.score_days.items() if n == node), default
139                     =0)
140     def observe_block(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray) ->
141         dict[str, float]:
142         actual = np.asarray(actual_block_kw, dtype=float)
143         if actual.ndim != 1:
144             raise ValueError("actual block must be 1-D")
145         out: dict[str, float] = {}
146         for name, forecast in decision.candidates.items():
147             segment = forecast[: len(actual)]
148             mae = float(np.mean(np.abs(segment - actual)))
149             self.score_days.setdefault((decision.node, name), []).append(mae)
150             out[name] = mae
151         return out
152     def observe_partial(self, node: int, actual_slots: np.ndarray) -> None:
153         if actual_slots.shape != (node,) and node != 0:
154             raise ValueError("partial block shape mismatch")
155         self.partial[:node] = actual_slots
156         self.partial_len = node
157     def commit_day(self, actual_full_day_kw: np.ndarray, day_index: int) -> None:
158         actual = np.asarray(actual_full_day_kw, dtype=float)
159         if actual.shape != (SLOTS_PER_DAY,):
160             raise ValueError("daily load must have 144 slots")
161         self.history.append((actual.copy(), self._target_weekday(day_index, 0)))
162         self.partial = np.zeros(SLOTS_PER_DAY, dtype=float)
163         self.partial_len = 0
164     def log_row(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray, date_text
165                 : str) -> dict:
166         actual = np.asarray(actual_block_kw, dtype=float)
167         forecast = decision.forecast_kw[: len(actual)]
168         error = forecast - actual

```

```

166         slot = int(decision.node * 10) // 60
167         minute = (decision.node * 10) % 60
168         label = f"{slot:02d}:{minute:02d}"
169         return {
170             "date": date_text,
171             "decision_time": label,
172             "history_days_used": decision.history_days_used,
173             "candidate": decision.name,
174             "method": decision.method,
175             "offset_corrected": decision.corrected,
176             "offset_kw": decision.offset_kw,
177             "mae_kw": float(np.mean(np.abs(error))),
178             "rmse_kw": float(np.sqrt(np.mean(error**2))),
179             "daily_energy_error_kwh": float(np.sum(error) * DT_HOURS),
180             "block_energy_error_kwh": float(np.sum(error) * DT_HOURS),
181             "historical_score": decision.historical_score,
182         }

```

B.6 因果电价预测 (price_forecast.py)

Listing 6:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS, SLOTS_PER_DAY
5  from .load_forecast import NODE_SLOTS
6  METHOD_WINDOWS: dict[str, int] = {
7      "persistence": 1,
8      "mean7": 7,
9      "mean14": 14,
10     "mean30": 30,
11     "weighted14": 14,
12     "weighted30": 30,
13     "weekday_mean": 28,
14 }
15 BASE_METHODS = tuple(METHOD_WINDOWS)
16 @dataclass(frozen=True)
17 class PriceForecastDecision:
18     day_index: int
19     node: int
20     name: str
21     method: str
22     historical_score: float | None
23     history_days_used: int
24     horizon: int

```

```

25     forecast_price: np.ndarray
26     candidates: dict[str, np.ndarray]
27 def _window_shape(history: list[tuple[np.ndarray, int]], method: str, weekday: int) -> np.
    ndarray:
28     if not history:
29         return np.full(SLOTS_PER_DAY, np.nan, dtype=float)
30     if method == "weekday_mean":
31         same = [h for h, w in history if w == weekday]
32         if len(same) >= 2:
33             return np.stack(same[-4:]).mean(axis=0)
34         method = "mean7"
35     n = min(METHOD_WINDOWS[method], len(history))
36     recent = np.stack([h for h, _ in history[-n:]])
37     if method == "persistence":
38         return recent[-1].copy()
39     if method.startswith("weighted"):
40         weights = np.arange(1, n + 1, dtype=float)
41         return np.average(recent, axis=0, weights=weights)
42     return recent.mean(axis=0)
43 class OnlinePriceForecaster:
44     def __init__(self, score_window: int = 30, prior_price: np.ndarray | None = None) ->
        None:
45         self.score_window = score_window
46         self.history: list[tuple[np.ndarray, int]] = []
47         self.prior_days = 0
48         if prior_price is not None:
49             prior = np.asarray(prior_price, dtype=float)
50             if prior.shape != (SLOTS_PER_DAY,):
51                 raise ValueError("prior price must have 144 slots")
52             self.history.append((prior.copy(), -1))
53             self.prior_days = 1
54         self.partial_len: int = 0
55         self.score_days: dict[tuple[int, str], list[float]] = {}
56     @property
57     def observed_days(self) -> int:
58         return len(self.history) - self.prior_days
59     def _weekday(self, day_index: int, offset: int) -> int:
60         return (day_index + offset) % 7
61     def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
62         head = _window_shape(self.history, method, self._weekday(day_index, 0))[node:]
63         if len(head) >= horizon:
64             return head[:horizon].copy()
65         tail_len = horizon - len(head)
66         tail = _window_shape(self.history, method, self._weekday(day_index, 1))
67         while len(tail) < tail_len:
68             tail = np.concatenate([tail, tail])

```

```

69         return np.concatenate([head, tail[:tail_len]])
70     def _fallback(self) -> np.ndarray:
71         if not self.history:
72             return np.full(SLOTS_PER_DAY, 0.5, dtype=float)
73         return self.history[-1][0].copy()
74     def predict_horizon(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY)
75         -> PriceForecastDecision:
76         if node not in NODE_SLOTS:
77             raise ValueError(f"node must be one of {NODE_SLOTS}")
78         candidates: dict[str, np.ndarray] = {}
79         for method in BASE_METHODS:
80             horizon_values = self._base_horizon(method, day_index, node, horizon)
81             if not np.isfinite(horizon_values).all():
82                 horizon_values = np.where(np.isfinite(horizon_values), horizon_values, self.
83                     _fallback()[node % SLOTS_PER_DAY])
84             candidates[method] = np.clip(horizon_values, 1e-6, None)
85         eligible = [n for n in BASE_METHODS if self.score_days.get((node, n))]
86         history_days = max((len(v) for (n, _), v in self.score_days.items() if n == node),
87             default=0)
88         if history_days < 7 or not eligible:
89             selected, hist = "persistence", None
90         else:
91             scores = {n: float(np.mean(self.score_days[(node, n)][-self.score_window :]))
92                 for n in eligible}
93             selected = min(scores, key=lambda n: (scores[n], n))
94             hist = scores[selected]
95         return PriceForecastDecision(
96             day_index=day_index,
97             node=node,
98             name=selected,
99             method=selected,
100             historical_score=hist,
101             history_days_used=self.observed_days,
102             horizon=horizon,
103             forecast_price=candidates[selected].copy(),
104             candidates=candidates,
105         )
106     def observe_block(self, decision: PriceForecastDecision, actual_block: np.ndarray) ->
107         dict[str, float]:
108         actual = np.asarray(actual_block, dtype=float)
109         out: dict[str, float] = {}
110         for name, forecast in decision.candidates.items():
111             segment = forecast[: len(actual)]
112             mae = float(np.mean(np.abs(segment - actual)))
113             self.score_days.setdefault((decision.node, name), []).append(mae)
114             out[name] = mae
115         return out

```

```

111     def observe_partial(self, node: int) -> None:
112         self.partial_len = node
113     def commit_day(self, actual_full_day: np.ndarray, day_index: int) -> None:
114         actual = np.asarray(actual_full_day, dtype=float)
115         if actual.shape != (SLOTS_PER_DAY,):
116             raise ValueError("daily price must have 144 slots")
117         self.history.append((actual.copy(), self._weekday(day_index, 0)))
118         self.partial_len = 0
119     def log_row(self, decision: PriceForecastDecision, actual_block: np.ndarray, date_text:
120         str) -> dict:
121         actual = np.asarray(actual_block, dtype=float)
122         forecast = decision.forecast_price[: len(actual)]
123         error = forecast - actual
124         slot = int(decision.node * 10) // 60
125         minute = (decision.node * 10) % 60
126         return {
127             "date": date_text,
128             "decision_time": f"{slot:02d}:{minute:02d}",
129             "history_days_used": decision.history_days_used,
130             "method": decision.method,
131             "price_mae": float(np.mean(np.abs(error))),
132             "price_rmse": float(np.sqrt(np.mean(error**2))),
133             "price_energy_weighted_error": float(
134                 np.sum(np.abs(error) * actual) / max(np.sum(actual), 1e-9)
135             ),
136             "daily_price_energy_weighted_error": float(
137                 np.sum(np.abs(error) * actual) * DT_HOURS
138             ),
139             "historical_score": decision.historical_score,
140         }

```

B.7 实时储能反馈与执行 (settle.py)

Listing 7:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS
5  from .lp_core import DispatchResult
6  from .params import DEFAULT_STORAGE, StorageParams
7  TOL = 1e-10
8  @dataclass(frozen=True)
9  class ExecutionResult:
10     grid: np.ndarray
11     charge: np.ndarray

```



```

12     discharge: np.ndarray
13     soc: np.ndarray
14     curtail: np.ndarray
15     emergency: np.ndarray
16 def execute_with_realtime_storage_feedback(
17     plan: DispatchResult,
18     load_kw: np.ndarray,
19     actual_pv_kw: np.ndarray,
20     *,
21     soc_initial: float,
22     storage: StorageParams = DEFAULT_STORAGE,
23 ) -> ExecutionResult:
24     load = np.asarray(load_kw, dtype=float)
25     pv = np.asarray(actual_pv_kw, dtype=float)
26     n = len(load)
27     if pv.shape != load.shape or len(plan.grid) != n:
28         raise ValueError("execution shape mismatch")
29     limit = storage.energy_limit
30     c_out = np.zeros(n)
31     h_out = np.zeros(n)
32     soc = np.zeros(n)
33     spill = np.zeros(n)
34     emergency = np.zeros(n)
35     state = float(soc_initial)
36     for t in range(n):
37         c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
            eta_ch))
38         h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
            storage.eta_dis))
39         net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
40         if net < -TOL:
41             deficit = -net
42             reduce = min(c, deficit)
43             c -= reduce
44             deficit -= reduce
45             max_h = max(0.0, (state + storage.eta_ch * c - storage.soc_min) * storage.
                eta_dis)
46             add = min(deficit, limit - h, max(0.0, max_h - h))
47             h += add
48             deficit -= add
49             emergency[t] = max(0.0, deficit)
50         elif net > TOL:
51             surplus = net
52             reduce = min(h, surplus)
53             h -= reduce
54             surplus -= reduce
55             max_c = max(0.0, (storage.soc_max - (state - h / storage.eta_dis)) / storage.

```

```

        eta_ch)
56     add = min(surplus, limit - c, max(0.0, max_c - c))
57     c += add
58     surplus -= add
59     spill[t] = max(0.0, surplus)
60     state = state + storage.eta_ch * c - h / storage.eta_dis
61     if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
62         raise RuntimeError(f"SOC invalid at {t}: {state}")
63     c_out[t] = c
64     h_out[t] = h
65     soc[t] = min(storage.soc_max, max(storage.soc_min, state))
66     return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
67 def execute_fixed_storage_plan(
68     plan: DispatchResult,
69     load_kw: np.ndarray,
70     actual_pv_kw: np.ndarray,
71     *,
72     soc_initial: float,
73     storage: StorageParams = DEFAULT_STORAGE,
74 ) -> ExecutionResult:
75     load = np.asarray(load_kw, dtype=float)
76     pv = np.asarray(actual_pv_kw, dtype=float)
77     n = len(load)
78     if pv.shape != load.shape or len(plan.grid) != n:
79         raise ValueError("execution shape mismatch")
80     limit = storage.energy_limit
81     c_out = np.zeros(n)
82     h_out = np.zeros(n)
83     soc = np.zeros(n)
84     spill = np.zeros(n)
85     emergency = np.zeros(n)
86     state = float(soc_initial)
87     for t in range(n):
88         c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
            eta_ch))
89         h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
            storage.eta_dis))
90         net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
91         if net < 0:
92             emergency[t] = -net
93         elif net > 0:
94             spill[t] = net
95         state = state + storage.eta_ch * c - h / storage.eta_dis
96         if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
97             raise RuntimeError(f"SOC invalid at {t}: {state}")
98         c_out[t] = c
99         h_out[t] = h

```

```

100         soc[t] = min(storage.soc_max, max(storage.soc_min, state))
101     return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
102 def execute_plan(
103     plan: DispatchResult,
104     load_kw: np.ndarray,
105     actual_pv_kw: np.ndarray,
106     *,
107     soc_initial: float,
108     realtime_feedback: bool = True,
109     storage: StorageParams = DEFAULT_STORAGE,
110 ) -> ExecutionResult:
111     if realtime_feedback:
112         return execute_with_realtime_storage_feedback(
113             plan, load_kw, actual_pv_kw, soc_initial=soc_initial, storage=storage
114         )
115     return execute_fixed_storage_plan(plan, load_kw, actual_pv_kw, soc_initial=soc_initial,
        storage=storage)

```

B.8 问题一求解 (q1.py)

Listing 8:

```

1  from __future__ import annotations
2  import csv,json,platform,sys
3  from pathlib import Path
4  import numpy as np
5  import scipy
6  import openpyxl
7  from .export import export_result1
8  from .io_data import DEFAULT_DATA_ROOT,DT_HOURS,read_attachment1,template_path
9  from .lp_core import ETA_CH,ETA_DIS,SOC_MAX,SOC_MIN,solve_dispatch
10 ROOT=Path(__file__).resolve().parents[1]; OUTPUT=ROOT/"outputs"/"q1"
11 def run_q1(*,data_root:Path=DEFAULT_DATA_ROOT)->dict:
12     data=read_attachment1(data_root)
13     result=solve_dispatch(data.load_kw,data.pv_kw,data.price,soc_initial=6000,soc_terminal
        =6000)
14     balance=result.grid+data.pv_kw*DT_HOURS+result.discharge-data.load_kw*DT_HOURS-result.
        charge-result.curtail
15     previous=np.r_[6000.0,result.soc[:-1]]
16     soc_res=result.soc-(previous+ETA_CH*result.charge-result.discharge/ETA_DIS)
17     no_storage_grid=np.maximum((data.load_kw-data.pv_kw)*DT_HOURS,0)
18     metrics={"cost_yuan":result.objective,"primary_cost_yuan":result.primary_objective,
19             "primary_cost_gap_yuan":result.objective-result.primary_objective,
20             "grid_energy_kwh":float(result.grid.sum()),"charge_energy_kwh":float(result.charge.
        sum()),
21             "discharge_energy_kwh":float(result.discharge.sum()),"curtailment_energy_kwh":float(

```

```

        result.curtail.sum()),
22     "no_storage_cost_yuan":float(np.dot(data.price,no_storage_grid)),
23     "savings_yuan":float(np.dot(data.price,no_storage_grid)-result.objective),
24     "savings_percent":float(100*(np.dot(data.price,no_storage_grid)-result.objective)/np
        .dot(data.price,no_storage_grid)),
25     "initial_soc_kwh":6000.0,"terminal_soc_kwh":float(result.soc[-1]),
26     "min_soc_kwh":float(result.soc.min()),"max_soc_kwh":float(result.soc.max()),
27     "max_balance_residual":float(np.max(np.abs(balance))),"max_soc_residual":float(np.
        max(np.abs(soc_res))),
28     "max_simultaneous_charge_discharge":float(np.max(np.minimum(result.charge,result.
        discharge))),
29     "solver_status":result.status,"solver_message":result.message,"iterations":result.
        nit,
30     "solve_seconds":result.solve_seconds}
31 metrics["passed"]=bool(metrics["max_balance_residual"]<1e-7 and metrics["
        max_soc_residual"]<1e-7 and
32     abs(result.soc[-1]-6000)<1e-7 and metrics["min_soc_kwh"]>=SOC_MIN-1e-7 and
33     metrics["max_soc_kwh"]<=SOC_MAX+1e-7 and metrics["max_simultaneous_charge_discharge"
        ]<1e-5)
34 OUTPUT.mkdir(parents=True,exist_ok=True)
35 with (OUTPUT/"solution.csv").open("w",newline="",encoding="utf-8-sig") as f:
36     fields=["slot","label","price","load_kw","pv_kw","grid_kwh","charge_kwh","
        discharge_kwh","soc_kwh","curtail_kwh"]
37     writer=csv.DictWriter(f,fieldnames=fields); writer.writeheader()
38     for t in range(144):
39         writer.writerow({"slot":t+1,"label":data.labels[t],"price":data.price[t],"
        load_kw":data.load_kw[t],
40         "pv_kw":data.pv_kw[t],"grid_kwh":result.grid[t],"charge_kwh":result.charge[t
        ],
41         "discharge_kwh":result.discharge[t],"soc_kwh":result.soc[t],"curtail_kwh":
        result.curtail[t]})
42     check=export_result1(template_path("result1.xlsx",data_root),OUTPUT/"result1.xlsx",
        result)
43     metrics["xlsx_passed"]=check["passed"]
44     (OUTPUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),
        encoding="utf-8")
45     (OUTPUT/"xlsx_verification.json").write_text(json.dumps(check,ensure_ascii=False,indent
        =2),encoding="utf-8")
46     env={"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":
        scipy.__version__,
47         "openpyxl":openpyxl.__version__,"algorithm":"HiGHS lexicographic LP"}
48     (OUTPUT/"environment.json").write_text(json.dumps(env,ensure_ascii=False,indent=2),
        encoding="utf-8")
49     return {"metrics":metrics,"result":result}
50 if __name__=="__main__": print(json.dumps(run_q1()["metrics"],ensure_ascii=False,indent=2))

```

B.9 问题二求解 (q2.py)

Listing 9:

```
1 from __future__ import annotations
2 import csv
3 import json
4 import platform
5 import sys
6 from collections import Counter
7 from pathlib import Path
8 import numpy as np
9 import scipy
10 from .export import export_result2
11 from .forecast import QUANTILES, OnlinePVForecaster
12 from .io_data import DEFAULT_DATA_ROOT, DT_HOURS, read_attachment1, read_attachment2,
    template_path
13 from .load_forecast import OnlineLoadForecaster
14 from .lp_core import solve_dispatch
15 from .params import DEFAULT_STORAGE, StorageParams
16 from .price_forecast import OnlinePriceForecaster
17 from .settle import execute_plan
18 ROOT = Path(__file__).resolve().parents[1]
19 OUTPUT = ROOT / "outputs" / "q2"
20 def _verify_day(record: dict, storage: StorageParams) -> dict:
21     execution = record["execution"]
22     load = record["load_kw"]
23     pv = record["actual_pv_kw"]
24     balance = (
25         execution.grid
26         + pv * DT_HOURS
27         + execution.discharge
28         + execution.emergency
29         - load * DT_HOURS
30         - execution.charge
31         - execution.curtail
32     )
33     previous = np.r_[record["soc_initial"], execution.soc[:-1]]
34     soc_residual = execution.soc - (previous + storage.eta_ch * execution.charge - execution
        .discharge / storage.eta_dis)
35     return {
36         "max_balance_residual": float(np.max(np.abs(balance))),
37         "max_soc_residual": float(np.max(np.abs(soc_residual))),
38         "min_soc": float(execution.soc.min()),
39         "max_soc": float(execution.soc.max()),
40         "max_simultaneous": float(np.max(np.minimum(execution.charge, execution.discharge)))
    },
```

```

41     }
42 def _emergency_intervals(emergency: np.ndarray, threshold: float = 1e-7) -> int:
43     active = emergency > threshold
44     if not active.any():
45         return 0
46     return int(np.sum(active[1:] & ~active[:-1]) + (1 if active[0] else 0))
47 def run_q2(
48     *,
49     data_root: Path = DEFAULT_DATA_ROOT,
50     max_days: int = 365,
51     price_matrix: np.ndarray | None = None,
52     output_dir: Path | None = None,
53     template_name: str = "result2.xlsx",
54     write_outputs: bool = True,
55     load_mode: str = "causal",
56     pv_risk_mode: str = "asymmetric",
57     realtime_feedback: bool = True,
58     terminal_policy: str = "daily_cycle",
59     price_mode: str = "perfect",
60     storage: StorageParams = DEFAULT_STORAGE,
61     use_typical_prior: bool = True,
62     tag: str = "",
63 ) -> dict:
64     if load_mode not in ("perfect", "causal"):
65         raise ValueError(f"unknown load_mode: {load_mode}")
66     if price_mode not in ("perfect", "causal"):
67         raise ValueError(f"unknown price_mode: {price_mode}")
68     if terminal_policy not in ("daily_cycle", "free"):
69         raise ValueError(f"unknown terminal_policy: {terminal_policy}")
70     day1 = read_attachment1(data_root)
71     annual = read_attachment2(data_root)
72     if max_days < 1 or max_days > 365:
73         raise ValueError("max_days must be 1..365")
74     prices = np.tile(day1.price, (365, 1)) if price_matrix is None else np.asarray(
75         price_matrix, dtype=float)
76     if prices.shape != (365, 144):
77         raise ValueError("price matrix must have shape (365,144)")
78     out_dir = OUTPUT if output_dir is None else Path(output_dir)
79     prior_pv = day1.pv_kw if use_typical_prior else None
80     prior_load = day1.load_kw if use_typical_prior else None
81     prior_price = day1.price if use_typical_prior else None
82     selector = OnlinePVForecaster(day1.price, risk_mode=pv_risk_mode, prior_pv=prior_pv)
83     load_forecaster = OnlineLoadForecaster(prior_kw=prior_load)
84     price_forecaster = OnlinePriceForecaster(prior_price=prior_price)
85     state = 6000.0
86     records: list[dict] = []
87     forecast_rows: list[dict] = []

```

```

87     load_rows: list[dict] = []
88     price_rows: list[dict] = []
89     verify_rows: list[dict] = []
90     for d in range(max_days):
91         initial = state
92         load_decision = load_forecaster.predict(d, 0, 144)
93         pv_decision = selector.predict()
94         if pv_decision.history_days_used != d:
95             raise RuntimeError("PV forecast causality counter mismatch")
96         price_decision = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
            causal" else None
97         load_plan = annual.load_kw[d] if load_mode == "perfect" else load_decision.
            forecast_kw
98         price_plan = prices[d] if price_mode == "perfect" else price_decision.forecast_price
99         soc_terminal = initial if terminal_policy == "daily_cycle" else None
100        plan = solve_dispatch(
101            load_plan,
102            pv_decision.forecast_kw,
103            price_plan,
104            soc_initial=initial,
105            soc_terminal=soc_terminal,
106            storage=storage,
107        )
108        execution = execute_plan(
109            plan,
110            annual.load_kw[d],
111            annual.pv_kw[d],
112            soc_initial=initial,
113            realtime_feedback=realtime_feedback,
114            storage=storage,
115        )
116        plan_cost = float(np.dot(prices[d], plan.grid))
117        emergency_cost = float(np.dot(5.0 * prices[d], execution.emergency))
118        state = float(execution.soc[-1])
119        record = {
120            "day_index": d,
121            "date": annual.dates[d],
122            "soc_initial": initial,
123            "plan": plan,
124            "execution": execution,
125            "load_kw": annual.load_kw[d],
126            "actual_pv_kw": annual.pv_kw[d],
127            "load_plan_kw": np.asarray(load_plan, dtype=float),
128            "pv_plan_kw": pv_decision.forecast_kw,
129            "price_plan": np.asarray(price_plan, dtype=float),
130            "plan_cost": plan_cost,
131            "emergency_cost": emergency_cost,

```

```

132         "total_cost": plan_cost + emergency_cost,
133         "load_decision": load_decision,
134         "pv_decision": pv_decision,
135         "adjusted_grid": execution.grid,
136     }
137     records.append(record)
138     verify_rows.append(_verify_day(record, storage))
139     forecast_row = {
140         "day_index": d,
141         "date": annual.dates[d].isoformat(),
142         "history_days_used": pv_decision.history_days_used,
143         "candidate": pv_decision.name,
144         "method": pv_decision.method,
145         "quantile": pv_decision.quantile,
146         "margin_kw": pv_decision.margin_kw,
147         "historical_score": pv_decision.historical_score,
148         "pv_forecast_mae_kw": float(np.mean(np.abs(pv_decision.forecast_kw - annual.
149             pv_kw[d]))),
150         "pv_forecast_rmse_kw": float(np.sqrt(np.mean((pv_decision.forecast_kw -
151             annual.pv_kw[d]) ** 2))),
152         "pv_daily_energy_error_kwh": float(np.sum(pv_decision.forecast_kw - annual.
153             pv_kw[d]) * DT_HOURS),
154     }
155     forecast_rows.append(forecast_row)
156     load_rows.append(load_forecaster.log_row(load_decision, annual.load_kw[d], annual.
157         dates[d].isoformat()))
158     if price_decision is not None:
159         price_rows.append(
160             price_forecaster.log_row(price_decision, prices[d], annual.dates[d].
161                 isoformat())
162         )
163     candidate_scores = selector.observe(
164         pv_decision, annual.pv_kw[d], price=np.asarray(price_plan, dtype=float)
165     )
166     if pv_risk_mode == "asymmetric":
167         for quantile in QUANTILES:
168             name = OnlinePVForecaster._name(pv_decision.method, quantile)
169             forecast_row[f"risk_q{int(100 * quantile):02d}_yuan"] = candidate_scores[
170                 name]
171     load_forecaster.observe_block(load_decision, annual.load_kw[d])
172     load_forecaster.commit_day(annual.load_kw[d], d)
173     if price_decision is not None:
174         price_forecaster.observe_block(price_decision, prices[d])
175     price_forecaster.commit_day(prices[d], d)
176     delivery = records[31:] if max_days == 365 else []
177     min_soc = min(r["min_soc"] for r in verify_rows)
178     max_soc = max(r["max_soc"] for r in verify_rows)

```



```

173 max_balance = max(r["max_balance_residual"] for r in verify_rows)
174 max_soc_res = max(r["max_soc_residual"] for r in verify_rows)
175 max_sim = max(r["max_simultaneous"] for r in verify_rows)
176 continuity = (
177     max(abs(records[i]["soc_initial"] - records[i - 1]["execution"].soc[-1]) for i in
178         range(1, len(records)))
179     if len(records) > 1
180     else 0.0
181 )
182 metrics = {
183     "case_tag": tag,
184     "days_simulated": max_days,
185     "warmup_days": 31 if max_days == 365 else None,
186     "delivery_days": len(delivery),
187     "load_mode": load_mode,
188     "pv_risk_mode": pv_risk_mode,
189     "price_mode": price_mode,
190     "realtime_feedback": bool(realtime_feedback),
191     "terminal_policy": terminal_policy,
192     "storage": storage.as_dict(),
193     "price_source": "attachment1" if price_matrix is None else "attachment4",
194     "load_information": (
195         "Attachment 2 same-day true load curve (perfect-information benchmark)"
196         if load_mode == "perfect"
197         else "Online causal forecast from days strictly before d"
198     ),
199     "pv_information": (
200         "Causal forecast from days strictly before d, "
201         + ("historical residual quantile safety margin" if pv_risk_mode == "asymmetric"
202           else "no safety margin (point)")
203     ),
204     "price_information": (
205         "Attachment 4 same-day true price curve (perfect-information benchmark)"
206         if price_mode == "perfect"
207         else "Online causal forecast from days strictly before d"
208     ),
209     "realtime_storage_feedback": bool(realtime_feedback),
210     "terminal_policy_definition": (
211         "planned S144 equals actual S0 of the same day" if terminal_policy == "
212         daily_cycle" else "no terminal SOC constraint"
213     ),
214     "max_balance_residual": max_balance,
215     "max_soc_residual": max_soc_res,
216     "max_day_boundary_soc_gap": continuity,
217     "min_soc_kwh": min_soc,
218     "max_soc_kwh": max_soc,
219     "max_simultaneous_charge_discharge": max_sim,

```

```

217     "forecast_selection_counts": dict(Counter(row["candidate"] for row in forecast_rows)
218     ),
219     "load_selection_counts": dict(Counter(row["candidate"] for row in load_rows)),
220     "price_selection_counts": dict(Counter(row["method"] for row in price_rows) if
221     price_rows else None,
222 }
223 if delivery:
224     metrics.update(
225     {
226         "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
227         "plan_purchase_cost_yuan": float(sum(r["plan_cost"] for r in delivery)),
228         "adjustment_cost_yuan": 0.0,
229         "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
230         "plan_purchase_energy_kwh": float(sum(r["plan"].grid.sum() for r in delivery
231         )),
232         "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
233         delivery)),
234         "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
235         delivery)),
236         "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
237         delivery)),
238         "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
239         in delivery)),
240         "emergency_interval_count": int(
241             sum(_emergency_intervals(r["execution"].emergency) for r in delivery)
242         ),
243         "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
244         delivery)),
245         "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
246         r in delivery)),
247         "storage_throughput_kwh": float(
248             sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
249             in delivery)
250         ),
251         "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
252         "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
253         [31:]])),
254         "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
255         [31:]])),
256         "pv_forecast_mae_kw": float(np.mean([row["pv_forecast_mae_kw"] for row in
257         forecast_rows[31:]])),
258         "pv_forecast_rmse_kw": float(np.mean([row["pv_forecast_rmse_kw"] for row in
259         forecast_rows[31:]])),
260         "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
261         [31:]])) if price_rows else None,
262         "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
263         price_rows[31:]])) if price_rows else None,

```

```

248         "runtime_seconds": float(sum(r["plan"].solve_seconds for r in delivery)),
249     }
250 )
251 if pv_risk_mode == "asymmetric":
252     metrics["pv_residual_quantile_evaluation"] = {
253         f"Q{quantile:.2f}": {
254             "output_quantile": 1.0 - quantile,
255             "sample_out_risk_proxy_yuan": float(
256                 sum(row[f"risk_q{int(100 * quantile):02d}_yuan"] for row in
257                     forecast_rows[31:]))
258             ),
259             "selected_days": int(
260                 sum(abs(float(row["quantile"]) - quantile) < 1e-12 for row in
261                     forecast_rows[31:]))
262             ),
263         }
264     }
265     for quantile in QUANTILES
266 }
267 metrics["passed"] = bool(
268     max_balance < 1e-7
269     and max_soc_res < 1e-7
270     and continuity < 1e-9
271     and min_soc >= storage.soc_min - 1e-7
272     and max_soc <= storage.soc_max + 1e-7
273     and max_sim < 1e-5
274 )
275 if write_outputs:
276     out_dir.mkdir(parents=True, exist_ok=True)
277     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
278         =2), encoding="utf-8")
279     _write_csv(out_dir / "forecast_log.csv", forecast_rows)
280     _write_csv(out_dir / "load_forecast_log.csv", load_rows)
281     if price_rows:
282         _write_csv(out_dir / "price_forecast_log.csv", price_rows)
283     if delivery:
284         fields = [
285             "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "pv_plan_kw", "
286             price",
287             "grid_plan_kwh", "grid_actual_kwh", "charge_actual_kwh", "
288             discharge_actual_kwh",
289             "soc_actual_kwh", "curtail_actual_kwh", "emergency_kwh",
290         ]
291     with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
292         :
293         writer = csv.DictWriter(f, fieldnames=fields)
294         writer.writeheader()
295         for r in delivery:

```

```

289         for t in range(144):
290             writer.writerow(
291                 {
292                     "date": r["date"].isoformat(),
293                     "slot": t + 1,
294                     "load_kw": r["load_kw"][t],
295                     "actual_pv_kw": r["actual_pv_kw"][t],
296                     "load_plan_kw": r["load_plan_kw"][t],
297                     "pv_plan_kw": r["pv_plan_kw"][t],
298                     "price": prices[r["day_index"], t],
299                     "grid_plan_kwh": r["plan"].grid[t],
300                     "grid_actual_kwh": r["adjusted_grid"][t],
301                     "charge_actual_kwh": r["execution"].charge[t],
302                     "discharge_actual_kwh": r["execution"].discharge[t],
303                     "soc_actual_kwh": r["execution"].soc[t],
304                     "curtail_actual_kwh": r["execution"].curtail[t],
305                     "emergency_kwh": r["execution"].emergency[t],
306                 }
307             )
308         daily_fields = [
309             "date", "soc_initial", "soc_terminal", "plan_cost", "emergency_cost", "
310             total_cost",
311             "plan_energy", "emergency_energy", "curtailment_energy",
312         ]
313         _write_csv(
314             out_dir / "daily_metrics.csv",
315             [
316                 {
317                     "date": r["date"].isoformat(),
318                     "soc_initial": r["soc_initial"],
319                     "soc_terminal": r["execution"].soc[-1],
320                     "plan_cost": r["plan_cost"],
321                     "emergency_cost": r["emergency_cost"],
322                     "total_cost": r["total_cost"],
323                     "plan_energy": r["plan"].grid.sum(),
324                     "emergency_energy": r["execution"].emergency.sum(),
325                     "curtailment_energy": r["execution"].curtail.sum(),
326                 }
327             ],
328             for r in delivery
329         )
330         check = export_result2(template_path(template_name, data_root), out_dir /
331                                template_name, delivery)
332         (out_dir / "xlsx_verification.json").write_text(
333             json.dumps(check, ensure_ascii=False, indent=2), encoding="utf-8"
334         )

```

```

334         metrics["xlsx_passed"] = check["passed"]
335         (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
336             indent=2), encoding="utf-8")
337     environment = {
338         "python": sys.version,
339         "platform": platform.platform(),
340         "numpy": np.__version__,
341         "scipy": scipy.__version__,
342         "algorithm": "scipy.optimize.linprog(method=highs), lexicographic 3-pass LP",
343         "case_tag": tag,
344     }
345     if write_outputs:
346         (out_dir / "environment.json").write_text(json.dumps(environment, ensure_ascii=False,
347             indent=2), encoding="utf-8")
348     return {"metrics": metrics, "records": records, "forecast_rows": forecast_rows, "
349         load_rows": load_rows}
350 def _write_csv(path: Path, rows: list[dict], fields: list[str] | None = None) -> None:
351     if not rows:
352         return
353     fieldnames = fields if fields is not None else list(rows[0])
354     with path.open("w", newline="", encoding="utf-8-sig") as f:
355         writer = csv.DictWriter(f, fieldnames=fieldnames)
356         writer.writeheader()
357         writer.writerows(rows)
358 if __name__ == "__main__":
359     print(json.dumps(run_q2()["metrics"], ensure_ascii=False, indent=2))

```

B.10 问题三求解 (q3.py)

Listing 10:

```

1  from __future__ import annotations
2  import json
3  import platform
4  import sys
5  from pathlib import Path
6  import numpy as np
7  import scipy
8  from .export import export_result3
9  from .io_data import (
10     DEFAULT_DATA_ROOT,
11     DT_HOURS,
12     hourly_to_ten_min,
13     read_attachment1,
14     read_attachment2,
15     read_attachment3,

```

```

16     template_path,
17 )
18 from .load_forecast import OnlineLoadForecaster
19 from .lp_core import DispatchResult, settlement_cost, solve_adjustment_dispatch,
    solve_dispatch
20 from .official_forecast import OnlineForecastBiasCalibrator
21 from .params import DEFAULT_STORAGE, StorageParams
22 from .price_forecast import OnlinePriceForecaster
23 from .q2 import _emergency_intervals, _write_csv
24 from .settle import ExecutionResult, execute_plan
25 ROOT = Path(__file__).resolve().parents[1]
26 OUTPUT = ROOT / "outputs" / "q3"
27 BLOCK = 36
28 ALL_NODES = (0, 36, 72, 108)
29 def _horizon(data: np.ndarray, day: int, start_slot: int, fallback: np.ndarray) -> np.
    ndarray:
30     flat = data.reshape(-1)
31     start = day * 144 + start_slot
32     values = flat[start : min(start + 144, len(flat))]
33     if len(values) < 144:
34         pieces = [values]
35         remaining = 144 - len(values)
36         while remaining > 0:
37             take = min(remaining, len(fallback))
38             pieces.append(fallback[:take])
39             remaining -= take
40         values = np.concatenate(pieces)
41     return np.asarray(values, dtype=float)
42 def _slice_dispatch(result: DispatchResult, start: int, stop: int) -> DispatchResult:
43     return DispatchResult(
44         result.grid[start:stop].copy(),
45         result.charge[start:stop].copy(),
46         result.discharge[start:stop].copy(),
47         result.soc[start:stop].copy(),
48         result.curtail[start:stop].copy(),
49         result.objective,
50         result.primary_objective,
51         float(result.curtail[start:stop].sum()),
52         float(result.charge[start:stop].sum() + result.discharge[start:stop].sum()),
53         result.status,
54         result.message,
55         result.nit,
56         result.solve_seconds,
57         result.equality_marginals.copy(),
58     )
59 def _replace_dispatch_tail(current: DispatchResult, horizon: DispatchResult, start: int) ->
    DispatchResult:

```

```

60     count = 144 - start
61     def merged(name: str) -> np.ndarray:
62         values = np.asarray(getattr(current, name), dtype=float).copy()
63         values[start:] = np.asarray(getattr(horizon, name), dtype=float)[:count]
64         return values
65     charge = merged("charge")
66     discharge = merged("discharge")
67     curtail = merged("curtail")
68     return DispatchResult(
69         merged("grid"),
70         charge,
71         discharge,
72         merged("soc"),
73         curtail,
74         horizon.objective,
75         horizon.primary_objective,
76         float(curtail.sum()),
77         float(charge.sum() + discharge.sum()),
78         horizon.status,
79         horizon.message,
80         horizon.nit,
81         horizon.solve_seconds,
82         horizon.equality_marginals.copy(),
83     )
84 def _reversal_count(revisions: np.ndarray) -> int:
85     count = 0
86     for slot in range(144):
87         values = revisions[:, slot]
88         values = values[np.isfinite(values)]
89         signs = np.sign(np.diff(values))
90         signs = signs[np.abs(signs) > 0]
91         if len(signs) > 1 and np.any(signs[1:] * signs[:-1] < 0):
92             count += 1
93     return count
94 def _node_label(node: int) -> str:
95     return f"{node // 6:02d}:00"
96 def run_q3(
97     *,
98     data_root: Path = DEFAULT_DATA_ROOT,
99     max_days: int = 365,
100     downscale: str = "linear",
101     calibrate: bool = True,
102     update_nodes: tuple[int, ...] = ALL_NODES,
103     pv_update_nodes: tuple[int, ...] | None = None,
104     settlement_mode: str = "replacement",
105     load_mode: str = "causal",
106     price_mode: str = "perfect",

```

```

107     realtime_feedback: bool = True,
108     terminal_policy: str = "daily_cycle",
109     price_matrix: np.ndarray | None = None,
110     storage: StorageParams = DEFAULT_STORAGE,
111     output_dir: Path | None = None,
112     template_name: str = "result3.xlsx",
113     write_outputs: bool = True,
114     use_typical_prior: bool = True,
115     tag: str = "",
116 ) -> dict:
117     if settlement_mode not in ("replacement", "additive"):
118         raise ValueError(f"unknown settlement_mode: {settlement_mode}")
119     if load_mode not in ("perfect", "causal") or price_mode not in ("perfect", "causal"):
120         raise ValueError("unknown information mode")
121     nodes = tuple(sorted(int(n) for n in update_nodes))
122     if not nodes or nodes[0] != 0 or any(n not in ALL_NODES for n in nodes):
123         raise ValueError(f"update_nodes must be a prefix of {ALL_NODES}, got {update_nodes}"
124             )
125     pv_nodes = nodes if pv_update_nodes is None else tuple(sorted(int(n) for n in
126         pv_update_nodes))
127     if not pv_nodes or pv_nodes[0] != 0 or any(n not in nodes for n in pv_nodes):
128         raise ValueError(f"pv_update_nodes must contain 0 and be a subset of update_nodes,
129             got {pv_nodes}")
130     typical = read_attachment1(data_root)
131     annual = read_attachment2(data_root)
132     forecasts = read_attachment3(data_root)
133     if annual.dates != forecasts.dates:
134         raise ValueError("attachment2/3 dates do not align")
135     if not 1 <= max_days <= 365:
136         raise ValueError("max_days must be 1..365")
137     prices = np.tile(typical.price, (365, 1)) if price_matrix is None else np.asarray(
138         price_matrix, dtype=float)
139     if prices.shape != (365, 144):
140         raise ValueError("price matrix must have shape (365,144)")
141     out_dir = OUTPUT if output_dir is None else Path(output_dir)
142     state = 6000.0
143     records: list[dict] = []
144     verify: list[tuple] = []
145     load_rows: list[dict] = []
146     price_rows: list[dict] = []
147     total_reversals = 0
148     calibrator = OnlineForecastBiasCalibrator()
149     load_forecaster = OnlineLoadForecaster(prior_kw=typical.load_kw if use_typical_prior
150         else None)
151     price_forecaster = OnlinePriceForecaster(prior_price=typical.price if use_typical_prior
152         else None)
153     for d in range(max_days):

```



```

148     initial = state
149     node_state = initial
150     soc_terminal = initial if terminal_policy == "daily_cycle" else None
151     load_decision0 = load_forecaster.predict(d, 0, 144)
152     price_decision0 = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
        causal" else None
153     load0 = annual.load_kw[d] if load_mode == "perfect" else load_decision0.forecast_kw
154     price0 = prices[d] if price_mode == "perfect" else price_decision0.forecast_price
155     raw_base = hourly_to_ten_min(forecasts.hourly_kw[d, 0], downscale)
156     calibration0 = calibrator.calibrate(0, raw_base)
157     base_pv = calibration0.forecast_kw if calibrate else raw_base
158     baseline = solve_dispatch(
159         load0, base_pv, price0, soc_initial=initial, soc_terminal=soc_terminal, storage=
            storage
160     )
161     active_plan = baseline
162     active_pv_day = np.asarray(base_pv, dtype=float).copy()
163     active_source_node = 0
164     adjusted = np.zeros(144)
165     charge = np.zeros(144)
166     discharge = np.zeros(144)
167     soc = np.zeros(144)
168     curtail = np.zeros(144)
169     emergency = np.zeros(144)
170     inc_acc = np.zeros(144)
171     dec_acc = np.zeros(144)
172     revisions = np.full((4, 144), np.nan)
173     revisions[0] = baseline.grid
174     plan_source_nodes = np.zeros(144, dtype=int)
175     block_forecast_mae: list[float] = []
176     calibration_margins: list[float] = []
177     calibration_quantiles: list = []
178     load_plan_day = np.asarray(load0, dtype=float).copy()
179     load_rows.append(load_forecaster.log_row(load_decision0, annual.load_kw[d, :36],
        annual.dates[d].isoformat()))
180     if price_decision0 is not None:
181         price_rows.append(price_forecaster.log_row(price_decision0, prices[d, :36],
            annual.dates[d].isoformat()))
182     node_load_decisions: dict[int, object] = {0: load_decision0}
183     for node_index, node in enumerate(ALL_NODES):
184         block_stop = node + BLOCK
185         if node_index == 0:
186             calibration = calibration0
187             block_plan = _slice_dispatch(baseline, 0, BLOCK)
188             block_forecast = base_pv[0:BLOCK]
189             block_inc = np.zeros(BLOCK)
190             block_dec = np.zeros(BLOCK)

```

```

191 elif node in nodes:
192     if node in pv_nodes:
193         raw_horizon = hourly_to_ten_min(forecasts.hourly_kw[d, node_index],
194                                         downscale)
195         calibration = calibrator.calibrate(node_index, raw_horizon)
196         forecast_horizon = calibration.forecast_kw if calibrate else raw_horizon
197     else:
198         calibration = None
199         forecast_horizon = np.r_[active_pv_day[node:], typical.pv_kw[:node]]
200     if load_mode == "perfect":
201         load_horizon = _horizon(annual.load_kw, d, node, typical.load_kw)
202     else:
203         load_decision = load_forecaster.predict(d, node, 144)
204         load_horizon = load_decision.forecast_kw
205         load_plan_day[node:] = load_horizon[: 144 - node]
206         node_load_decisions[node] = load_decision
207         load_rows.append(
208             load_forecaster.log_row(load_decision, annual.load_kw[d, node:
209                                     block_stop], annual.dates[d].isoformat())
210         )
211     if price_mode == "perfect":
212         price_horizon = _horizon(prices, d, node, typical.price)
213     else:
214         price_decision = price_forecaster.predict_horizon(d, node, 144)
215         price_horizon = price_decision.forecast_price
216         price_rows.append(
217             price_forecaster.log_row(price_decision, prices[d, node:block_stop],
218                                     annual.dates[d].isoformat())
219         )
220     baseline_horizon = np.r_[baseline.grid[node:], np.zeros(node)]
221     mask = np.r_[np.ones(144 - node, dtype=bool), np.zeros(node, dtype=bool)]
222     rolling = solve_adjustment_dispatch(
223         load_horizon,
224         forecast_horizon,
225         price_horizon,
226         baseline_horizon,
227         mask,
228         soc_initial=node_state,
229         soc_terminal=None if terminal_policy == "free" else node_state,
230         settlement_mode=settlement_mode,
231         storage=storage,
232     )
233     horizon_plan = rolling.dispatch
234     active_plan = _replace_dispatch_tail(active_plan, horizon_plan, node)
235     active_pv_day[node:] = forecast_horizon[: 144 - node]
236     active_source_node = node
237     block_plan = _slice_dispatch(active_plan, node, block_stop)

```

```

235         revisions[node_index, node:] = active_plan.grid[node:]
236         block_forecast = active_pv_day[node:block_stop]
237         base_block = baseline.grid[node:block_stop]
238         block_inc = np.maximum(block_plan.grid - base_block, 0.0)
239         block_dec = np.maximum(base_block - block_plan.grid, 0.0)
240     else:
241         calibration = None
242         block_plan = _slice_dispatch(active_plan, node, block_stop)
243         block_forecast = active_pv_day[node:block_stop]
244         base_block = baseline.grid[node:block_stop]
245         block_inc = np.maximum(block_plan.grid - base_block, 0.0)
246         block_dec = np.maximum(base_block - block_plan.grid, 0.0)
247     plan_source_nodes[node:block_stop] = active_source_node
248     execution = execute_plan(
249         block_plan,
250         annual.load_kw[d, node:block_stop],
251         annual.pv_kw[d, node:block_stop],
252         soc_initial=node_state,
253         realtime_feedback=realtime_feedback,
254         storage=storage,
255     )
256     adjusted[node:block_stop] = execution.grid
257     charge[node:block_stop] = execution.charge
258     discharge[node:block_stop] = execution.discharge
259     soc[node:block_stop] = execution.soc
260     curtail[node:block_stop] = execution.curtail
261     emergency[node:block_stop] = execution.emergency
262     inc_acc[node:block_stop] = block_inc
263     dec_acc[node:block_stop] = block_dec
264     if calibration is not None:
265         calibrator.observe(calibration, annual.pv_kw[d, node:block_stop], prices[d,
266                                node:block_stop])
267         calibration_margins.append(calibration.margin_kw if calibrate else 0.0)
268         calibration_quantiles.append(calibration.quantile if calibrate else None)
269         block_forecast_mae.append(float(np.mean(np.abs(block_forecast - annual.pv_kw[d,
270                                node:block_stop]))))
271     node_state = float(execution.soc[-1])
272     if load_mode == "causal" and node in node_load_decisions:
273         load_forecaster.observe_block(node_load_decisions[node], annual.load_kw[d,
274                                node:block_stop])
275         load_forecaster.observe_partial(block_stop, annual.load_kw[d, :block_stop])
276     state = node_state
277     actual = ExecutionResult(adjusted.copy(), charge, discharge, soc, curtail, emergency
278                             )
279     gp = baseline.grid
280     ga = adjusted
281     increase = np.maximum(ga - gp, 0.0)

```

```

278 decrease = np.maximum(gp - ga, 0.0)
279 settlement = settlement_cost(gp, ga, prices[d], settlement_mode)
280 if settlement_mode == "replacement":
281     lp_side = float(np.dot(prices[d], ga + 0.5 * inc_acc + 0.5 * dec_acc))
282 else:
283     lp_side = float(np.dot(prices[d], gp) + np.dot(prices[d], 0.5 * dec_acc + 1.5 *
284                                     inc_acc))
285 emergency_cost = float(np.dot(5 * prices[d], emergency))
286 settled_base_cost = float(np.dot(prices[d], np.minimum(gp, ga)))
287 down_violation_cost = float(np.dot(0.5 * prices[d], decrease))
288 up_adjustment_cost = float(np.dot(1.5 * prices[d], increase))
289 down_adjustment_effect = -down_violation_cost if settlement_mode == "replacement"
290     else down_violation_cost
291 reversals = _reversal_count(revisions)
292 total_reversals += reversals
293 record = {
294     "day_index": d,
295     "date": annual.dates[d],
296     "soc_initial": initial,
297     "baseline_plan": baseline,
298     "adjusted_grid": adjusted,
299     "execution": actual,
300     "load_kw": annual.load_kw[d],
301     "actual_pv_kw": annual.pv_kw[d],
302     "load_plan_kw": load_plan_day,
303     "revisions": revisions,
304     "plan_source_nodes": plan_source_nodes.copy(),
305     "baseline_cost": float(np.dot(prices[d], gp)),
306     "settlement_cost": settlement,
307     "emergency_cost": emergency_cost,
308     "total_cost": settlement + emergency_cost,
309     "adjustment_cost_yuan": settlement - float(np.dot(prices[d], gp)),
310     "settled_base_cost_yuan": settled_base_cost,
311     "down_violation_cost_yuan": down_violation_cost,
312     "down_adjustment_effect_yuan": down_adjustment_effect,
313     "up_adjustment_cost_yuan": up_adjustment_cost,
314     "linearization_gap": abs(settlement - lp_side),
315     "up_energy": float(increase.sum()),
316     "down_energy": float(decrease.sum()),
317     "reversal_slots": reversals,
318     "forecast_mae": float(np.mean(block_forecast_mae)),
319     "calibration_margins": calibration_margins,
320     "calibration_quantiles": calibration_quantiles,
321 }
322 records.append(record)
323 balance = ga + annual.pv_kw[d] * DT_HOURS + discharge + emergency - annual.load_kw[d
    ] * DT_HOURS - charge - curtail

```

```

322     previous = np.r_[initial, soc[:-1]]
323     soc_res = soc - (previous + storage.eta_ch * charge - discharge / storage.eta_dis)
324     verify.append(
325         (
326             float(np.max(np.abs(balance))),
327             float(np.max(np.abs(soc_res))),
328             float(soc.min()),
329             float(soc.max()),
330             float(np.max(np.minimum(charge, discharge))),
331         )
332     )
333     if load_mode == "causal":
334         load_forecaster.commit_day(annual.load_kw[d], d)
335     if price_mode == "causal":
336         price_forecaster.commit_day(prices[d], d)
337     else:
338         price_forecaster.commit_day(prices[d], d)
339     delivery = records[31:] if max_days == 365 else []
340     metrics = {
341         "case_tag": tag,
342         "days_simulated": max_days,
343         "warmup_days": 31 if max_days == 365 else None,
344         "delivery_days": len(delivery),
345         "downscale_method": downscale,
346         "calibration_enabled": calibrate,
347         "update_nodes": list(nodes),
348         "update_nodes_label": "S" + str(len(nodes) - 1),
349         "pv_update_nodes": list(pv_nodes),
350         "settlement_mode": settlement_mode,
351         "load_mode": load_mode,
352         "price_mode": price_mode,
353         "realtime_feedback": bool(realtime_feedback),
354         "terminal_policy": terminal_policy,
355         "storage": storage.as_dict(),
356         "price_source": "attachment1" if price_matrix is None else "attachment4",
357         "load_information": (
358             "Attachment 2 same-day true load curve (perfect-information benchmark)"
359             if load_mode == "perfect"
360             else "Online causal forecast from days strictly before d"
361         ),
362         "pv_information": (
363             "Attachment 3 rolling forecasts with causal issue-specific calibration"
364             if calibrate
365             else "raw Attachment 3 forecasts"
366         )
367     } + f"; newly issued forecasts used at nodes {list(pv_nodes)}; actual PV used only
        after block execution",

```

```

368     "price_information": (
369         "Attachment 4 same-day true price curve (perfect-information benchmark)"
370         if price_mode == "perfect"
371         else "Online causal price forecast from days strictly before d"
372     ),
373     "max_balance_residual": max(v[0] for v in verify),
374     "max_soc_residual": max(v[1] for v in verify),
375     "min_soc_kwh": min(v[2] for v in verify),
376     "max_soc_kwh": max(v[3] for v in verify),
377     "max_simultaneous_charge_discharge": max(v[4] for v in verify),
378     "replacement_settlement_formula": "min(Gp,Ga) + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
379     "additive_settlement_formula": "Gp + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
380     "cross_midnight": "24h optimize, execute current 6h block only; carry SOC only",
381 }
382 if delivery:
383     metrics.update(
384         {
385             "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
386             "baseline_purchase_cost_yuan": float(sum(r["baseline_cost"] for r in
387                 delivery)),
388             "adjustment_settlement_cost_yuan": float(sum(r["settlement_cost"] for r in
389                 delivery)),
390             "adjustment_cost_yuan": float(sum(r["adjustment_cost_yuan"] for r in
391                 delivery)),
392             "settled_base_cost_yuan": float(sum(r["settled_base_cost_yuan"] for r in
393                 delivery)),
394             "down_violation_cost_yuan": float(sum(r["down_violation_cost_yuan"] for r in
395                 delivery)),
396             "down_adjustment_effect_yuan": float(sum(r["down_adjustment_effect_yuan"]
397                 for r in delivery)),
398             "up_adjustment_cost_yuan": float(sum(r["up_adjustment_cost_yuan"] for r in
399                 delivery)),
400             "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
401             "plan_purchase_energy_kwh": float(sum(r["baseline_plan"].grid.sum() for r in
402                 delivery)),
403             "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
404                 delivery)),
405             "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
406                 delivery)),
407             "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
408                 delivery)),
409             "up_adjustment_energy_kwh": float(sum(r["up_energy"] for r in delivery)),
410             "down_adjustment_energy_kwh": float(sum(r["down_energy"] for r in delivery))
411         },
412         "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
413             in delivery)),
414         "emergency_interval_count": int(sum(_emergency_intervals(r["execution"]).

```

```

emergency) for r in delivery)),
402     "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
delivery)),
403     "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
r in delivery)),
404     "storage_throughput_kwh": float(
405         sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
in delivery)
406     ),
407     "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
408     "direction_reversal_slots": int(sum(r["reversal_slots"] for r in delivery)),
409     "max_linearization_gap": float(max(r["linearization_gap"] for r in delivery)
),
410     "executed_block_forecast_mae_kw": float(np.mean([r["forecast_mae"] for r in
delivery])),
411     "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
[31:]])) if load_rows else None,
412     "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
[31:]])) if load_rows else None,
413     "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
[31:]])) if price_rows else None,
414     "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
price_rows[31:]])) if price_rows else None,
415     "runtime_seconds": float(
416         sum(r["baseline_plan"].solve_seconds for r in delivery)
417     ),
418 }
419 )
420 metrics["passed"] = bool(
421     metrics["max_balance_residual"] < 1e-7
422     and metrics["max_soc_residual"] < 1e-7
423     and metrics["min_soc_kwh"] >= storage.soc_min - 1e-7
424     and metrics["max_soc_kwh"] <= storage.soc_max + 1e-7
425     and metrics["max_simultaneous_charge_discharge"] < 1e-5
426     and (not delivery or metrics["max_linearization_gap"] < 1e-6)
427 )
428 if write_outputs:
429     out_dir.mkdir(parents=True, exist_ok=True)
430     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
=2), encoding="utf-8")
431     _write_csv(out_dir / "load_forecast_log.csv", load_rows)
432     if price_rows:
433         _write_csv(out_dir / "price_forecast_log.csv", price_rows)
434     if delivery:
435         fields = [
436             "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "price", "
baseline_grid_kwh",

```

```

437         "adjusted_grid_kwh", "charge_actual_kwh", "discharge_actual_kwh", "
            soc_actual_kwh",
438         "curtail_actual_kwh", "emergency_kwh", "revision_0", "revision_6", "
            revision_12", "revision_18",
439     ]
440     with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
441         :
442         import csv as _csv
443         writer = _csv.DictWriter(f, fieldnames=fields)
444         writer.writeheader()
445         for r in delivery:
446             for t in range(144):
447                 rev = r["revisions"][t, :]
448                 writer.writerow(
449                     {
450                         "date": r["date"].isoformat(),
451                         "slot": t + 1,
452                         "load_kw": r["load_kw"][t],
453                         "actual_pv_kw": r["actual_pv_kw"][t],
454                         "load_plan_kw": r["load_plan_kw"][t],
455                         "price": prices[r["day_index"], t],
456                         "baseline_grid_kwh": r["baseline_plan"].grid[t],
457                         "adjusted_grid_kwh": r["adjusted_grid"][t],
458                         "charge_actual_kwh": r["execution"].charge[t],
459                         "discharge_actual_kwh": r["execution"].discharge[t],
460                         "soc_actual_kwh": r["execution"].soc[t],
461                         "curtail_actual_kwh": r["execution"].curtail[t],
462                         "emergency_kwh": r["execution"].emergency[t],
463                         "revision_0": rev[0],
464                         "revision_6": rev[1],
465                         "revision_12": rev[2],
466                         "revision_18": rev[3],
467                     }
468                 )
469     fields_d = [
470         "date", "soc_initial", "soc_terminal", "baseline_cost", "settlement_cost", "
            emergency_cost",
471         "total_cost", "up_energy", "down_energy", "reversal_slots", "forecast_mae",
472     ]
473     _write_csv(
474         out_dir / "daily_metrics.csv",
475         [
476             {
477                 "date": r["date"].isoformat(),
478                 "soc_initial": r["soc_initial"],
479                 "soc_terminal": r["execution"].soc[-1],
480                 **{k: r[k] for k in fields_d[3:]},

```



```

480         }
481         for r in delivery
482     ],
483     fields_d,
484 )
485     check = export_result3(template_path(template_name, data_root), out_dir /
486                             template_name, delivery)
487     (out_dir / "xlsx_verification.json").write_text(json.dumps(check, ensure_ascii=
488                                                         False, indent=2), encoding="utf-8")
489     metrics["xlsx_passed"] = check["passed"]
490     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
491                                                         indent=2), encoding="utf-8")
492
493     env = {
494         "python": sys.version,
495         "platform": platform.platform(),
496         "numpy": np.__version__,
497         "scipy": scipy.__version__,
498         "algorithm": "HiGHS MILP with binary charge/discharge exclusivity and explicit (
499                     inc, dec) split",
500         "case_tag": tag,
501     }
502     (out_dir / "environment.json").write_text(json.dumps(env, ensure_ascii=False, indent
503                                                         =2), encoding="utf-8")
504
505     return {"metrics": metrics, "records": records}
506
507 if __name__ == "__main__":
508     print(json.dumps(run_q3()["metrics"], ensure_ascii=False, indent=2))

```

B.11 问题四求解 (q4.py)

Listing 11:

```

1  from __future__ import annotations
2  import json
3  from pathlib import Path
4  from .io_data import DEFAULT_DATA_ROOT, read_attachment4
5  from .q2 import run_q2
6  from .q3 import run_q3
7  ROOT = Path(__file__).resolve().parents[1]
8  OUTPUT = ROOT / "outputs"
9  def run_q4(
10     *,
11     data_root: Path = DEFAULT_DATA_ROOT,
12     max_days: int = 365,
13     price_mode_q2: str = "perfect",
14     price_mode_q3: str = "perfect",
15     load_mode: str = "causal",

```

```

16     settlement_mode: str = "replacement",
17     update_nodes: tuple[int, ...] = (0, 36, 72, 108),
18     realtime_feedback: bool = True,
19     write_outputs: bool = True,
20     tag_prefix: str = "",
21 ) -> dict:
22     annual_price = read_attachment4(data_root)
23     prices = annual_price.price
24     suffix = f"--{tag_prefix}" if tag_prefix else ""
25     out2 = OUTPUT / f"q4-2{suffix}"
26     out3 = OUTPUT / f"q4-3{suffix}"
27     q4_2 = run_q2(
28         data_root=data_root,
29         max_days=max_days,
30         price_matrix=prices,
31         output_dir=out2,
32         template_name="result4-2.xlsx",
33         write_outputs=write_outputs,
34         load_mode=load_mode,
35         price_mode=price_mode_q2,
36         realtime_feedback=realtime_feedback,
37         tag=f"q4-2 {tag_prefix}".strip(),
38     )
39     q4_3 = run_q3(
40         data_root=data_root,
41         max_days=max_days,
42         price_matrix=prices,
43         output_dir=out3,
44         template_name="result4-3.xlsx",
45         write_outputs=write_outputs,
46         load_mode=load_mode,
47         price_mode=price_mode_q3,
48         settlement_mode=settlement_mode,
49         update_nodes=update_nodes,
50         realtime_feedback=realtime_feedback,
51         tag=f"q4-3 {tag_prefix}".strip(),
52     )
53     c2 = q4_2["metrics"].get("total_cost_yuan")
54     c3 = q4_3["metrics"].get("total_cost_yuan")
55     comparison = {
56         "q4_2_total_cost_yuan": c2,
57         "q4_3_total_cost_yuan": c3,
58         "rolling_saving_yuan": None if (c2 is None or c3 is None) else c2 - c3,
59         "rolling_saving_percent": None if (c2 is None or c3 is None) else 100 * (c2 - c3) /
60             c2,
61         "q4_2_emergency_energy_kwh": q4_2["metrics"].get("emergency_energy_kwh"),
62         "q4_3_emergency_energy_kwh": q4_3["metrics"].get("emergency_energy_kwh"),

```

```

62     "q4_2_price_mode": price_mode_q2,
63     "q4_3_price_mode": price_mode_q3,
64     "q4_2_price_mae": q4_2["metrics"].get("price_forecast_mae"),
65     "q4_3_price_mae": q4_3["metrics"].get("price_forecast_mae"),
66     "q4_2_passed": bool(q4_2["metrics"]["passed"]),
67     "q4_3_passed": bool(q4_3["metrics"]["passed"]),
68 }
69 comparison["passed"] = comparison["q4_2_passed"] and comparison["q4_3_passed"]
70 if write_outputs:
71     out2.mkdir(parents=True, exist_ok=True)
72     (out2 / "q4_comparison.json").write_text(
73         json.dumps(comparison, ensure_ascii=False, indent=2), encoding="utf-8"
74     )
75     return {"q4_2": q4_2, "q4_3": q4_3, "comparison": comparison}
76 if __name__ == "__main__":
77     print(json.dumps(run_q4()["comparison"], ensure_ascii=False, indent=2))

```

B.12 官方结果文件导出 (export.py)

Listing 12:

```

1  from __future__ import annotations
2  from copy import copy
3  from datetime import datetime, time
4  from pathlib import Path
5  from shutil import copy2
6  import numpy as np
7  import openpyxl
8  from .lp_core import DispatchResult
9  def export_result1(template: Path, destination: Path, result: DispatchResult) -> dict:
10     destination.parent.mkdir(parents=True, exist_ok=True); copy2(template, destination)
11     wb=openpyxl.load_workbook(destination); plan, storage=wb.worksheets[:2]
12     for row, value in enumerate(result.grid, start=2):
13         plan.cell(row, 2, float(value)); plan.cell(row, 2).number_format="0.0000"
14     for block in range(6):
15         lo, hi=block*24, (block+1)*24
16         storage.cell(block+2, 2, float(result.charge[lo:hi].sum()))
17         storage.cell(block+2, 3, float(result.discharge[lo:hi].sum()))
18     storage.cell(2, 5, 6000.0); storage.cell(3, 5, float(result.soc[-1])); wb.save(destination)
19     check=openpyxl.load_workbook(destination, data_only=True, read_only=True)
20     values=np.asarray([check.worksheets[0].cell(i, 2).value for i in range(2, 146)], dtype=
21         float)
22     diff=float(np.max(np.abs(values-result.grid)))
23     return {"rows": len(values), "max_plan_roundtrip_diff": diff, "passed": bool(diff<1e-9)}
24 def _copy_row_style(ws, source_row: int, target_row: int, max_col: int) -> None:
25     for col in range(1, max_col+1):

```

```

25     src=ws.cell(source_row,col); dst=ws.cell(target_row,col)
26     if src.has_style: dst._style=copy(src._style)
27     dst.alignment=copy(src.alignment); dst.protection=copy(src.protection)
28     ws.row_dimensions[target_row].height=ws.row_dimensions[source_row].height
29 def _segments(values:np.ndarray,tolerance:float=1e-7)->list[tuple[int,int,float]]:
30     result=[]; start=None
31     for idx,flag in enumerate(np.r_[np.asarray(values)>tolerance,False]):
32         if flag and start is None: start=idx
33         elif not flag and start is not None:
34             result.append((start,idx,float(np.sum(values[start:idx])))); start=None
35     return result
36 def _slot_time(slot:int)->str:
37     minutes=slot*10
38     return "24:00" if minutes==1440 else f"{minutes//60}:{minutes%60:02d}"
39 def export_result2(template:Path,destination:Path,days:list[dict])->dict:
40     if len(days)!=334: raise ValueError(f"result2 requires 334 days, got {len(days)}")
41     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
42     wb=openpyxl.load_workbook(destination); plan_ws,storage_ws,emergency_ws=wb.worksheets
43     [:3]
44     for idx,day in enumerate(days):
45         row=idx+2; plan=day["plan"]
46         plan_ws.cell(row,1,datetime.combine(day["date"],time()))
47         for slot,value in enumerate(plan.grid,start=2):
48             plan_ws.cell(row,slot,float(value)); plan_ws.cell(row,slot).number_format="
49                 0.0000"
50             plan_ws.cell(row,146,float(np.sum(plan.grid))); plan_ws.cell(row,147,float(day["
51                 plan_cost"]))
52             plan_ws.cell(row,146).number_format=plan_ws.cell(row,147).number_format="0.0000"
53 target=1+6*len(days)
54 if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
55     storage_ws.max_row)
56 periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
57 for idx,day in enumerate(days):
58     actual=day["execution"]
59     for block in range(6):
60         row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
61         storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
62             )
63         storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
64         storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
65         storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
66         storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
67             )
68         storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
69             soc[-1]) if block==1 else None))
70         for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
71 if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)

```

```

65     rows=[]
66     for day in days:
67         segments=_segments(day["execution"].emergency)
68         if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
69         else:
70             for k,(start,end,total) in enumerate(segments):
71                 rows.append((datetime.combine(day["date"],time()) if k==0 else None,
72                             f"{_slot_time(start)}-{_slot_time(end)}",total))
73     target_e=1+len(rows)
74     if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
75                       =target_e-emergency_ws.max_row)
76     for row,(date_value,period,total) in enumerate(rows,start=2):
77         _copy_row_style(emergency_ws,2,row,3)
78         emergency_ws.cell(row,1,date_value); emergency_ws.cell(row,2,period)
79         emergency_ws.cell(row,3,float(total)); emergency_ws.cell(row,3).number_format="
80             0.0000"
81     if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
82                       max_row-target_e)
83     wb.save(destination)
84     check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
85     plan_rows=list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col
86                       =145,values_only=True))
87     values=np.asarray(plan_rows,dtype=float)
88     expected=np.stack([d["plan"].grid for d in days]); diff=float(np.max(np.abs(values-
89                       expected)))
90     last_row=3+6*(len(days)-1)
91     last_soc=float(next(check.worksheets[1].iter_rows(min_row=last_row,max_row=last_row,
92                       min_col=6,max_col=6,values_only=True))[0])
93     return {"delivery_days":334,"plan_rows":values.shape[0],"plan_slots":values.shape[1],
94            "storage_rows":check.worksheets[1].max_row-1,"emergency_rows":check.worksheets
95            [2].max_row-1,
96            "max_plan_roundtrip_diff":diff,"last_terminal_soc":last_soc,
97            "passed":bool(values.shape==(334,144) and diff<1e-9 and check.worksheets[1].
98            max_row-1==2004)}
99
100 def export_result3(template:Path,destination:Path,days:list[dict])>->dict:
101     if len(days)!=334: raise ValueError(f"result3 requires 334 days, got {len(days)}")
102     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
103     wb=openpyxl.load_workbook(destination); plan_ws,adjust_ws,storage_ws,emergency_ws=wb.
104           worksheets[:4]
105     for idx,day in enumerate(days):
106         row=idx+2; plan=day["baseline_plan"]; adjusted=day["adjusted_grid"]
107         for ws,values,cost in ((plan_ws,plan.grid,day["baseline_cost"]), (adjust_ws,adjusted,
108           day["settlement_cost"])):
109             ws.cell(row,1,datetime.combine(day["date"],time()))
110             for slot,value in enumerate(values,start=2):
111                 ws.cell(row,slot,float(value)); ws.cell(row,slot).number_format="0.0000"
112             ws.cell(row,146,float(np.sum(values))); ws.cell(row,147,float(cost))

```

```

1102         ws.cell(row,146).number_format=ws.cell(row,147).number_format="0.0000"
1103     target=1+6*len(days)
1104     if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
        storage_ws.max_row)
1105     periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
1106     for idx,day in enumerate(days):
1107         actual=day["execution"]
1108         for block in range(6):
1109             row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
1110             storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
                )
1111             storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
1112             storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
1113             storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
1114             storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
                )
1115             storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
                soc[-1]) if block==1 else None))
1116             for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
1117     if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
1118     rows=[]
1119     for day in days:
1120         segments=_segments(day["execution"].emergency)
1121         if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
1122         else:
1123             for k,(start,end,total) in enumerate(segments):
1124                 rows.append((datetime.combine(day["date"],time()) if k==0 else None,
1125                     f"{_slot_time(start)}--{_slot_time(end)}",total))
1126     target_e=1+len(rows)
1127     if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
        =target_e-emergency_ws.max_row)
1128     for row,(date_value,period,total) in enumerate(rows,start=2):
1129         _copy_row_style(emergency_ws,2,row,3); emergency_ws.cell(row,1,date_value)
1130         emergency_ws.cell(row,2,period); emergency_ws.cell(row,3,float(total))
1131         emergency_ws.cell(row,3).number_format="0.0000"
1132     if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
        max_row-target_e)
1133     wb.save(destination)
1134     check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
1135     plan_values=np.asarray(list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col
        =2,max_col=145,values_only=True)),dtype=float)
1136     adjust_values=np.asarray(list(check.worksheets[1].iter_rows(min_row=2,max_row=335,
        min_col=2,max_col=145,values_only=True)),dtype=float)
1137     expected_plan=np.stack([d["baseline_plan"].grid for d in days]); expected_adjust=np.
        stack([d["adjusted_grid"] for d in days])
1138     return {"delivery_days":334,"plan_shape":list(plan_values.shape),"adjust_shape":list(
        adjust_values.shape),

```

```

139         "storage_rows":check.worksheets[2].max_row-1,"emergency_rows":check.worksheets
140             [3].max_row-1,
141         "max_plan_roundtrip_diff":float(np.max(np.abs(plan_values-expected_plan))),
142         "max_adjust_roundtrip_diff":float(np.max(np.abs(adjust_values-expected_adjust)))
143     },
144     "passed":bool(plan_values.shape==(334,144) and adjust_values.shape==(334,144)
        and
        check.worksheets[2].max_row-1==2004 and
        np.max(np.abs(plan_values-expected_plan))<1e-9 and np.max(np.abs(
            adjust_values-expected_adjust))<1e-9)}

```

B.13 模型审计与消融实验总控 (ablation.py)

Listing 13:

```

1  from __future__ import annotations
2  import json
3  import time
4  from pathlib import Path
5  import numpy as np
6  from .io_data import DEFAULT_DATA_ROOT, read_attachment1, read_attachment4
7  from .params import DEFAULT_STORAGE, EFFICIENCY_VARIANTS
8  from .q1 import run_q1
9  from .q2 import run_q2
10 from .q3 import run_q3
11 ROOT = Path(__file__).resolve().parents[1]
12 VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
13 Q2_CASES = {
14     "A_perfectload_risk_feedback": dict(load_mode="perfect", pv_risk_mode="asymmetric",
15         realtime_feedback=True),
16     "B_causalload_risk_feedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
17         realtime_feedback=True),
18     "C_causalload_point_feedback": dict(load_mode="causal", pv_risk_mode="point",
19         realtime_feedback=True),
20     "D_causalload_risk_nofeedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
21         realtime_feedback=False),
22 }
23 Q3_MAIN_CASES = {
24     "S0_0": dict(update_nodes=(0,)),
25     "S1_0_6": dict(update_nodes=(0, 36)),
26     "S2_0_6_12": dict(update_nodes=(0, 36, 72)),
27     "S3_0_6_12_18": dict(update_nodes=(0, 36, 72, 108)),
28 }
29 Q3_CONTROL_CASES = {
30     "C1_load6": dict(update_nodes=(0, 36), pv_update_nodes=(0,)),
31     "C2_load12": dict(update_nodes=(0, 36, 72), pv_update_nodes=(0, 36)),
32 }

```

```

28     "C3_load18": dict(update_nodes=(0, 36, 72, 108), pv_update_nodes=(0, 36, 72)),
29 }
30 Q3_CASES = {**Q3_MAIN_CASES, **Q3_CONTROL_CASES}
31 Q4_CASES = {
32     "q42_perfect_price": ("q2", dict(price_mode="perfect")),
33     "q42_causal_price": ("q2", dict(price_mode="causal")),
34     "q43_perfect_price": ("q3", dict(price_mode="perfect")),
35     "q43_causal_price": ("q3", dict(price_mode="causal")),
36 }
37 SEASONS = {
38     "spring": (3, 4, 5),
39     "summer": (6, 7, 8),
40     "autumn": (9, 10, 11),
41     "winter": (12, 1, 2),
42 }
43 def _season(month: int) -> str:
44     for name, months in SEASONS.items():
45         if month in months:
46             return name
47     raise ValueError(month)
48 def run_q2_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
49     out = VARIANTS / "q2"
50     summary = {}
51     for name, kwargs in Q2_CASES.items():
52         started = time.perf_counter()
53         result = run_q2(
54             data_root=data_root,
55             max_days=max_days,
56             output_dir=out / name,
57             template_name="result2.xlsx",
58             write_outputs=True,
59             tag=name,
60             **kwargs,
61         )
62         metrics = result["metrics"]
63         metrics["wall_seconds"] = time.perf_counter() - started
64         (out / name / "metrics.json").write_text(
65             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
66         )
67         summary[name] = metrics
68         print(f"[Q2] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
69     _dump(out / "summary.json", summary)
70     return summary
71 def run_q3_ablation(
72     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "
replacement"

```



```

73 ) -> dict:
74     out = VARIANTS / f"q3_{settlement_mode}"
75     summary = {}
76     for name, kwargs in Q3_CASES.items():
77         started = time.perf_counter()
78         result = run_q3(
79             data_root=data_root,
80             max_days=max_days,
81             output_dir=out / name,
82             template_name="result3.xlsx",
83             write_outputs=True,
84             settlement_mode=settlement_mode,
85             tag=name,
86             **kwargs,
87         )
88         metrics = result["metrics"]
89         metrics["wall_seconds"] = time.perf_counter() - started
90         (out / name / "metrics.json").write_text(
91             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
92         )
93         (out / name / "by_season.csv").write_text(_season_table(result["records"]), encoding="utf-8-sig")
94         summary[name] = metrics
95         print(f"[Q3-{settlement_mode}] {name}: total={metrics.get('total_cost_yuan')}")
96     _dump(out / "summary.json", summary)
97     return summary
98 def run_q3_controls(
99     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "
    replacement"
100 ) -> dict:
101     out = VARIANTS / f"q3_{settlement_mode}"
102     summary_path = out / "summary.json"
103     summary = json.loads(summary_path.read_text(encoding="utf-8")) if summary_path.exists()
        else {}
104     for name, kwargs in Q3_CONTROL_CASES.items():
105         started = time.perf_counter()
106         result = run_q3(
107             data_root=data_root,
108             max_days=max_days,
109             output_dir=out / name,
110             template_name="result3.xlsx",
111             write_outputs=True,
112             settlement_mode=settlement_mode,
113             tag=name,
114             **kwargs,
115         )
116         metrics = result["metrics"]

```

```

117     metrics["wall_seconds"] = time.perf_counter() - started
118     (out / name / "metrics.json").write_text(
119         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
120     )
121     (out / name / "by_season.csv").write_text(
122         _season_table(result["records"]), encoding="utf-8-sig"
123     )
124     summary[name] = metrics
125     print(f"[Q3-{{settlement_mode}}-control] {name}: total={{metrics.get('total_cost_yuan')}}")
126
127     _dump(summary_path, summary)
128     return summary
129
130 def _season_table(records: list[dict]) -> str:
131     delivery = records[31:]
132     rows = ["season,days,total_cost_yuan,baseline_cost_yuan,emergency_energy_kwh,up_energy_kwh,down_energy_kwh,curtailment_kwh"]
133     for season in ("spring", "summer", "autumn", "winter"):
134         subset = [r for r in delivery if _season(r["date"]).month == season]
135         if not subset:
136             continue
137         rows.append(
138             ", ".join(
139                 str(x)
140                 for x in [
141                     season,
142                     len(subset),
143                     round(sum(r["total_cost"] for r in subset), 4),
144                     round(sum(r["baseline_cost"] for r in subset), 4),
145                     round(sum(r["execution"].emergency.sum() for r in subset), 4),
146                     round(sum(r["up_energy"] for r in subset), 4),
147                     round(sum(r["down_energy"] for r in subset), 4),
148                     round(sum(r["execution"].curtail.sum() for r in subset), 4),
149                 ]
150             )
151         )
152     return "\n".join(rows) + "\n"
153
154 def run_q4_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
155     out = VARIANTS / "q4"
156     prices = read_attachment4(data_root).price
157     summary = {}
158     for name, (kind, kwargs) in Q4_CASES.items():
159         started = time.perf_counter()
160         if kind == "q2":
161             result = run_q2(
162                 data_root=data_root,
163                 max_days=max_days,
164                 price_matrix=prices,

```

```

162         output_dir=out / name,
163         template_name="result4-2.xlsx",
164         write_outputs=True,
165         load_mode="causal",
166         tag=name,
167         **kwargs,
168     )
169     else:
170         result = run_q3(
171             data_root=data_root,
172             max_days=max_days,
173             price_matrix=prices,
174             output_dir=out / name,
175             template_name="result4-3.xlsx",
176             write_outputs=True,
177             load_mode="causal",
178             settlement_mode="replacement",
179             update_nodes=(0, 36, 72, 108),
180             tag=name,
181             **kwargs,
182         )
183     metrics = result["metrics"]
184     metrics["wall_seconds"] = time.perf_counter() - started
185     (out / name / "metrics.json").write_text(
186         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
187     )
188     summary[name] = metrics
189     print(f"[Q4] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
190     _dump(out / "summary.json", summary)
191     return summary
192 def run_q43_ablation(
193     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "replacement"
194 ) -> dict:
195     out = VARIANTS / "q4"
196     summary_path = out / "summary.json"
197     summary = json.loads(summary_path.read_text(encoding="utf-8")) if summary_path.exists()
198     else {}
199     prices = read_attachment4(data_root).price
200     selected = ("q43_perfect_price", "q43_causal_price")
201     for name in selected:
202         _, kwargs = Q4_CASES[name]
203         started = time.perf_counter()
204         result = run_q3(
205             data_root=data_root,

```

```

206         price_matrix=prices,
207         output_dir=out / name,
208         template_name="result4-3.xlsx",
209         write_outputs=True,
210         load_mode="causal",
211         settlement_mode=settlement_mode,
212         update_nodes=(0, 36, 72, 108),
213         tag=name,
214         **kwargs,
215     )
216     metrics = result["metrics"]
217     metrics["wall_seconds"] = time.perf_counter() - started
218     (out / name / "metrics.json").write_text(
219         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
220     )
221     summary[name] = metrics
222     print(f"[Q4-3] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get(
223         ('emergency_energy_kwh'))}")
224     _dump(summary_path, summary)
225     return {name: summary[name] for name in selected}
226 def run_sensitivity(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
227     out = VARIANTS / "sensitivity"
228     summary = {"terminal": {}, "efficiency": {}}
229     for policy in ("daily_cycle", "free"):
230         started = time.perf_counter()
231         result = run_q2(
232             data_root=data_root,
233             max_days=max_days,
234             output_dir=out / "terminal" / policy,
235             template_name="result2.xlsx",
236             write_outputs=True,
237             load_mode="causal",
238             pv_risk_mode="asymmetric",
239             realtime_feedback=True,
240             terminal_policy=policy,
241             tag=f"terminal_{policy}",
242         )
243         metrics = result["metrics"]
244         metrics["wall_seconds"] = time.perf_counter() - started
245         daily_soc = [float(r["execution"].soc[-1]) for r in result["records"][31:]]
246         metrics["daily_boundary_soc"] = {
247             "min": float(np.min(daily_soc)),
248             "max": float(np.max(daily_soc)),
249             "mean": float(np.mean(daily_soc)),
250             "std": float(np.std(daily_soc)),
251         }
252         (out / "terminal" / policy / "metrics.json").write_text(

```

```

252         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
253     )
254     summary["terminal"][policy] = metrics
255     print(f"[P1-terminal] {policy}: total={metrics.get('total_cost_yuan')} endSOC={
        metrics.get('ending_soc_kwh')}")
256 for name, storage in EFFICIENCY_VARIANTS.items():
257     started = time.perf_counter()
258     result = run_q2(
259         data_root=data_root,
260         max_days=max_days,
261         output_dir=out / "efficiency" / name,
262         template_name="result2.xlsx",
263         write_outputs=True,
264         load_mode="causal",
265         pv_risk_mode="asymmetric",
266         realtime_feedback=True,
267         storage=storage,
268         tag=f"efficiency_{name}",
269     )
270     metrics = result["metrics"]
271     metrics["wall_seconds"] = time.perf_counter() - started
272     (out / "efficiency" / name / "metrics.json").write_text(
273         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
274     )
275     summary["efficiency"][name] = metrics
276     print(f"[P1-efficiency] {name}: total={metrics.get('total_cost_yuan')}")
277     _dump(out / "summary.json", summary)
278     return summary
279 def _dump(path: Path, payload: dict) -> None:
280     path.parent.mkdir(parents=True, exist_ok=True)
281     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str), encoding
        ="utf-8")
282 def main() -> None:
283     run_q1()
284     run_q2_ablation()
285     run_q3_ablation(settlement_mode="replacement")
286     run_q3_ablation(settlement_mode="additive")
287     run_q4_ablation()
288     run_sensitivity()
289 if __name__ == "__main__":
290     main()

```

B.14 官方光伏预报处理 (official_forecast.py)

Listing 14:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS
5  QUANTILES=(0.80,0.85,0.90,0.95)
6  @dataclass(frozen=True)
7  class CalibrationDecision:
8      issue_index:int; quantile:float; margin_kw:float; history_days_used:int
9      raw_forecast_kw:np.ndarray; forecast_kw:np.ndarray; candidates:dict[float,np.ndarray]
10     historical_score:float|None
11  class OnlineForecastBiasCalibrator:
12     def __init__(self,residual_window:int=30,score_window:int=30):
13         self.residual_window=residual_window; self.score_window=score_window
14         self.residual_days=[[] for _ in range(4)]
15         self.score_days=[{q:[] for q in QUANTILES} for _ in range(4)]
16     def calibrate(self,issue_index:int,raw_forecast_kw:np.ndarray)->CalibrationDecision:
17         raw=np.asarray(raw_forecast_kw,dtype=float)
18         days=self.residual_days[issue_index][-self.residual_window:]
19         residual=np.concatenate(days) if days else np.array([])
20         margins={q:(max(0.0,float(np.quantile(residual,q))) if residual.size else 0.0) for q
21                  in QUANTILES}
22         candidates={q:np.clip(raw-margins[q],0.0,None) for q in QUANTILES}
23         history_count=len(self.residual_days[issue_index])
24         if history_count<7: q=0.80; hist=None
25         else:
26             scores={x:float(np.mean(self.score_days[issue_index][x][-self.score_window:]))
27                    for x in QUANTILES}
28             q=min(scores,key=lambda x:(scores[x],x)); hist=scores[q]
29         return CalibrationDecision(issue_index,q,margins[q],history_count,raw.copy(),
30                                   candidates[q],candidates,hist)
31     def observe(self,decision:CalibrationDecision,actual_block_kw:np.ndarray,price_block:np.
32                ndarray)->None:
33         actual=np.asarray(actual_block_kw,dtype=float); price=np.asarray(price_block,dtype=
34                                float)
35         if actual.shape!=(36,) or price.shape!=(36,): raise ValueError("calibration block
36                                must contain 36 slots")
37         issue=decision.issue_index
38         if decision.history_days_used!=len(self.residual_days[issue]): raise ValueError("
39                                stale calibration decision")
40         for q,forecast in decision.candidates.items():
41             error=forecast[:36]-actual
42             score=float(np.sum(price*(4*np.maximum(error,0)+np.maximum(-error,0))*DT_HOURS)
43                      self.score_days[issue][q].append(score)
44         raw_error=decision.raw_forecast_kw[:36]-actual
45         mask=(decision.raw_forecast_kw[:36]>1e-9)|(actual>1e-9)
46         self.residual_days[issue].append(raw_error[mask].copy())

```

B.15 指定日期论文表格生成 (paper_tables.py)

Listing 15:

```
1 from __future__ import annotations
2 import csv
3 from collections import defaultdict
4 from pathlib import Path
5 import numpy as np
6 import openpyxl
7 ROOT = Path(__file__).resolve().parents[1]
8 VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
9 GENERATED = ROOT / "paper" / "generated"
10 TEMPLATE = ROOT.parent / "CUMCM2026Problems" / "C题" / "附件" / "附件5" / "result2.xlsx"
11 DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
12 INTERVALS = ("10:00-10:10", "12:00-12:10", "14:00-14:10", "16:00-16:10", "18:00-18:10", "
13             20:00-20:10")
14 PERIODS = ("0:00--4:00", "4:00--8:00", "8:00--12:00", "12:00--16:00", "16:00--20:00", "
15             20:00--24:00")
16 SOURCES = {
17     "q2": VARIANTS / "q2" / "B_causalload_risk_feedback",
18     "q3": VARIANTS / "q3_replacement" / "S3_0_6_12_18",
19     "q42": VARIANTS / "q4" / "q42_perfect_price",
20     "q43": VARIANTS / "q4" / "q43_perfect_price",
21 }
22
23 def _rows(path: Path) -> list[dict[str, str]]:
24     with path.open(encoding="utf-8-sig", newline="") as handle:
25         return list(csv.DictReader(handle))
26
27 def _slot_indices() -> dict[str, int]:
28     workbook = openpyxl.load_workbook(TEMPLATE, data_only=True, read_only=True)
29     headers = list(next(workbook.worksheets[0].iter_rows(min_row=1, max_row=1, values_only=
30                     True)))
31     workbook.close()
32     return {label: headers.index(label) - 1 for label in INTERVALS}
33
34 def emergency_segments(values: np.ndarray, tol: float = 1e-7) -> list[tuple[str, float]]:
35     output: list[tuple[str, float]] = []
36     start = None
37
38     def stamp(slot: int) -> str:
39         minutes = slot * 10
40         return "24:00" if minutes == 1440 else f"{minutes // 60}:{minutes % 60:02d}"
41
42     for index, active in enumerate(np.r_[np.asarray(values, dtype=float) > tol, False]):
43         if active and start is None:
44             start = index
45         elif not active and start is not None:
46             output.append((f"{stamp(start)}--{stamp(index)}", float(np.sum(values[start:
47                                     index]))))
48             start = None
```

```

40     return output
41 def _dataset(key: str) -> tuple[dict[str, list[dict[str, str]]], dict[str, dict[str, str]]]:
42     grouped: dict[str, list[dict[str, str]]] = defaultdict(list)
43     for row in _rows(SOURCES[key] / "solution.csv"):
44         grouped[row["date"]].append(row)
45     daily = {row["date"]: row for row in _rows(SOURCES[key] / "daily_metrics.csv")}
46     missing = [date for date in DATES if date not in grouped or date not in daily]
47     if missing:
48         raise ValueError(f"{key} 缺少指定日期: {missing}")
49     return grouped, daily
50 def _purchase_table(key: str, caption: str, label: str, adjusted: bool) -> str:
51     grouped, daily = _dataset(key)
52     indices = _slot_indices()
53     lines = [
54         r"\begin{table}[htbp]",
55         r"\centering\zihao{-5}",
56         rf"\caption{{{caption}}}",
57         rf"\label{{{label}}}",
58         r"\resizebox{\textwidth}{!}{%",
59     ]
60     if adjusted:
61         lines += [
62             r"\begin{tabular}{c c r r r r r r r r}",
63             r"\toprule",
64             r"日期 & 类别 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 费用/",
65             r"元 \\",
66             r"\midrule",
67         ]
68     else:
69         lines += [
70             r"\begin{tabular}{c r r r r r r r r}",
71             r"\toprule",
72             r"日期 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 计划费/元 \\",
73             r"\midrule",
74         ]
75     for date in DATES:
76         day = grouped[date]
77         drow = daily[date]
78         if adjusted:
79             for row_index, (name, column, cost_field) in enumerate(
80                 (("计划", "baseline_grid_kwh", "baseline_cost"), ("调整", "adjusted_grid_kwh", "settlement_cost"))
81             ):
82                 values = [float(day[indices[item]][column]) for item in INTERVALS]
83                 total = sum(float(row[column]) for row in day)
84                 date_cell = rf"\multirow{{{2}}}{{{*}}}{{{date}}}" if row_index == 0 else ""

```



```

84         lines.append(
85             f"{date_cell} & {name} & " + " & ".join(f"{value:.2f}" for value in
            values)
86             + f" & {total:.2f} & {float(drow[cost_field]):.2f} \\\\"
87         )
88     else:
89         values = [float(day[indices[item]]["grid_plan_kwh"]) for item in INTERVALS]
90         total = sum(float(row["grid_plan_kwh"]) for row in day)
91         lines.append(
92             f"{date} & " + " & ".join(f"{value:.2f}" for value in values)
93             + f" & {total:.2f} & {float(drow['plan_cost']):.2f} \\\\"
94         )
95     lines += [r"\bottomrule", r"\end{tabular}}", r"\end{table}"]
96     return "\n".join(lines)
97 def _storage_table(key: str, caption: str, label: str) -> str:
98     grouped, daily = _dataset(key)
99     lines = [
100         r"\begin{table}[htbp]",
101         r"\centering\zihao{-5}",
102         rf"\caption{{{caption}}}",
103         rf"\label{{{label}}}",
104         r"\resizebox{\textwidth}{!}{%",
105         r"\begin{tabular}{c r r r r r r r}",
106         r"\toprule",
107         r"日期 & \multicolumn{6}{c}{各 4 h 时段充电量/放电量 (kWh) } & $$S_{0:00}$ & $$S_{24:00}$ \\",
108         r"\cmidrule(lr){2-7}",
109         r"日期 & " + " & ".join(PERIODS) + r" & kWh & kWh \\",
110         r"\midrule",
111     ]
112     for date in DATES:
113         day = grouped[date]
114         charge = np.asarray([float(row["charge_actual_kwh"]) for row in day])
115         discharge = np.asarray([float(row["discharge_actual_kwh"]) for row in day])
116         pairs = [f"{charge[i * 24:(i + 1) * 24].sum():.2f}/{discharge[i * 24:(i + 1) * 24].sum():.2f}" for i in range(6)]
117         lines.append(
118             f"{date} & " + " & ".join(pairs)
119             + f" & {float(daily[date]['soc_initial']):.2f} & {float(daily[date]['soc_terminal']):.2f} \\\\"
120         )
121     lines += [r"\bottomrule", r"\end{tabular}}", r"\end{table}"]
122     return "\n".join(lines)
123 def _emergency_table(key: str, caption: str, label: str) -> str:
124     grouped, daily = _dataset(key)
125     lines = [
126         r"\begin{table}[htbp]",

```

```

127     r"\centering\zihao{-5}",
128     rf"\caption{{{caption}}}",
129     rf"\label{{{label}}}",
130     r"\begin{tabularx}{0.98\textwidth}{c >{\raggedright\arraybackslash}X r r}",
131     r"\toprule",
132     r"日期 & 紧急购电时段及对应电量/kWh & 合计/kWh & 当日总费用/元 \\",
133     r"\midrule",
134 ]
135 for date in DATES:
136     day = grouped[date]
137     segments = emergency_segments(np.asarray([float(row["emergency_kwh"]) for row in day
138         ]))
139     description = "无" if not segments else "; ".join(f"{period} ({value:.2f})" for
140         period, value in segments)
141     total = sum(value for _, value in segments)
142     lines.append(f"{date} & {description} & {total:.2f} & {float(daily[date]['total_cost
143         ']):.2f} \\\\" )
144     lines += [r"\bottomrule", r"\end{tabularx}", r"\end{table}"]
145     return "\n".join(lines)
146 def _bundle(key: str, title: str, prefix: str, adjusted: bool) -> str:
147     return "\n\n".join(
148         (
149             _purchase_table(key, f"{title}指定日期购电结果", f"tab:{prefix}-purchase",
150                 adjusted),
151             _storage_table(key, f"{title}指定日期储能结果", f"tab:{prefix}-storage"),
152             _emergency_table(key, f"{title}指定日期紧急购电结果", f"tab:{prefix}-emergency")
153         ),
154         "\n"
155     )
156 def build() -> None:
157     GENERATED.mkdir(parents=True, exist_ok=True)
158     outputs = {
159         "q2_required_tables.tex": _bundle("q2", "问题二", "q2-required", False),
160         "q3_required_tables.tex": _bundle("q3", "问题三", "q3-required", True),
161         "q4_required_tables.tex": (
162             _bundle("q42", "问题四对应问题二", "q42-required", False)
163             + "\n"
164             + _bundle("q43", "问题四对应问题三", "q43-required", True)
165         ),
166     }
167     for name, content in outputs.items():
168         path = GENERATED / name
169         path.write_text("% 由 MathModel.solve.paper_tables 自动生成，勿手工修改。\\n" +
170             content, encoding="utf-8")
171         print("[table]", path)
172 if __name__ == "__main__":
173     build()

```

B.16 问题二结果验证 (validate_q2.py)

Listing 16:

```
1 from __future__ import annotations
2 import csv,json
3 from pathlib import Path
4 import numpy as np
5 import openpyxl
6 ROOT=Path(__file__).resolve().parents[1]
7 OUT=ROOT/"outputs"/"q2"
8 def validate()->dict:
9     with (OUT/"solution.csv").open(encoding="utf-8-sig",newline="") as f:
10         rows=list(csv.DictReader(f))
11         if len(rows)!=334*144: raise ValueError(f"solution rows: {len(rows)}")
12         grid=np.asarray([float(r["grid_plan_kwh"]) for r in rows]).reshape(334,144)
13         charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
14         discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
15         emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
16         with (OUT/"daily_metrics.csv").open(encoding="utf-8-sig",newline="") as f:
17             daily=list(csv.DictReader(f))
18             wb=openpyxl.load_workbook(OUT/"result2.xlsx",data_only=True,read_only=True)
19             template=openpyxl.load_workbook(ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"/"
20                 result2.xlsx",
21                 data_only=True,read_only=True)
22             plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
23                 values_only=True))
24             plan_values=np.asarray([r[:144] for r in plan_rows],dtype=float)
25             total_values=np.asarray([r[144] for r in plan_rows],dtype=float)
26             cost_values=np.asarray([r[145] for r in plan_rows],dtype=float)
27             storage_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=2005,min_col=1,max_col=6,
28                 values_only=True))
29             storage_charge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
30             storage_discharge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
31             storage_initial=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
32             storage_terminal=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
33             expected_charge=charge.reshape(334,6,24).sum(axis=2)
34             expected_discharge=discharge.reshape(334,6,24).sum(axis=2)
35             emergency_rows=list(wb.worksheets[2].iter_rows(min_row=2,min_col=1,max_col=3,values_only
36                 =True))
37             emergency_sheet_total=float(sum(float(r[2] or 0) for r in emergency_rows))
38             expected_initial=np.asarray([float(r["soc_initial"]) for r in daily])
39             expected_terminal=np.asarray([float(r["soc_terminal"]) for r in daily])
40             expected_cost=np.asarray([float(r["plan_cost"]) for r in daily])
```

```

37     result={
38         "sheet_names_preserved":wb.sheetnames==template.sheetnames,
39         "solution_rows":len(rows),"plan_shape":list(plan_values.shape),"storage_rows":len(
            storage_rows),
40         "max_plan_diff":float(np.max(np.abs(plan_values-grid))),
41         "max_plan_total_diff":float(np.max(np.abs(total_values-grid.sum(axis=1)))),
42         "max_plan_cost_diff":float(np.max(np.abs(cost_values-expected_cost))),
43         "max_storage_charge_diff":float(np.max(np.abs(storage_charge-expected_charge))),
44         "max_storage_discharge_diff":float(np.max(np.abs(storage_discharge-
            expected_discharge))),
45         "max_storage_initial_soc_diff":float(np.max(np.abs(storage_initial-expected_initial)
            )),
46         "max_storage_terminal_soc_diff":float(np.max(np.abs(storage_terminal-
            expected_terminal))),
47         "emergency_sheet_total_kwh":emergency_sheet_total,
48         "emergency_csv_total_kwh":float(emergency.sum()),
49         "emergency_total_diff":abs(emergency_sheet_total-float(emergency.sum()))),
50     }
51     result["passed"]=bool(result["sheet_names_preserved"] and result["plan_shape"
        ]==[334,144] and
52         len(storage_rows)==2004 and max(result[k] for k in result if k.startswith("max_")<1
            e-7 and
53         result["emergency_total_diff"]<1e-7)
54     (OUT/"xlsx_verification.json").write_text(json.dumps(result,ensure_ascii=False,indent=2)
        ,encoding="utf-8")
55     metrics=json.loads((OUT/"metrics.json").read_text(encoding="utf-8"))
56     metrics["xlsx_passed"]=result["passed"]
57     (OUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding
        ="utf-8")
58     return result
59 if __name__=="__main__": print(json.dumps(validate(),ensure_ascii=False,indent=2))

```

B.17 全部方案物理验证 (validate_all.py)

Listing 17:

```

1  from __future__ import annotations
2  import csv,json
3  from pathlib import Path
4  import numpy as np
5  import openpyxl
6  ROOT=Path(__file__).resolve().parents[1]
7  DATA=ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"
8  def _csv(path:Path)->list[dict]:
9      with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
10 def _maxdiff(a,b)->float: return float(np.max(np.abs(np.asarray(a,dtype=float)-np.asarray(b,

```

```

        dtype=float))))
11 def validate_q1()->dict:
12     folder=ROOT/"outputs"/"q1"; rows=_csv(folder/"solution.csv")
13     wb=openpyxl.load_workbook(folder/"result1.xlsx",data_only=True,read_only=True)
14     template=openpyxl.load_workbook(DATA/"result1.xlsx",data_only=True,read_only=True)
15     grid=np.asarray([float(r["grid_kwh"]) for r in rows])
16     xgrid=np.asarray(list(wb.worksheets[0].iter_rows(min_row=2,max_row=145,min_col=2,max_col
        =2,values_only=True)),dtype=float).ravel()
17     result={"rows":len(rows),"sheet_names_preserved":wb.sheetnames==template.sheetnames,
18             "max_grid_diff":_maxdiff(grid,xgrid)}
19     result["passed"]=bool(len(rows)==144 and result["sheet_names_preserved"] and result["
        max_grid_diff"]<1e-7)
20     return result
21 def validate_annual(folder:Path,xlsx_name:str,kind:str)->dict:
22     rows=_csv(folder/"solution.csv"); daily=_csv(folder/"daily_metrics.csv")
23     if len(rows)!=334*144 or len(daily)!=334: raise ValueError(f"{folder}: row count
        mismatch")
24     wb=openpyxl.load_workbook(folder/xlsx_name,data_only=True,read_only=True)
25     template=openpyxl.load_workbook(DATA/xlsx_name,data_only=True,read_only=True)
26     has_adjust=kind=="q3"; storage_index=2 if has_adjust else 1; emergency_index=3 if
        has_adjust else 2
27     plan_field="baseline_grid_kwh" if has_adjust else "grid_plan_kwh"
28     plan=np.asarray([float(r[plan_field]) for r in rows]).reshape(334,144)
29     charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
30     discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
31     emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
32     plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
        values_only=True))
33     xplan=np.asarray([r[:144] for r in plan_rows],dtype=float)
34     xttotal=np.asarray([r[144] for r in plan_rows],dtype=float)
35     xcost=np.asarray([r[145] for r in plan_rows],dtype=float)
36     expected_plan_cost=np.asarray([float(r["baseline_cost"]) if has_adjust else r["plan_cost"
        ]) for r in daily])
37     result={"solution_rows":len(rows),"daily_rows":len(daily),"sheet_names_preserved":wb.
        sheetnames==template.sheetnames,
38             "max_plan_diff":_maxdiff(xplan,plan),"max_plan_total_diff":_maxdiff(xttotal,plan.
        sum(axis=1)),
39             "max_plan_cost_diff":_maxdiff(xcost,expected_plan_cost)}
40     if has_adjust:
41         adjusted=np.asarray([float(r["adjusted_grid_kwh"]) for r in rows]).reshape(334,144)
42         adjust_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=335,min_col=2,max_col
            =147,values_only=True))
43         xadjust=np.asarray([r[:144] for r in adjust_rows],dtype=float)
44         xa_total=np.asarray([r[144] for r in adjust_rows],dtype=float)
45         xa_cost=np.asarray([r[145] for r in adjust_rows],dtype=float)
46         expected_cost=np.asarray([float(r["settlement_cost"]) for r in daily])
47         result.update({"max_adjust_diff":_maxdiff(xadjust,adjusted),

```

```

48         "max_adjust_total_diff":_maxdiff(xa_total,adjusted.sum(axis=1)),
49         "max_adjust_cost_diff":_maxdiff(xa_cost,expected_cost)})
50     storage_rows=list(wb.worksheets[storage_index].iter_rows(min_row=2,max_row=2005,min_col
        =1,max_col=6,values_only=True))
51     xcharge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
52     xdischarge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
53     xs0=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
54     xs1=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
55     result.update({"storage_rows":len(storage_rows),
56         "max_storage_charge_diff":_maxdiff(xcharge,charge.reshape(334,6,24).sum(axis=2)),
57         "max_storage_discharge_diff":_maxdiff(xdischarge,discharge.reshape(334,6,24).sum(
            axis=2)),
58         "max_storage_initial_diff":_maxdiff(xs0,[float(r["soc_initial"]) for r in daily]),
59         "max_storage_terminal_diff":_maxdiff(xs1,[float(r["soc_terminal"]) for r in daily])
        })
60     erows=list(wb.worksheets[emergency_index].iter_rows(min_row=2,min_col=1,max_col=3,
        values_only=True))
61     x_emergency=sum(float(r[2] or 0) for r in erows)
62     result.update({"emergency_rows":len(erows),"emergency_sheet_total":x_emergency,
63         "emergency_csv_total":float(emergency.sum()),
64         "emergency_total_diff":abs(x_emergency-float(emergency.sum()))})
65     numerical=[v for k,v in result.items() if k.startswith("max_") or k=="
        emergency_total_diff"]
66     result["passed"]=bool(result["sheet_names_preserved"] and len(storage_rows)==2004 and
        max(numerical)<1e-7)
67     return result
68 def validate_all()->dict:
69     cases={
70         "q1":validate_q1(),
71         "q2":validate_annual(ROOT/"outputs"/"q2","result2.xlsx","q2"),
72         "q3_main":validate_annual(ROOT/"outputs"/"q3","result3.xlsx","q3"),
73         "q3_raw_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"raw_step","result3.
            xlsx","q3"),
74         "q3_calibrated_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"calibrated_step
            ","result3.xlsx","q3"),
75         "q4_2":validate_annual(ROOT/"outputs"/"q4-2","result4-2.xlsx","q2"),
76         "q4_3":validate_annual(ROOT/"outputs"/"q4-3","result4-3.xlsx","q3"),
77     }
78     result={"cases":cases,"passed":all(v["passed"] for v in cases.values())}
79     (ROOT/"outputs"/"verification_summary.json").write_text(json.dumps(result,ensure_ascii=
        False,indent=2),encoding="utf-8")
80     return result
81 if __name__=="__main__": print(json.dumps(validate_all(),ensure_ascii=False,indent=2))

```

B.18 复现清单生成 (build_manifest.py)

Listing 18:

```

1  from __future__ import annotations
2  import hashlib,json
3  from datetime import datetime
4  from pathlib import Path
5  ROOT=Path(__file__).resolve().parents[1]
6  OUTPUTS=ROOT/"outputs"
7  VARIANTS={
8      "q1":{"path":"q1","parameters":{"question":1}},
9      "q2_fixed_online":{"path":"q2","parameters":{"question":2,"price":"attachment1","
10         forecast":"strictly_online"}},
11      "q3_main_linear_calibrated":{"path":"q3","parameters":{"question":3,"downscale":"linear"
12         ,"calibration":True,"price":"attachment1"}},
13      "q3_raw_step":{"path":"q3/variants/raw_step","parameters":{"question":3,"downscale":"
14         step","calibration":False,"price":"attachment1"}},
15      "q3_calibrated_step":{"path":"q3/variants/calibrated_step","parameters":{"question":3,"
16         downscale":"step","calibration":True,"price":"attachment1"}},
17      "q4_2_dynamic":{"path":"q4-2","parameters":{"question":"4-2","price":"attachment4","
18         forecast":"strictly_online"}},
19      "q4_3_dynamic":{"path":"q4-3","parameters":{"question":"4-3","price":"attachment4","
20         downscale":"linear","calibration":True}},
21  }
22  def digest(path:Path)->str:
23      h=hashlib.sha256()
24      with path.open("rb") as f:
25          for block in iter(lambda:f.read(1024*1024),b""): h.update(block)
26      return h.hexdigest()
27  def build()->dict:
28      result={"generated_at":datetime.now().astimezone().isoformat(),"authority_root":str(ROOT
29          ),
30          "main":{"q1":"q1","q2":"q2_fixed_online","q3":"q3_main_linear_calibrated",
31          "q4_2":"q4_2_dynamic","q4_3":"q4_3_dynamic"},"variants":{},"source_files
32          ":{}}
33      for name,spec in VARIANTS.items():
34          folder=OUTPUTS/spec["path"]; files={}
35          for path in sorted(folder.glob("*")):
36              if path.is_file():
37                  files[path.name]={"bytes":path.stat().st_size,"sha256":digest(path)}
38          metrics_path=folder/"metrics.json"
39          metrics=json.loads(metrics_path.read_text(encoding="utf-8")) if metrics_path.exists
40              () else None
41          result["variants"][name]={"relative_path":spec["path"],"parameters":spec["parameters
42              "],
43              "metrics":metrics,"files":files}
44      for path in sorted((ROOT/"solve").glob("*.py")):
45          result["source_files"][str(path.relative_to(ROOT))]={"bytes":path.stat().st_size,"

```

```

        sha256":digest(path)}
36 for pattern in ("reports/*.md","tests/*.py","README.md","requirements.txt","run_all.ps1"
    ):
37     for path in sorted(ROOT.glob(pattern)):
38         if path.is_file(): result["source_files"][str(path.relative_to(ROOT))]={ "bytes":
            path.stat().st_size,"sha256":digest(path)}
39 result["figures"]={}
40 for path in sorted((ROOT/"figures").rglob("*.pdf")):
41     result["figures"][str(path.relative_to(ROOT))]={ "bytes":path.stat().st_size,"sha256"
        :digest(path)}
42 result["governance_outputs"]={}
43 for name in ("verification_summary.json","q4_comparison.json"):
44     path=OUTPUTS/name
45     if path.exists(): result["governance_outputs"][name]={ "bytes":path.stat().st_size,"
        sha256":digest(path)}
46 (OUTPUTS/"MANIFEST.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),
    encoding="utf-8")
47 return result
48 if __name__=="__main__":
49     manifest=build(); print(json.dumps({"variants":list(manifest["variants"]),
50         "source_files":len(manifest["source_files"]),"manifest":str(OUTPUTS/"MANIFEST.json")
        },ensure_ascii=False,indent=2))

```

B.19 科研绘图公共样式 (plot_style.py)

Listing 19:

```

1 from __future__ import annotations
2 from pathlib import Path
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6 FIGURES = Path(__file__).resolve().parents[1] / "paper" / "figures"
7 FIGURES.mkdir(parents=True, exist_ok=True)
8 PALETTE = {
9     "navy": "#1F3B73",
10    "blue": "#3E6FB0",
11    "sky": "#7FB2E5",
12    "teal": "#2E8B84",
13    "green": "#5FA05A",
14    "olive": "#9BAF3B",
15    "gold": "#D8A22B",
16    "orange": "#E07B39",
17    "red": "#C0392B",
18    "rose": "#D46A8C",
19    "purple": "#7A5BA6",

```



```

20     "gray": "#7F8C99",
21     "lightgray": "#D5DBE1",
22     "ink": "#1A2028",
23 }
24 CYCLE = [PALETTE[k] for k in ("navy", "orange", "teal", "red", "purple", "gold", "sky", "
    green")]
25 SCENARIO_COLORS = {
26     "Q1": PALETTE["navy"],
27     "Q2": PALETTE["orange"],
28     "Q3": PALETTE["teal"],
29     "Q4-2": PALETTE["red"],
30     "Q4-3": PALETTE["purple"],
31     "legacy": PALETTE["gray"],
32 }
33 def apply_style() -> None:
34     plt.rcParams.update(
35         {
36             "font.family": "sans-serif",
37             "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
38             "font.serif": ["SimSun", "Times New Roman"],
39             "axes.unicode_minus": False,
40             "font.size": 11,
41             "axes.titlesize": 12.5,
42             "axes.labelsize": 11.5,
43             "axes.titleweight": "bold",
44             "legend.fontsize": 10,
45             "xtick.labelsize": 10,
46             "ytick.labelsize": 10,
47             "axes.linewidth": 0.9,
48             "axes.edgecolor": PALETTE["ink"],
49             "axes.labelcolor": PALETTE["ink"],
50             "text.color": PALETTE["ink"],
51             "xtick.color": PALETTE["ink"],
52             "ytick.color": PALETTE["ink"],
53             "axes.grid": True,
54             "grid.color": PALETTE["lightgray"],
55             "grid.linewidth": 0.6,
56             "grid.alpha": 0.75,
57             "axes.axisbelow": True,
58             "figure.dpi": 130,
59             "savefig.dpi": 330,
60             "savefig.bbox": "tight",
61             "savefig.facecolor": "white",
62             "figure.facecolor": "white",
63             "axes.spines.top": False,
64             "axes.spines.right": False,
65             "legend.frameon": False,

```

```

66         "lines.linewidth": 1.8,
67         "lines.markersize": 4.5,
68     }
69 )
70 def panel_label(ax, text: str, dx: float = -0.085, dy: float = 1.06) -> None:
71     ax.text(dx, dy, text, transform=ax.transAxes, fontsize=12.5, fontweight="bold",
72           va="bottom", ha="left", color=PALETTE["ink"])
73 def save(fig, name: str, *, pdf: bool = True) -> Path:
74     png = FIGURES / f"{name}.png"
75     fig.savefig(png)
76     if pdf:
77         fig.savefig(FIGURES / f"{name}.pdf")
78     plt.close(fig)
79     return png
80 def wan(value: float) -> float:
81     return value / 1e4

```

B.20 数据总览绘图 (fig_data.py)

Listing 20:

```

1  from __future__ import annotations
2  import sys
3  from pathlib import Path
4  import numpy as np
5  sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
6  from MathModel.solve.io_data import (
7      DEFAULT_DATA_ROOT,
8      DT_HOURS,
9      hourly_to_ten_min,
10     read_attachment1,
11     read_attachment2,
12     read_attachment3,
13     read_attachment4,
14 )
15 from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save
16 apply_style()
17 import matplotlib.dates as mdates
18 import matplotlib.pyplot as plt
19 import seaborn as sns
20 from matplotlib.colors import TwoSlopeNorm
21 MONTH_EDGES = np.cumsum([0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31])
22 MONTH_LABELS = [f"{m}月" for m in range(1, 13)]
23 DAYS_PER_MONTH = np.diff(MONTH_EDGES)
24 def fig_data_overview(root: Path = DEFAULT_DATA_ROOT) -> None:
25     day1 = read_attachment1(root)

```

```

26     annual = read_attachment2(root)
27     price = read_attachment4(root)
28     fig = plt.figure(figsize=(13.6, 8.2))
29     gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.22)
30     ax = fig.add_subplot(gs[0, 0])
31     hours = (np.arange(144) + 1) / 6.0
32     ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], label="小区负载")
33     ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], label="光伏发电预测")
34     ax.set_xlabel("时刻 / h")
35     ax.set_ylabel("功率 / MW")
36     ax.set_title("典型日负载与光伏出力的错配")
37     ax.set_xlim(0, 24)
38     ax.set_xticks(range(0, 25, 4))
39     ax2 = ax.twinx()
40     ax2.plot(hours, day1.price, color=PALETTE["red"], linestyle="--", linewidth=1.5, label="
    外网电价")
41     ax2.set_ylabel("电价 / (元·kWh$^{-1}$)", color=PALETTE["red"])
42     ax2.tick_params(axis="y", colors=PALETTE["red"])
43     ax2.grid(False)
44     ax2.spines["right"].set_visible(True)
45     handles1, labels1 = ax.get_legend_handles_labels()
46     handles2, labels2 = ax2.get_legend_handles_labels()
47     ax.legend(handles1 + handles2, labels1 + labels2, loc="upper left", ncol=1, fontsize=9)
48     panel_label(ax, "(a)")
49     for idx, (data, title, cmap, unit) in enumerate(
50         [
51             (annual.load_kw, "全年负载分布", "YlGnBu", "kW"),
52             (annual.pv_kw, "全年光伏出力分布", "YlOrBr", "kW"),
53             (price.price, "全年实时电价分布", "RdYlBu_r", "元/kWh"),
54         ]
55     ):
56         ax = fig.add_subplot(gs[(1 + idx) // 2, (1 + idx) % 2])
57         im = ax.imshow(
58             data / (1e3 if unit == "kW" else 1),
59             aspect="auto",
60             cmap=cmap,
61             interpolation="nearest",
62             extent=[0, 24, 365, 1],
63         )
64         ax.set_xlabel("时刻 / h")
65         ax.set_ylabel("日期 (第 n 天)")
66         ax.set_title(title)
67         ax.set_xticks(range(0, 25, 4))
68         ax.grid(False)
69         cb = fig.colorbar(im, ax=ax, pad=0.02)
70         cb.set_label("MW" if unit == "kW" else "元/kWh", fontsize=9)
71         cb.ax.tick_params(labelsize=8.5)

```

```

72     panel_label(ax, "%(s)" % "bcd"[idx])
73     save(fig, "data_overview")
74 def fig_netload_season(root: Path = DEFAULT_DATA_ROOT) -> None:
75     annual = read_attachment2(root)
76     net = annual.load_kw - annual.pv_kw
77     month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
78     fig = plt.figure(figsize=(13.6, 4.4))
79     gs = fig.add_gridspec(1, 3, wspace=0.28)
80     ax = fig.add_subplot(gs[0, 0])
81     data = [net[month_of_day == m].ravel() / 1e3 for m in range(12)]
82     bp = ax.boxplot(data, patch_artist=True, widths=0.62, showfliers=False,
83                     medianprops=dict(color=PALETTE["ink"], linewidth=1.3),
84                     whiskerprops=dict(color=PALETTE["gray"]),
85                     capprops=dict(color=PALETTE["gray"]),
86                     boxprops=dict(facecolor=PALETTE["sky"], edgecolor=PALETTE["navy"],
87                                   linewidth=0.9))
88     ax.axhline(0, color=PALETTE["red"], linewidth=1.1, linestyle="--")
89     ax.set_xticks(range(1, 13))
90     ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
91     ax.set_xlabel("月份")
92     ax.set_ylabel("净负荷 / MW")
93     ax.set_title("净负荷的月度分布")
94     panel_label(ax, "(a)")
95     ax = fig.add_subplot(gs[0, 1])
96     penetration = annual.pv_kw / np.maximum(annual.load_kw, 1e-9)
97     data = [np.clip(penetration[month_of_day == m].ravel(), 0, 3) for m in range(12)]
98     parts = ax.violinplot(data, showmedians=True, widths=0.85)
99     for body in parts["bodies"]:
100         body.set_facecolor(PALETTE["gold"])
101         body.set_edgecolor(PALETTE["orange"])
102         body.set_alpha(0.75)
103     for key in ("cmmedians", "cbars", "cmins", "cmaxes"):
104         parts[key].set_color(PALETTE["ink"])
105         parts[key].set_linewidth(1.1)
106     ax.axhline(1.0, color=PALETTE["red"], linestyle="--", linewidth=1.1)
107     ax.set_xticks(range(1, 13))
108     ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
109     ax.set_xlabel("月份")
110     ax.set_ylabel("光伏/负载 比值")
111     ax.set_title("光伏渗透率的月度分布")
112     ax.set_ylim(0, 3)
113     panel_label(ax, "(b)")
114     ax = fig.add_subplot(gs[0, 2])
115     surplus_slots = (annual.pv_kw - annual.load_kw > 0).sum(axis=1)
116     big_surplus = (annual.pv_kw - annual.load_kw > 5000).sum(axis=1)
117     hours_surplus = np.array([surplus_slots[month_of_day == m].mean() * 10 / 60 for m in
118                               range(12)])

```

```

117     hours_big = np.array([big_surplus[month_of_day == m].mean() * 10 / 60 for m in range(12)
118                           ])
119     x = np.arange(12)
120     ax.bar(x - 0.2, hours_surplus, width=0.4, color=PALETTE["green"], label="光伏富余时段")
121     ax.bar(x + 0.2, hours_big, width=0.4, color=PALETTE["red"], label="富余超 5000 kW 时段")
122     ax.set_xticks(x)
123     ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
124     ax.set_xlabel("月份")
125     ax.set_ylabel("日均时长 / h")
126     ax.set_title("光伏富余的月度强度")
127     ax.legend(fontsize=9)
128     panel_label(ax, "(c)")
129     save(fig, "netload_season")
130
131 def fig_forecast_diagnosis(root: Path = DEFAULT_DATA_ROOT) -> None:
132     annual = read_attachment2(root)
133     forecasts = read_attachment3(root)
134     month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
135     err = np.zeros_like(forecasts.hourly_kw)
136     for node in range(4):
137         for hour in range(24):
138             target_slot = (node * 36 + (hour + 1) * 6 - 1) % 144
139             target_day = np.arange(365) + (node * 36 + (hour + 1) * 6 - 1) // 144
140             valid = target_day < 365
141             err[valid, node, hour] = forecasts.hourly_kw[valid, node, hour] - annual.pv_kw[
142                 target_day[valid], target_slot]
143
144     fig = plt.figure(figsize=(13.6, 4.6))
145     gs = fig.add_gridspec(1, 3, wspace=0.28)
146     ax = fig.add_subplot(gs[0, 0])
147     labels = ["0:00", "6:00", "12:00", "18:00"]
148     colors = [PALETTE["navy"], PALETTE["orange"], PALETTE["teal"], PALETTE["purple"]]
149     for n, (label, color) in enumerate(zip(labels, colors)):
150         values = np.sort(np.abs(err[:, n, :].ravel()))
151         values = np.maximum(values, 1e-3)
152         share = np.arange(1, values.size + 1) / values.size
153         step = max(1, values.size // 4000)
154         ax.plot(values[:step], share[:step], color=color, linewidth=2.1, label=label)
155         p95 = np.percentile(values, 95)
156         ax.plot([p95], [0.95], marker="o", color=color, markersize=6,
157                 markeredgewidth=1.0, zorder=5)
158         ax.axhline(0.95, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
159         ax.text(1.6, 0.955, f"$P_{95}$", fontsize=9, color=PALETTE["gray"])
160         ax.set_xscale("log")
161         ax.set_xlim(1, 6000)
162         ax.set_ylim(0, 1.0)
163         ax.set_xlabel("绝对预报误差 / kW (对数轴)")
164         ax.set_ylabel("经验累积分布 ECDF")
165         ax.set_title("分发布时刻的预报误差累积分布")

```

```

162 ax.legend(fontsize=9, loc="lower right", title="发布时刻", title_fontsize=9)
163 panel_label(ax, "(a)")
164 ax = fig.add_subplot(gs[0, 1])
165 mae = [np.mean(np.abs(err[month_of_day == m])) for m in range(12)]
166 bias = [np.mean(err[month_of_day == m]) for m in range(12)]
167 x = np.arange(12)
168 ax.bar(x, mae, color=PALETTE["sky"], edgecolor=PALETTE["navy"], linewidth=0.7, label="
    MAE")
169 ax.plot(x, bias, color=PALETTE["red"], marker="o", markersize=4.5, label="平均偏差")
170 ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
171 ax.set_xticks(x)
172 ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
173 ax.set_xlabel("月份")
174 ax.set_ylabel("误差 / kW")
175 ax.set_title("预报误差的月度演变")
176 ax.legend(fontsize=9)
177 panel_label(ax, "(b)")
178 ax = fig.add_subplot(gs[0, 2])
179 raw_step, raw_linear = [], []
180 for d in range(365):
181     for n in range(4):
182         lo = n * 36
183         hi = lo + 36
184         actual = annual.pv_kw[d, lo:hi]
185         raw_step.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "
            step"))[:36] - actual)))
186         raw_linear.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "
            linear"))[:36] - actual)))
187 mae_step, mae_linear = float(np.mean(raw_step)), float(np.mean(raw_linear))
188 bars = ax.bar(["阶梯保持", "线性插值"], [mae_step, mae_linear],
189             color=[PALETTE["gray"], PALETTE["teal"]], width=0.5,
190             edgecolor=PALETTE["ink"], linewidth=0.8)
191 for bar, value in zip(bars, [mae_step, mae_linear]):
192     ax.text(bar.get_x() + bar.get_width() / 2, value * 1.02, f"{value:.1f} kW",
193         ha="center", va="bottom", fontsize=10, fontweight="bold")
194 ax.set_ylabel("执行块 MAE / kW")
195 ax.set_title("小时预报降尺度方法的精度对比")
196 ax.set_ylim(0, max(mae_step, mae_linear) * 1.25)
197 panel_label(ax, "(c)")
198 save(fig, "forecast_diagnosis")
199 print(f"[fig] 降尺度执行块 MAE: 阶梯={mae_step:.4f} kW, 线性={mae_linear:.4f} kW")
200 def fig_correlation(root: Path = DEFAULT_DATA_ROOT) -> None:
201     annual = read_attachment2(root)
202     price = read_attachment4(root)
203     load = annual.load_kw.ravel()
204     pv = annual.pv_kw.ravel()
205     net = load - pv

```

```

206 p = price.price.ravel()
207 slot_phase = np.tile(np.arange(144) / 144.0, 365)
208 frame = {
209     "小区负载": load, "光伏出力": pv, "净负荷": net,
210     "实时电价": p, "日内相位": slot_phase,
211 }
212 keys = list(frame)
213 matrix = np.corrcoef(np.stack([frame[k] for k in keys]))
214 matrix = matrix / np.sqrt(np.outer(np.diag(matrix), np.diag(matrix)))
215 fig = plt.figure(figsize=(13.0, 4.8))
216 gs = fig.add_gridspec(1, 2, width_ratios=[1.0, 1.15], wspace=0.3)
217 ax = fig.add_subplot(gs[0, 0])
218 im = ax.imshow(matrix, cmap="RdBu_r", vmin=-1, vmax=1)
219 ax.set_xticks(range(len(keys)))
220 ax.set_xticklabels(keys, rotation=32, ha="right", fontsize=9)
221 ax.set_yticks(range(len(keys)))
222 ax.set_yticklabels(keys, fontsize=9)
223 for i in range(len(keys)):
224     for j in range(len(keys)):
225         ax.text(j, i, f"{matrix[i, j]:.2f}", ha="center", va="center", fontsize=9.5,
226                 color="white" if abs(matrix[i, j]) > 0.55 else PALETTE["ink"])
227 ax.grid(False)
228 ax.set_title("关键变量的相关系数矩阵")
229 cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.9)
230 cb.set_label("Pearson 相关系数", fontsize=9)
231 panel_label(ax, "(a)")
232 ax = fig.add_subplot(gs[0, 1])
233 sample = np.random.default_rng(0).choice(load.size, size=min(50000, load.size), replace=
        False)
234 hb = ax.hexbin(load[sample] / 1e3, net[sample] / 1e3, gridsize=48, cmap="viridis",
235                bins="log", mincnt=1, linewidths=0)
236 ax.axhline(0, color=PALETTE["red"], linestyle="--", linewidth=1.2)
237 ax.set_xlabel("小区负载 / MW")
238 ax.set_ylabel("净负荷 / MW")
239 ax.set_title("负载—净负荷联合分布（含零线）")
240 cb = fig.colorbar(hb, ax=ax, pad=0.02, shrink=0.9)
241 cb.set_label("时段数（对数）", fontsize=9)
242 panel_label(ax, "(b)")
243 save(fig, "correlation_structure")
244 def main() -> None:
245     root = DEFAULT_DATA_ROOT
246     fig_data_overview(root)
247     fig_netload_season(root)
248     fig_forecast_diagnosis(root)
249     fig_correlation(root)
250     print("[fig] 数据层配图完成 →", (Path(__file__).resolve().parents[1] / "paper" / "
        figures"))

```

```

251 if __name__ == "__main__":
252     main()

```

B.21 问题一绘图 (fig_q1.py)

Listing 21:

```

1 from __future__ import annotations
2 import sys
3 from pathlib import Path
4 import numpy as np
5 import pandas as pd
6 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
7 from MathModel.solve.io_data import DT_HOURS, read_attachment1
8 from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save
9 apply_style()
10 import matplotlib.pyplot as plt
11 ROOT = Path(__file__).resolve().parents[1]
12 def main() -> None:
13     frame = pd.read_csv(ROOT / "outputs" / "q1" / "solution.csv", encoding="utf-8-sig")
14     day1 = read_attachment1()
15     hours = frame["slot"].to_numpy() / 6.0
16     fig = plt.figure(figsize=(13.6, 7.6))
17     gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
18     ax = fig.add_subplot(gs[0, 0])
19     ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], linewidth=1.9, label="小区负载")
20     ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], linewidth=1.9, label="光伏预测出力")
21     ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
22                     where=(day1.pv_kw < day1.load_kw), color=PALETTE["sky"], alpha=0.35,
23                     label="净负荷缺口")
24     ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
25                     where=(day1.pv_kw >= day1.load_kw), color=PALETTE["green"], alpha=0.32,
26                     label="光伏富余")
27     ax.set_xlabel("时刻 / h")
28     ax.set_ylabel("功率 / MW")
29     ax.set_title("典型日净负荷结构")
30     ax.set_xlim(0, 24)
31     ax.legend(fontsize=8.5, ncol=2)
32     panel_label(ax, "(a)")
33     ax = fig.add_subplot(gs[0, 1])
34     no_storage = np.maximum((day1.load_kw - day1.pv_kw) * DT_HOURS, 0)
35     ax.plot(hours, frame["grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth=1.9, label="含储能最优购电")
36     ax.plot(hours, no_storage * 6 / 1e3, color=PALETTE["gray"], linewidth=1.6, linestyle="--")

```



```

    ", label="无储能基准购电")
35 ax.fill_between(hours, frame["grid_kwh"] * 6 / 1e3, no_storage * 6 / 1e3,
36                 where=(no_storage > frame["grid_kwh"]), color=PALETTE["teal"], alpha
                 =0.25, label="储能削减的购电")
37 ax.set_xlabel("时刻 / h")
38 ax.set_ylabel("购电功率 / MW")
39 ax.set_title("储能对购电曲线的削峰填谷")
40 ax.set_xlim(0, 24)
41 ax.legend(fontsize=8.5)
42 panel_label(ax, "(b)")
43 ax = fig.add_subplot(gs[1, 0])
44 ax.bar(hours, frame["charge_kwh"], width=0.13, color=PALETTE["purple"],
45         edgecolor=PALETTE["ink"], linewidth=0.5, label="充电量")
46 ax.bar(hours, -frame["discharge_kwh"], width=0.13, color=PALETTE["teal"],
47         edgecolor=PALETTE["ink"], linewidth=0.5, label="放电量")
48 ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
49 ax.axhline(833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
50 ax.axhline(-833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
51 ax.text(0.2, 860, "单时段功率上限 833.33 kWh", fontsize=8, color=PALETTE["red"])
52 ax.set_xlabel("时刻 / h")
53 ax.set_ylabel("电量 / kWh (10 min)")
54 ax.set_title("储能充放电计划")
55 ax.set_xlim(0, 24)
56 ax.legend(fontsize=9)
57 panel_label(ax, "(c)")
58 ax = fig.add_subplot(gs[1, 1])
59 ax.plot(hours, frame["soc_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.2)
60 ax.fill_between(hours, frame["soc_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha=0.3)
61 ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
62 ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
63 ax.axhline(6.0, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
64 ax.text(0.2, 1.35, "下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
65 ax.text(0.2, 10.95, "上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
66 ax.text(0.2, 6.15, "首末储电量 6000 kWh", fontsize=8.5, color=PALETTE["gray"])
67 ax.set_xlabel("时刻 / h")
68 ax.set_ylabel("储电量 / MWh")
69 ax.set_title("储电量日内轨迹 (首末闭合)")
70 ax.set_xlim(0, 24)
71 ax.set_ylim(0.5, 12.2)
72 panel_label(ax, "(d)")
73 save(fig, "q1_dispatch")
74 print("[fig] q1_dispatch 完成")
75 if __name__ == "__main__":
76     main()

```

B.22 结果与消融绘图 (fig_results.py)

Listing 22:

```
1 from __future__ import annotations
2 import json
3 import sys
4 from pathlib import Path
5 import numpy as np
6 import pandas as pd
7 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
8 from MathModel.solve.plot_style import (
9     CYCLE,
10     PALETTE,
11     SCENARIO_COLORS,
12     apply_style,
13     panel_label,
14     save,
15 )
16 apply_style()
17 import matplotlib.pyplot as plt
18 ROOT = Path(__file__).resolve().parents[1]
19 VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
20 FIGS = ROOT / "paper" / "figures"
21 Q2_ORDER = ["A_perfectload_risk_feedback", "B_causalload_risk_feedback",
22             "C_causalload_point_feedback", "D_causalload_risk_nofeedback"]
23 Q2_LABEL = {
24     "A_perfectload_risk_feedback": "A 完美负载\n风险边际•有反馈",
25     "B_causalload_risk_feedback": "B 因果负载\n风险边际•有反馈",
26     "C_causalload_point_feedback": "C 因果负载\n点预测•有反馈",
27     "D_causalload_risk_nofeedback": "D 因果负载\n风险边际•无反馈",
28 }
29 Q3_ORDER = ["S0_0", "S1_0_6", "S2_0_6_12", "S3_0_6_12_18"]
30 Q3_LABEL = {"S0_0": "S0\n仅0:00", "S1_0_6": "S1\n+6:00", "S2_0_6_12": "S2\n+12:00", "
31             S3_0_6_12_18": "S3\n+18:00"}
32 Q4_ORDER = ["q42_perfect_price", "q42_causal_price", "q43_perfect_price", "q43_causal_price"
33             ]
34 Q4_LABEL = {"q42_perfect_price": "Q4-2\n附件4直用", "q42_causal_price": "Q4-2\n因果扩展",
35             "q43_perfect_price": "Q4-3\n附件4直用", "q43_causal_price": "Q4-3\n因果扩展"}
36 def load(path: Path) -> dict:
37     return json.loads(path.read_text(encoding="utf-8"))
38 def load_group(*parts: str) -> dict:
39     *dirs, stem = parts
40     return load(VARIANTS.joinpath(*dirs) / f"{stem}.json")
41 def _bar_labels(ax, bars, values, fmt="{:.2f}", dy=0.012, fontsize=9.5):
42     span = ax.get_ylim()[1] - ax.get_ylim()[0]
43     for bar, value in zip(bars, values):
```

```

42         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + span * dy,
43                 fmt.format(value), ha="center", va="bottom", fontsize=fontsize, fontweight="
                    bold")
44 def fig_q2_attribution() -> None:
45     summary = load_group("q2", "summary")
46     totals = [summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER]
47     emergencies = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q2_ORDER]
48     curtails = [summary[k]["curtailment_energy_kwh"] / 1e3 for k in Q2_ORDER]
49     c = {k: summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER}
50     fig = plt.figure(figsize=(13.8, 8.6))
51     gs = fig.add_gridspec(2, 2, hspace=0.48, wspace=0.26)
52     ax = fig.add_subplot(gs[0, 0])
53     colors = [PALETTE["blue"], PALETTE["teal"], PALETTE["yellow"] if "yellow" in PALETTE
                    else PALETTE["gold"], PALETTE["red"]]
54     bars = ax.bar(range(4), totals, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8,
                    width=0.62)
55     ax.set_xticks(range(4))
56     ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
57     ax.set_ylabel("全年总费用 / 万元")
58     ax.set_title("问题二四种信息集/机制组合的总费用")
59     ax.set_ylim(0, max(totals) * 1.16)
60     _bar_labels(ax, bars, totals, "{:.1f}")
61     panel_label(ax, "(a)")
62     ax = fig.add_subplot(gs[0, 1])
63     x = np.arange(4)
64     bars = ax.bar(x - 0.19, emergencies, width=0.38, color=PALETTE["orange"],
65                 edgecolor=PALETTE["ink"], linewidth=0.7, label="紧急购电")
66     bars2 = ax.bar(x + 0.19, curtails, width=0.38, color=PALETTE["sky"],
67                 edgecolor=PALETTE["ink"], linewidth=0.7, label="弃光")
68     ax.set_xticks(x)
69     ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
70     ax.set_ylabel("电量 / MWh")
71     ax.set_title("紧急购电与弃光电量")
72     ax.legend(fontsize=9)
73     _bar_labels(ax, bars, emergencies, "{:.0f}")
74     panel_label(ax, "(b)")
75     ax = fig.add_subplot(gs[1, 0])
76     delta_load = c["B_causalload_risk_feedback"] - c["A_perfectload_risk_feedback"]
77     delta_pv = c["C_causalload_point_feedback"] - c["B_causalload_risk_feedback"]
78     delta_fb = c["D_causalload_risk_nofeedback"] - c["B_causalload_risk_feedback"]
79     labels = ["完美负载信息\n$\Delta C_{load}$", "去掉光伏风险边际\n$\Delta C_{PVrisk}$",
                    "去掉实时储能反馈\n$\Delta C_{feedback}$"]
80     values = [delta_load, delta_pv, delta_fb]
81     bars = ax.bar(range(3), values, width=0.55,
82                 color=[PALETTE["red"], PALETTE["orange"], PALETTE["gold"]],
83                 edgecolor=PALETTE["ink"], linewidth=0.8)
84     _bar_labels(ax, bars, values, "{:+.1f}")

```

```

85     for i, value in enumerate(values):
86         share = 100 * value / c["B_causalload_risk_feedback"]
87         ax.text(i, value * 0.5, f"占因果基准\n{share:.1f}%", ha="center", va="center",
88                 fontsize=9, color="white", fontweight="bold")
89     ax.set_xticks(range(3))
90     ax.set_xticklabels(labels, fontsize=8.8)
91     ax.set_ylabel("费用增量 / 万元")
92     ax.set_title("三项机制各自的费用贡献 (万元)")
93     ax.set_ylim(0, max(values) * 1.24)
94     panel_label(ax, "(c)")
95     ax = fig.add_subplot(gs[1, 1])
96     rows = []
97     for key in Q2_ORDER:
98         m = summary[key]
99         rows.append([m["plan_purchase_cost_yuan"] / 1e4, m["emergency_cost_yuan"] / 1e4])
100    rows = np.asarray(rows)
101    ax.bar(range(4), rows[:, 0], width=0.62, color=PALETTE["navy"], label="计划购电费",
102           edgecolor=PALETTE["ink"], linewidth=0.8)
103    ax.bar(range(4), rows[:, 1], bottom=rows[:, 0], width=0.62, color=PALETTE["orange"],
104           label="紧急购电费", edgecolor=PALETTE["ink"], linewidth=0.8)
105    for i, (plan, em) in enumerate(rows):
106        ax.text(i, plan / 2, f"{plan:.0f}", ha="center", va="center", color="white",
107                fontsize=9.5, fontweight="bold")
108        ax.text(i, plan + em + max(rows[:, 0]) * 0.012, f"{plan + em:.1f}", ha="center", va=
109                "bottom",
110                fontsize=9.5, fontweight="bold")
111    ax.set_xticks(range(4))
112    ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
113    ax.set_ylabel("费用 / 万元")
114    ax.set_title("费用构成：计划购电与紧急购电")
115    ax.set_ylim(0, (rows.sum(axis=1).max()) * 1.16)
116    ax.legend(fontsize=9)
117    panel_label(ax, "(d)")
118    save(fig, "q2_attribution")
119
120    def fig_q3_forecast_value() -> None:
121        summary = load_group("q3_replacement", "summary")
122        costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER])
123        fig = plt.figure(figsize=(13.8, 8.6))
124        gs = fig.add_gridspec(2, 2, hspace=0.46, wspace=0.26)
125        ax = fig.add_subplot(gs[0, 0])
126        ax.plot(range(4), costs, marker="o", color=PALETTE["navy"], markersize=7, linewidth=2.2)
127        for i, value in enumerate(costs):
128            ax.annotate(f"{value:.1f}", (i, value), textcoords="offset points", xytext=(0, 10),
129                        ha="center", fontsize=10, fontweight="bold")
130        ax.set_xticks(range(4))
131        ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
132        ax.set_xlabel("可用的决策更新节点")

```

```

130 ax.set_ylabel("全年总费用 / 万元")
131 ax.set_title("新增决策节点带来的费用下降")
132 ax.set_ylim(costs.min() * 0.965, costs.max() * 1.022)
133 panel_label(ax, "(a)")
134 ax = fig.add_subplot(gs[0, 1])
135 controls = [summary["C1_load6"]["total_cost_yuan"], summary["C2_load12"]["
    total_cost_yuan"],
136             summary["C3_load18"]["total_cost_yuan"]]
137 load_value = np.array([summary[Q3_ORDER[i]]["total_cost_yuan"] - controls[i] for i in
    range(3)]) / 1e4
138 pv_value = np.array([controls[i] - summary[Q3_ORDER[i + 1]]["total_cost_yuan"] for i in
    range(3)]) / 1e4
139 labels = ["$V_6$", "$V_{12}$", "$V_{18}$"]
140 bars = ax.bar(labels, load_value, color=PALETTE["orange"],
141              edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="负载观测刷新")
142 ax.bar(labels, pv_value, bottom=load_value, color=PALETTE["teal"],
143        edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="光伏预报刷新")
144 values = load_value + pv_value
145 ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
146 for index, value in enumerate(values):
147     ax.text(index, value + max(values) * 0.025, f"{value:+.1f}", ha="center", va="bottom
    ",
148            fontsize=9.5, fontweight="bold")
149 ax.set_ylabel("增量价值 / 万元")
150 ax.set_title("节点价值的配对分解")
151 ax.set_ylim(min(0, min(values)) - 12, max(values) * 1.24)
152 ax.legend(fontsize=8.5)
153 panel_label(ax, "(b)")
154 ax = fig.add_subplot(gs[1, 0])
155 by_season = []
156 for key in Q3_ORDER:
157     frame = pd.read_csv(VARIANTS / "q3_replacement" / key / "by_season.csv", encoding="
    utf-8-sig")
158     frame = frame.set_index("season").reindex(["spring", "summer", "autumn", "winter"])
159     by_season.append(frame)
160 seasons = ["spring", "summer", "autumn", "winter"]
161 names = ["春季", "夏季", "秋季", "冬季"]
162 matrix = np.array([[f.loc[s, "total_cost_yuan"] / 1e4 for s in seasons] for f in
    by_season])
163 im = ax.imshow(matrix, cmap="YlGnBu", aspect="auto")
164 ax.set_xticks(range(4))
165 ax.set_xticklabels(names)
166 ax.set_yticks(range(4))
167 ax.set_yticklabels([Q3_LABEL[k].replace("\n", " ") for k in Q3_ORDER], fontsize=9)
168 for i in range(4):
169     for j in range(4):
170         ax.text(j, i, f"{matrix[i, j]:.0f}", ha="center", va="center", fontsize=9.5,

```

```

171         color="white" if matrix[i, j] > matrix.mean() else PALETTE["ink"])
172     ax.grid(False)
173     ax.set_title("季节维度的费用下降（万元）")
174     cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.92)
175     cb.set_label("万元", fontsize=9)
176     panel_label(ax, "(c)")
177     ax = fig.add_subplot(gs[1, 1])
178     up = [summary[k]["up_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
179     down = [summary[k]["down_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
180     em = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q3_ORDER]
181     x = np.arange(4)
182     ax.bar(x - 0.21, up, width=0.4, color=PALETTE["red"], label="向上调整",
183           edgecolor=PALETTE["ink"], linewidth=0.7)
184     ax.bar(x + 0.21, np.array(down) * -1, width=0.4, color=PALETTE["green"], label="向下调整",
185           edgecolor=PALETTE["ink"], linewidth=0.7)
186     ax.plot(x, em, color=PALETTE["orange"], marker="s", markersize=6, linewidth=2.0, label="
    紧急购电")
187     ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
188     ax.set_xticks(x)
189     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
190     ax.set_ylabel("电量 / MWh")
191     ax.set_title("调整电量与紧急购电")
192     ax.legend(fontsize=9, ncol=1)
193     panel_label(ax, "(d)")
194     save(fig, "q3_forecast_value")
195 def fig_q3_settlement() -> None:
196     repl = load_group("q3_replacement", "summary")
197     addi = load_group("q3_additive", "summary")
198     fig, axes = plt.subplots(1, 3, figsize=(13.8, 4.4))
199     plt.subplots_adjust(wspace=0.3)
200     ax = axes[0]
201     x = np.arange(4)
202     r = [repl[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
203     a = [addi[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
204     ax.bar(x - 0.2, r, width=0.4, color=PALETTE["navy"], label="差额部分结算（主方案）",
205           edgecolor=PALETTE["ink"], linewidth=0.8)
206     ax.bar(x + 0.2, a, width=0.4, color=PALETTE["gold"], label="并列相加（对照）",
207           edgecolor=PALETTE["ink"], linewidth=0.8)
208     ax.set_xticks(x)
209     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
210     ax.set_ylabel("全年总费用 / 万元")
211     ax.set_title("两种结算方式分别重优化")
212     ax.set_ylim(0, max(a) * 1.16)
213     ax.legend(fontsize=8.5)
214     panel_label(ax, "(a)")
215     ax = axes[1]

```

```

216 gap = [a[i] - r[i] for i in range(4)]
217 bars = ax.bar(x, gap, width=0.55, color=PALETTE["rose"], edgecolor=PALETTE["ink"],
218             linewidth=0.8)
219 _bar_labels(ax, bars, gap, "{:+.2f}")
220 ax.set_xticks(x)
221 ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
222 ax.set_ylabel("对照方案减主方案 / 万元")
223 ax.set_title("两种计费方式的费用差异")
224 ax.set_ylim(0, max(gap) * 1.25)
225 panel_label(ax, "(b)")
226 ax = axes[2]
227 for label, data, color in (("差额部分结算", r, PALETTE["navy"]), ("并列相加", a, PALETTE
228                             ["gold"])):
229     ax.plot(range(4), data, marker="o", color=color, linewidth=2.0, markersize=6.5,
230             label=label)
231 ax.set_xticks(range(4))
232 ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
233 ax.set_ylabel("全年总费用 / 万元")
234 ax.set_title("更新节点价值的计费稳健性")
235 ax.legend(fontsize=9)
236 panel_label(ax, "(c)")
237 save(fig, "q3_settlement")
238
239 def fig_q4_price_value() -> None:
240     summary = load_group("q4", "summary")
241     costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q4_ORDER])
242     fig = plt.figure(figsize=(13.8, 4.6))
243     gs = fig.add_gridspec(1, 3, wspace=0.3)
244     ax = fig.add_subplot(gs[0, 0])
245     colors = [PALETTE["sky"], PALETTE["red"], PALETTE["teal"], PALETTE["purple"]]
246     bars = ax.bar(range(4), costs, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8,
247                 width=0.6)
248     ax.set_xticks(range(4))
249     ax.set_xticklabels([Q4_LABEL[k] for k in Q4_ORDER], fontsize=8.5)
250     ax.set_ylabel("全年总费用 / 万元")
251     ax.set_title("波动电价下的四组结果")
252     ax.set_ylim(0, costs.max() * 1.16)
253     _bar_labels(ax, bars, costs, "{:.1f}")
254     panel_label(ax, "(a)")
255     ax = fig.add_subplot(gs[0, 1])
256     value_q2 = costs[1] - costs[0]
257     value_q3 = costs[3] - costs[2]
258     value_roll_perfect = costs[0] - costs[2]
259     value_roll_causal = costs[1] - costs[3]
260     labels = ["Q4-2\n发布信息差额", "Q4-3\n发布信息差额", "附件4直用下\n滚动调整价值", "因果
261               扩展下\n滚动调整价值"]
262     values = [value_q2, value_q3, value_roll_perfect, value_roll_causal]
263     bars = ax.bar(range(4), values,

```

```

258         color=[PALETTE["red"], PALETTE["purple"], PALETTE["teal"], PALETTE["green"]
259               ],
260         edgcolor=PALETTE["ink"], linewidth=0.8, width=0.6)
261 _bar_labels(ax, bars, values, "{:.1f}")
262 ax.set_xticks(range(4))
263 ax.set_xticklabels(labels, fontsize=8.5)
264 ax.set_ylabel("费用差额 / 万元")
265 ax.set_title("价格信息与滚动调整的价值")
266 ax.set_ylim(0, max(values) * 1.28)
267 panel_label(ax, "(b)")
268 ax = fig.add_subplot(gs[0, 2])
269 frame = pd.read_csv(VARIANTS / "q4" / "q43_causal_price" / "price_forecast_log.csv",
270                    encoding="utf-8-sig")
271 frame["hour"] = frame["decision_time"].str.slice(0, 2).astype(int)
272 grouped = frame.groupby("hour")["price_mae"].mean()
273 hours = grouped.index.values
274 ax.bar(hours, grouped.values, color=PALETTE["purple"], edgcolor=PALETTE["ink"],
275        linewidth=0.7, width=0.6)
276 ax.set_xticks(hours)
277 ax.set_xticklabels([f"{h}" for h in hours])
278 ax.set_xlabel("决策时刻 / h")
279 ax.set_ylabel("电价预测 MAE / (元·kWh-1)")
280 ax.set_title("因果电价预测的分时精度")
281 panel_label(ax, "(c)")
282 save(fig, "q4_price_value")
283 def fig_sensitivity() -> None:
284     sens = load_group("sensitivity", "summary")
285     terminal = sens["terminal"]
286     efficiency = sens["efficiency"]
287     fig = plt.figure(figsize=(13.8, 4.6))
288     gs = fig.add_gridspec(1, 3, wspace=0.3)
289     ax = fig.add_subplot(gs[0, 0])
290     policies = ["daily_cycle", "free"]
291     names = ["daily_cycle\n(计划 S$1$4$4$4$=S$0$)", "free\n(无终端约束)"]
292     totals = [terminal[p]["total_cost_yuan"] / 1e4 for p in policies]
293     bars = ax.bar(range(2), totals, color=[PALETTE["navy"], PALETTE["gold"]],
294                 edgcolor=PALETTE["ink"], linewidth=0.8, width=0.5)
295     _bar_labels(ax, bars, totals, "{:.1f}")
296     ax.set_xticks(range(2))
297     ax.set_xticklabels(names, fontsize=9)
298     ax.set_ylabel("全年总费用 / 万元")
299     ax.set_title("终端储电量策略敏感性")
300     ax.set_ylim(0, max(totals) * 1.16)
301     panel_label(ax, "(a)")
302     ax = fig.add_subplot(gs[0, 1])
303     keys = list(efficiency)
304     short = ["E1\n\\eta_c=\\eta_d=0.90$", "E2\n\\eta_c=\\eta_d=\\sqrt{0.9}$"]

```



```

302 totals = [efficiency[k]["total_cost_yuan"] / 1e4 for k in keys]
303 throughput = [efficiency[k]["storage_throughput_kwh"] / 1e3 for k in keys]
304 bars = ax.bar(range(len(keys)), totals, color=PALETTE["teal"], PALETTE["rose"]),
305             edgecolor=PALETTE["ink"], linewidth=0.8, width=0.5)
306 _bar_labels(ax, bars, totals, "{:.1f}")
307 for i, value in enumerate(throughput):
308     ax.text(i, totals[i] * 0.5, f"吞吐\n{value:.0f} MWh", ha="center", va="center",
309            fontsize=9, color="white", fontweight="bold")
310 ax.set_xticks(range(len(keys)))
311 ax.set_xticklabels(short, fontsize=9)
312 ax.set_ylabel("全年总费用 / 万元")
313 ax.set_title("储能效率口径敏感性")
314 ax.set_ylim(0, max(totals) * 1.16)
315 panel_label(ax, "(b)")
316 ax = fig.add_subplot(gs[0, 2])
317 data = [terminal[p]["daily_boundary_soc"]["mean"] for p in policies]
318 x = np.arange(2)
319 ax.bar(x, data, color=PALETTE["navy"], PALETTE["gold"]), edgecolor=PALETTE["ink"],
320       linewidth=0.8, width=0.5, label="日均日末储电量")
321 for i, p in enumerate(policies):
322     info = terminal[p]["daily_boundary_soc"]
323     ax.errorbar(i, info["mean"], yerr=[[info["mean"] - info["min"]], [info["max"] - info
324                                ["mean"]]],
325               fmt="none", ecolor=PALETTE["ink"], elinewidth=1.4, capsize=7)
326     ax.text(i, info["max"] + 200, f"std={info['std']:.0f}", ha="center", fontsize=9)
327 ax.set_xticks(x)
328 ax.set_xticklabels(names, fontsize=9)
329 ax.set_ylabel("日末储电量 / kWh")
330 ax.set_title("日边界储电量的分布")
331 ax.set_ylim(0, 12500)
332 panel_label(ax, "(c)")
333 save(fig, "sensitivity")
334 def fig_dispatch_overview(day_label: str = "2025-06-21", scenario: str = "q3_replacement/
335 S3_0_6_12_18") -> None:
336     path = VARIANTS / scenario / "solution.csv"
337     frame = pd.read_csv(path, encoding="utf-8-sig")
338     frame = frame[frame["date"] == day_label].reset_index(drop=True)
339     if frame.empty:
340         raise ValueError(f"{day_label} not found in {path}")
341     hours = (frame["slot"].to_numpy()) / 6.0
342     fig = plt.figure(figsize=(13.8, 8.4))
343     gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
344     ax = fig.add_subplot(gs[0, 0])
345     ax.fill_between(hours, 0, frame["load_kw"] / 1e3, color=PALETTE["sky"], alpha=0.55,
346                   label="小区负载")
347     ax.plot(hours, frame["actual_pv_kw"] / 1e3, color=PALETTE["gold"], linewidth=1.9, label=
348           "实际光伏")

```

```

345 ax.plot(hours, frame["discharge_actual_kwh"] * 6 / 1e3, color=PALETTE["teal"], linewidth
      =1.5, label="储能放电功率")
346 ax.plot(hours, -frame["charge_actual_kwh"] * 6 / 1e3, color=PALETTE["purple"], linewidth
      =1.5, label="储能充电功率")
347 ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
348 ax.set_xlabel("时刻 / h")
349 ax.set_ylabel("功率 / MW")
350 ax.set_title(f"{day_label} 功率平衡与储能充放电")
351 ax.set_xlim(0, 24)
352 ax.legend(fontsize=8.5, ncol=2)
353 panel_label(ax, "(a)")
354 ax = fig.add_subplot(gs[0, 1])
355 ax.plot(hours, frame["baseline_grid_kwh"] * 6 / 1e3, color=PALETTE["gray"], linewidth
      =1.6,
356         linestyle="--", label="0:00 基准计划 $G^{p}$")
357 ax.plot(hours, frame["adjusted_grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth
      =1.9,
358         label="最终调整购电 $G^{a}$")
359 ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"]
      * 6 / 1e3,
360                 where=(frame["adjusted_grid_kwh"] > frame["baseline_grid_kwh"]),
361                 color=PALETTE["red"], alpha=0.28, label="向上调整")
362 ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"]
      * 6 / 1e3,
363                 where=(frame["adjusted_grid_kwh"] <= frame["baseline_grid_kwh"]),
364                 color=PALETTE["green"], alpha=0.28, label="向下调整")
365 ax.set_xlabel("时刻 / h")
366 ax.set_ylabel("购电功率 / MW")
367 ax.set_title("计划与调整购电策略")
368 ax.set_xlim(0, 24)
369 ax.legend(fontsize=8.5)
370 panel_label(ax, "(b)")
371 ax = fig.add_subplot(gs[1, 0])
372 ax.plot(hours, frame["soc_actual_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.0)
373 ax.fill_between(hours, frame["soc_actual_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha
      =0.28)
374 ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
375 ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
376 ax.text(0.2, 1.35, "SOC 下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
377 ax.text(0.2, 10.95, "SOC 上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
378 ax.set_xlabel("时刻 / h")
379 ax.set_ylabel("储电量 / MWh")
380 ax.set_title("储能荷电状态的日内轨迹")
381 ax.set_xlim(0, 24)
382 ax.set_ylim(0.5, 12.2)
383 panel_label(ax, "(c)")
384 ax = fig.add_subplot(gs[1, 1])

```

```

385     ax.bar(hours, frame["emergency_kwh"] * 6, width=0.13, color=PALETTE["red"],
386            edgecolor=PALETTE["ink"], linewidth=0.5, label="紧急购电")
387     ax.bar(hours, frame["curtail_actual_kwh"] * 6, width=0.13, color=PALETTE["olive"],
388            edgecolor=PALETTE["ink"], linewidth=0.5, label="弃光")
389     ax.set_xlabel("时刻 / h")
390     ax.set_ylabel("功率 / kW")
391     ax.set_title("紧急购电与弃光的功率分布")
392     ax.set_xlim(0, 24)
393     ax.legend(fontsize=9)
394     panel_label(ax, "(d)")
395     save(fig, f"dispatch_{day_label.replace('-', '')}")
396 def main() -> None:
397     fig_q2_attribution()
398     fig_q3_forecast_value()
399     fig_q4_price_value()
400     fig_sensitivity()
401     for day in ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"):
402         try:
403             fig_dispatch_overview(day)
404         except ValueError as exc:
405             print("[warn]", exc)
406     print("[fig] 结果层配图完成 →", FIGS)
407 if __name__ == "__main__":
408     main()

```

B.23 技术路线图生成 (fig_route.py)

Listing 23:

```

1  from __future__ import annotations
2  import sys
3  from pathlib import Path
4  import matplotlib
5  matplotlib.use("Agg")
6  import matplotlib.pyplot as plt
7  from matplotlib.patches import FancyArrowPatch, FancyBboxPatch
8  ROOT = Path(__file__).resolve().parents[1]
9  FIGS = ROOT / "paper" / "figures"
10 INK = "#1A2028"
11 NAVY = "#1F3B73"
12 BANDS = [
13     ("数据层", "#E8F0FA", "#3E6FB0"),
14     ("统一内核", "#E6F4F1", "#2E8B84"),
15     ("四问模型", "#FBF0DE", "#D8A22B"),
16     ("求解算法", "#EDE9F6", "#7A5BA6"),
17     ("验证交付", "#EAF3E8", "#5FA05A"),

```

```

18 ]
19 CONTENT = [
20     ["附件 1\n典型日电价•负载•光伏", "附件 2\n全年 144 时段负载与光伏实际值",
21     "附件 3\n每日 4 次未来 24 h 光伏预报", "附件 4\n全年 144 时段实时电价"],
22     ["母线能量平衡\n式 (1)", "储能 SOC 状态转移\n $S_t=S_{t-1}+\eta_c C_t-H_t/\eta_d$ \n式
        (2) 核心状态方程",
23     "容量与功率限值\n式 (3) (4)", "购电非负与弃光上界\n式 (5) (6)"],
24     ["问题一\n确定性日前 LP\n首末 SOC 闭环",
25     "问题二\n因果预测•报童定价\n $5\$$  倍电价  $\rightarrow$   $0.2\$$  分位点\n实时储能反馈",
26     "问题三\n四时刻滚动优化\n两种结算口径\n分别重优化",
27     "问题四\n波动电价重算\nperfect / causal\n价格信息边界"],
28     ["HiGHS 线性规划\n原始一对偶容差  $10^{-9}$ ", "三层词典序目标\n费用  $\rightarrow$  吞吐量  $\rightarrow$  弃光",
29     "滚动在线选型\n仅用已发生误差", "因果偏差校正\n分发布节点分位数"],
30     ["物理残差校验\n平衡/SOC 残差  $<10^{-12}$ ", "未来信息泄漏\nmutation 测试",
31     "结算口径单元测试\n与 legacy 回归", "官方 result1-4\nExcel 回填与回读"],
32 ]
33 def main() -> None:
34     plt.rcParams.update({
35         "font.family": "sans-serif",
36         "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
37         "axes.unicode_minus": False,
38         "mathtext.fontset": "dejavusans",
39     })
40     band_h = 1.0
41     gap = 0.34
42     n_bands = len(BANDS)
43     fig_h = n_bands * band_h + (n_bands - 1) * gap + 0.5
44     fig, ax = plt.subplots(figsize=(13.2, fig_h * 1.35))
45     ax.set_xlim(0, 13.2)
46     ax.set_ylim(0, fig_h)
47     ax.axis("off")
48     left = 1.42
49     box_w = 2.79
50     box_gap = 0.155
51     for b, (band_name, fill, edge) in enumerate(BANDS):
52         y = fig_h - 0.25 - (b + 1) * band_h - b * gap
53         ax.add_patch(FancyBboxPatch(
54             (0.10, y), 13.00, band_h,
55             boxstyle="round,pad=0.008,rounding_size=0.10",
56             facecolor=fill, edgecolor=edge, linewidth=1.5, alpha=0.55, zorder=1,
57         ))
58         ax.text(0.72, y + band_h / 2, band_name, ha="center", va="center",
59             fontsize=11.0, fontweight="bold", color=edge, rotation=90, zorder=3)
60     for j, text in enumerate(CONTENT[b]):
61         x = left + j * (box_w + box_gap)
62         emphasized = b == 1 and j == 1
63         ax.add_patch(FancyBboxPatch(

```

```

64         (x, y + 0.09), box_w, band_h - 0.18,
65         boxstyle="round,pad=0.01,rounding_size=0.09",
66         facecolor="white", edgecolor=edge,
67         linewidth=2.0 if emphasized else 1.1, zorder=2,
68     ))
69     ax.text(x + box_w / 2, y + band_h / 2, text, ha="center", va="center",
70            fontsize=9.8 if emphasized else 10.2, color=INK, zorder=3, linespacing
71            =1.55)
72     for b in range(n_bands - 1):
73         y_top = fig_h - 0.25 - (b + 1) * band_h - b * gap
74         y_bottom = y_top - gap
75         for j in range(4):
76             x = left + j * (box_w + box_gap) + box_w / 2
77             ax.add_patch(FancyArrowPatch(
78                 (x, y_top), (x, y_bottom + 0.001),
79                 arrowstyle="->", mutation_scale=13,
80                 linewidth=1.5, color=NAVY, alpha=0.75, zorder=4,
81                 shrinkA=0, shrinkB=0,
82             ))
83     FIGS.mkdir(parents=True, exist_ok=True)
84     fig.savefig(FIGS / "technical_route.png", dpi=320, bbox_inches="tight", facecolor="white")
85     fig.savefig(FIGS / "technical_route.pdf", bbox_inches="tight", facecolor="white")
86     plt.close(fig)
87     print("[fig] technical_route 完成 →", FIGS / "technical_route.png")
88 if __name__ == "__main__":
89     main()

```

B.24 绘图总控 (plot_results.py)

Listing 24:

```

1  from __future__ import annotations
2  import csv,json
3  from datetime import datetime
4  from pathlib import Path
5  import numpy as np
6  import matplotlib.pyplot as plt
7  from matplotlib.font_manager import FontProperties
8  ROOT=Path(__file__).resolve().parents[1]
9  FONT=Path(r"C:\Windows\Fonts\msyh.ttc")
10 fp=FontProperties(fname=str(FONT)) if FONT.exists() else None
11 plt.rcParams["axes.unicode_minus"]=False
12 COLORS={"Q2":"#557A95","Q3":"#1B998B","Q4-2":"#D9A441","Q4-3":"#C8553D"}
13 def j(path): return json.loads(path.read_text(encoding="utf-8"))
14 def csvrows(path):

```

```

15     with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
16 def label(ax,x,y,fmt="{:.2f}"):
17     for xi,yi in zip(x,y): ax.text(xi,yi,fmt.format(yi),ha="center",va="bottom",fontsize=9,
18         fontproperties=fp)
19 def save(fig,path):
20     path.parent.mkdir(parents=True,exist_ok=True); fig.tight_layout(); fig.savefig(path,
21         bbox_inches="tight"); plt.close(fig)
22 def main():
23     rows=csvrows(ROOT/"outputs"/"q1"/"solution.csv"); x=np.arange(144)/6
24     load=np.array([float(r["load_kw"]) for r in rows]); pv=np.array([float(r["pv_kw"]) for r
25         in rows])
26     grid=np.array([float(r["grid_kwh"]) for r in rows])*6
27     ch=np.array([float(r["charge_kwh"]) for r in rows])*6; dis=np.array([float(r["
28         discharge_kwh"]) for r in rows])*6
29     soc=np.array([float(r["soc_kwh"]) for r in rows]); price=np.array([float(r["price"]) for
30         r in rows])
31     fig,(ax1,ax2)=plt.subplots(2,1,figsize=(11,7),sharex=True)
32     ax1.plot(x,load,label="负载",lw=1.8,color="#263238"); ax1.plot(x,pv,label="光伏",lw=1.8,
33         color="#E6A700")
34     ax1.plot(x,grid,label="外网购电",lw=1.6,color="#386FA4")
35     ax1.fill_between(x,0,ch,alpha=.25,color="#1B998B",label="充电功率")
36     ax1.fill_between(x,0,-dis,alpha=.25,color="#C8553D",label="放电功率")
37     ax1.set_ylabel("功率 / kW",fontproperties=fp); ax1.grid(alpha=.2); ax1.legend(ncol=5,
38         prop=fp,loc="upper center")
39     ax2.plot(x,soc,color="#6A4C93",lw=2,label="SOC"); ax2.set_ylabel("储电量 / kWh",
40         fontproperties=fp)
41     ax2.set_xlabel("时刻 / h",fontproperties=fp); ax2.grid(alpha=.2)
42     axp=ax2.twinx(); axp.step(x,price,where="post",color="#8D6E63",alpha=.75,label="电价")
43     axp.set_ylabel("电价 / 元/kWh",fontproperties=fp)
44     fig.suptitle("问题1: 典型日购电-储能联合调度",fontproperties=fp,fontsize=15)
45     save(fig,ROOT/"figures"/"q1"/"q1_dispatch.pdf")
46     metrics={name:j(ROOT/"outputs"/folder/"metrics.json") for name,folder in
47         [("<Q2","q2"),("<Q3","q3"),("<Q4-2","q4-2"),("<Q4-3","q4-3")]}
48     names=list(metrics); xpos=np.arange(4)
49     costs=np.array([metrics[n]["total_cost_yuan"]]/1e4 for n in names])
50     fig,ax=plt.subplots(figsize=(8,5)); bars=ax.bar(xpos, costs,color=[COLORS[n] for n in
51         names],width=.62)
52     ax.set_xticks(xpos,names); ax.set_ylabel("全年总费用 / 万元",fontproperties=fp); ax.grid
53         (axis="y",alpha=.2)
54     ax.set_title("固定与波动电价下的全年费用对比",fontproperties=fp,fontsize=14); label(ax,
55         xpos, costs)
56     save(fig,ROOT/"figures"/"comparison"/"annual_cost_comparison.pdf")
57     emergency=np.array([metrics[n]["emergency_energy_kwh"]]/1e4 for n in names])
58     fig,ax=plt.subplots(figsize=(8,5)); ax.bar(xpos,emergency,color=[COLORS[n] for n in
59         names],width=.62)
60     ax.set_xticks(xpos,names); ax.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.
61         grid(axis="y",alpha=.2)

```

```

49     ax.set_title("滚动预报对 5 倍紧急购电的抑制",fontproperties=fp,fontsize=14); label(ax,
        xpos,emergency,fmt="{:.3f}")
50     save(fig,ROOT/"figures"/"comparison"/"emergency_energy_comparison.pdf")
51     variants=[("原始阶梯","q3/variants/raw_step"),("校正阶梯","q3/variants/calibrated_step")
        ,("校正线性","q3")]
52     vm=[j(ROOT/"outputs"/p/"metrics.json") for _,p in variants]
53     vcost=np.array([m["total_cost_yuan"]/1e4 for m in vm]); vem=np.array([m["
        emergency_energy_kwh"]/1e4 for m in vm])
54     fig,ax=plt.subplots(figsize=(9,5.5)); xx=np.arange(3); width=.34
55     b1=ax.bar(xx-width/2,vcost,width,color="#557A95",label="总费用（万元）")
56     ax.set_ylabel("总费用 / 万元",fontproperties=fp); ax.set_xticks(xx,[v[0] for v in
        variants],fontproperties=fp)
57     ax2=ax.twinx(); b2=ax2.bar(xx+width/2,vem,width,color="#C8553D",label="紧急购电（万kWh）
        ")
58     ax2.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.grid(axis="y",alpha=.2)
59     ax.set_title("Q3 策略变更的完整对照",fontproperties=fp,fontsize=14)
60     ax.bar_label(b1,fmt="%.2f",padding=3,fontsize=8)
61     ax2.bar_label(b2,fmt="%.3f",padding=3,fontsize=8)
62     ax.legend(handles=[b1,b2],labels=["总费用","紧急购电"],prop=fp,loc="upper right")
63     save(fig,ROOT/"figures"/"comparison"/"q3_variant_comparison.pdf")
64     fig,ax=plt.subplots(figsize=(11,5.5))
65     for name,folder in [("Q2","q2"),("Q3","q3"),("Q4-2","q4-2"),("Q4-3","q4-3")]:
66         rows=csvrows(ROOT/"outputs"/folder/"daily_metrics.csv")
67         dates=np.array([datetime.fromisoformat(r["date"]) for r in rows])
68         daily=np.array([float(r["total_cost"]) for r in rows])
69         smooth=np.convolve(daily,np.ones(7)/7,mode="same")
70         ax.plot(dates[3:-3],smooth[3:-3]/1e4,label=name,color=COLORS[name],lw=1.5)
71         ax.set_ylabel("7日移动平均费用 / 万元/日",fontproperties=fp); ax.grid(alpha=.2)
72         ax.set_title("全年日费用变化（7日移动平均）",fontproperties=fp,fontsize=14); ax.legend(
            prop=fp,ncol=4)
73         save(fig,ROOT/"figures"/"comparison"/"daily_cost_timeseries.pdf")
74         print("generated 5 PDF figures")
75     if __name__=="__main__": main()

```

B.25 mutation 测试辅助函数（mutate.py）

Listing 25:

```

1  from __future__ import annotations
2  import shutil
3  from pathlib import Path
4  import openpyxl
5  from ..solve.io_data import DEFAULT_DATA_ROOT
6  ATTACHMENTS = ("附件1.xlsx", "附件2.xlsx", "附件3.xlsx", "附件4.xlsx")
7  def make_root(tmp: Path, source: Path = DEFAULT_DATA_ROOT) -> Path:
8      tmp.mkdir(parents=True, exist_ok=True)

```

```

9     for name in ATTACHMENTS:
10         target = tmp / name
11         if not target.exists():
12             shutil.copy2(source / name, target)
13     return tmp
14 def _mutate_sheet(path: Path, sheet: str, day_index: int, from_slot: int, factor: float) ->
    None:
15     workbook = openpyxl.load_workbook(path)
16     worksheet = workbook[sheet] if sheet in workbook.sheetnames else workbook.worksheets[0]
17     row = day_index + 2
18     for col in range(from_slot + 2, worksheet.max_column + 1):
19         cell = worksheet.cell(row, col)
20         if cell.value is None:
21             continue
22         cell.value = float(cell.value) * factor
23     workbook.save(path)
24     workbook.close()
25 def mutate_attachment2_load(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
26     _mutate_sheet(root / "附件2.xlsx", "小区负载", day_index, from_slot, factor)
27 def mutate_attachment2_pv(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
28     _mutate_sheet(root / "附件2.xlsx", "光伏发电实际功率", day_index, from_slot, factor)
29 def mutate_attachment4_price(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
30     _mutate_sheet(root / "附件4.xlsx", "Sheet1", day_index, from_slot, factor)
31 def max_abs_diff(left, right) -> float:
32     import numpy as np
33     return float(np.max(np.abs(np.asarray(left, dtype=float) - np.asarray(right, dtype=float
        ))))

```

B.26 未来信息泄漏 mutation 测试 (test_causality.py)

Listing 26:

```

1 from __future__ import annotations
2 import shutil
3 import tempfile
4 import unittest
5 from pathlib import Path
6 import numpy as np
7 from ..solve.io_data import read_attachment4
8 from ..solve.q2 import run_q2
9 from ..solve.q3 import run_q3
10 from .mutate import (
11     make_root,

```



```

12     mutate_attachment2_load,
13     mutate_attachment2_pv,
14     mutate_attachment4_price,
15 )
16 TARGET_DAY = 35
17 DAYS = 36
18 TOL = 1e-10
19 class CausalityTestBase(unittest.TestCase):
20     @classmethod
21     def setUpClass(cls) -> None:
22         cls._tmpdir = tempfile.mkdtemp(prefix="mathmodel_causal_")
23         cls.tmp = Path(cls._tmpdir)
24         cls.base = make_root(cls.tmp / "base")
25         cls.mut_load = make_root(cls.tmp / "mut_load")
26         mutate_attachment2_load(cls.mut_load, TARGET_DAY, 0, 10.0)
27         cls.mut_pv = make_root(cls.tmp / "mut_pv")
28         mutate_attachment2_pv(cls.mut_pv, TARGET_DAY, 0, 10.0)
29         cls.mut_both_after_6 = make_root(cls.tmp / "mut_both_after6")
30         mutate_attachment2_load(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
31         mutate_attachment2_pv(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
32         cls.mut_price_after_6 = make_root(cls.tmp / "mut_price_after6")
33         mutate_attachment4_price(cls.mut_price_after_6, TARGET_DAY, 36, 10.0)
34     @classmethod
35     def tearDownClass(cls) -> None:
36         shutil.rmtree(cls._tmpdir, ignore_errors=True)
37     @staticmethod
38     def _run_q2(root: Path, **kwargs):
39         return run_q2(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
40             , **kwargs)
41     @staticmethod
42     def _run_q3(root: Path, **kwargs):
43         kwargs.setdefault("settlement_mode", "replacement")
44         return run_q3(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
45             , **kwargs)
46 class Q2LoadCausalityTest(CausalityTestBase):
47     def test_future_load_cannot_change_plan(self):
48         base = self._run_q2(self.base)
49         mutated = self._run_q2(self.mut_load)
50         b = base["records"][TARGET_DAY]["load_plan_kw"]
51         m = mutated["records"][TARGET_DAY]["load_plan_kw"]
52         self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实负载被篡改后 0:00 计
53             划输入发生变化")
54     def test_future_load_changes_execution_only(self):
55         base = self._run_q2(self.base)
56         mutated = self._run_q2(self.mut_load)
57         b = base["records"][TARGET_DAY]["execution"].emergency.sum()
58         m = mutated["records"][TARGET_DAY]["execution"].emergency.sum()

```

```

56         self.assertNotAlmostEqual(b, m, places=3)
57     class Q2PVCausalityTest(CausalityTestBase):
58         def test_future_pv_cannot_change_plan(self):
59             base = self._run_q2(self.base)
60             mutated = self._run_q2(self.mut_pv)
61             b = base["records"][TARGET_DAY]["pv_plan_kw"]
62             m = mutated["records"][TARGET_DAY]["pv_plan_kw"]
63             self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实光伏被篡改后 0:00 计
                划输入发生变化")
64     class Q3NodeCausalityTest(CausalityTestBase):
65         def test_node_inputs_do_not_see_future_load_or_pv(self):
66             base = self._run_q3(self.base)
67             mutated = self._run_q3(self.mut_both_after_6)
68             for day in (TARGET_DAY,):
69                 b_load = base["records"][day]["load_plan_kw"][36:72]
70                 m_load = mutated["records"][day]["load_plan_kw"][36:72]
71                 self.assertLess(float(np.max(np.abs(b_load - m_load))), TOL, "6:00 负载预测输入
                    受到未来真实负载影响")
72                 b_block = base["records"][day]["adjusted_grid"][36:72]
73                 m_block = mutated["records"][day]["adjusted_grid"][36:72]
74                 self.assertLess(float(np.max(np.abs(b_block - m_block))), TOL, "6:00 优化结果受
                    到未来真实负载/光伏影响")
75                 b_base = base["records"][day]["baseline_plan"].grid
76                 m_base = mutated["records"][day]["baseline_plan"].grid
77                 self.assertLess(float(np.max(np.abs(b_base - m_base))), TOL, "0:00 基准计划受到
                    未来真实负载/光伏影响")
78     class Q4PriceCausalityTest(CausalityTestBase):
79         def test_causal_price_ignores_future_realized_price(self):
80             prices = read_attachment4(self.base).price
81             base = self._run_q3(self.base, price_matrix=prices, price_mode="causal")
82             mutated = self._run_q3(self.mut_price_after_6, price_matrix=prices, price_mode="
                causal")
83             b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
84             m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
85             self.assertLess(float(np.max(np.abs(b - m))), TOL, "causal 电价模式下提前看到未来真
                实电价")
86         def test_perfect_price_does_see_future_realized_price(self):
87             base = self._run_q3(self.base, price_matrix=read_attachment4(self.base).price,
                price_mode="perfect")
88             mutated = self._run_q3(
89                 self.mut_price_after_6,
90                 price_matrix=read_attachment4(self.mut_price_after_6).price,
91                 price_mode="perfect",
92             )
93             self.assertNotAlmostEqual(
94                 base["metrics"]["total_cost_yuan"],
95                 mutated["metrics"]["total_cost_yuan"],

```

```

96         places=3,
97     )
98     if __name__ == "__main__":
99         unittest.main()

```

B.27 主结果与计划持续性测试 (test_q2_q3.py)

Listing 27:

```

1  import unittest
2  import numpy as np
3  from ..solve.io_data import read_attachment1,read_attachment2,read_attachment3,
    read_attachment4,hourly_to_ten_min
4  from ..solve.forecast import OnlinePVForecaster
5  from ..solve.lp_core import solve_dispatch,solve_adjustment_dispatch,SOC_MIN,SOC_MAX
6  from ..solve.q3 import run_q3
7  from ..solve.settle import execute_with_realtime_storage_feedback
8  class Q2Q3Tests(unittest.TestCase):
9      def test_attachment_dimensions(self):
10         data=read_attachment2(); forecast=read_attachment3(); prices=read_attachment4()
11         self.assertEqual(data.load_kw.shape,(365,144)); self.assertEqual(data.pv_kw.shape
            ,(365,144))
12         self.assertEqual(forecast.hourly_kw.shape,(365,4,24)); self.assertEqual(prices.price
            .shape,(365,144))
13         step=hourly_to_ten_min(forecast.hourly_kw[0,0],"step")
14         self.assertAlmostEqual(float(step.sum()/6),float(forecast.hourly_kw[0,0].sum()),
            places=9)
15     def test_forecast_is_sequential(self):
16         f=OnlinePVForecaster(np.ones(144)); first=f.predict(); self.assertEqual(first.
            history_days_used,0)
17         actual=np.r_[np.zeros(40),np.ones(60)*100,np.zeros(44)]; f.observe(first,actual)
18         second=f.predict(); self.assertEqual(second.history_days_used,1)
19         self.assertTrue(np.allclose(second.bases["persistence"],actual))
20         with self.assertRaises(ValueError): f.observe(first,actual)
21     def test_realtime_controller_constraints(self):
22         day=read_attachment1(); plan=solve_dispatch(day.load_kw,np.zeros(144),day.price,
            soc_initial=6000,soc_terminal=6000)
23         actual=np.maximum(day.pv_kw*1.3,0); execution=execute_with_realtime_storage_feedback
            (plan,day.load_kw,actual,soc_initial=6000)
24         balance=execution.grid+actual/6+execution.discharge+execution.emergency-day.load_kw
            /6-execution.charge-execution.curtail
25         self.assertLess(float(np.max(np.abs(balance))),1e-7)
26         self.assertGreaterEqual(float(execution.soc.min()),SOC_MIN-1e-7)
27         self.assertLessEqual(float(execution.soc.max()),SOC_MAX+1e-7)
28         self.assertLess(float(np.max(np.minimum(execution.charge,execution.discharge))),1e
            -7)

```

```

29     def test_adjustment_linearization(self):
30         gp=np.array([900.0,900.0,700.0]); ga=np.array([700.0,900.0,900.0])
31         b=np.maximum(ga-gp,0); linear=0.5*gp+0.5*ga+b
32         piece=np.minimum(gp,ga)+0.5*np.maximum(gp-ga,0)+1.5*np.maximum(ga-gp,0)
33         self.assertTrue(np.allclose(linear,[800,900,1000])); self.assertTrue(np.allclose(
            linear,piece))
34     def test_adjustment_solver_balance(self):
35         day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
            soc_initial=6000,soc_terminal=6000)
36         result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,np.
            ones(144,dtype=bool),
37                                     soc_initial=6000,soc_terminal=6000)
38         d=result.dispatch
39         residual=d.grid+day.pv_kw*0.9/6+d.discharge-day.load_kw/6-d.charge-d.curtail
40         self.assertLess(float(np.max(np.abs(residual))),1e-7)
41         self.assertLess(float(np.max(np.abs(result.increase-np.maximum(d.grid-base.grid,0)))
            ),1e-6)
42     def test_adjustment_milp_enforces_storage_exclusivity(self):
43         day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
            soc_initial=6000,soc_terminal=6000)
44         for mode in ("replacement","additive"):
45             result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,
                np.ones(144,dtype=bool),
46                                     soc_initial=6000,soc_terminal=6000,
                settlement_mode=mode)
47             self.assertLess(float(np.max(np.minimum(result.dispatch.charge,result.dispatch.
                discharge))),1e-9)
48     def test_disabled_nodes_keep_latest_active_plan(self):
49         result=run_q3(max_days=2,write_outputs=False,update_nodes=(0,36),settlement_mode="
            replacement")
50         record=result["records"][-1]
51         self.assertTrue(np.all(record["plan_source_nodes"][:36]==0))
52         self.assertTrue(np.all(record["plan_source_nodes"][36:]==36))
53         self.assertLess(float(np.max(np.abs(record["adjusted_grid"][72:]-record["revisions"
            ][1,72:])))),1e-9)
54 if __name__=="__main__": unittest.main()

```

B.28 论文数值与提交口径一致性测试 (test_paper_consistency.py)

Listing 28:

```

1 from __future__ import annotations
2 import hashlib
3 import json
4 from pathlib import Path
5 ROOT = Path(__file__).resolve().parents[1]

```

```

6 VARIANTS = ROOT / "outputs" / "variants" / "model_audit"
7 DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
8 def _json(path: Path) -> dict:
9     return json.loads(path.read_text(encoding="utf-8"))
10 def _sha256(path: Path) -> str:
11     return hashlib.sha256(path.read_bytes()).hexdigest()
12 def test_replacement_main_cost_decomposition_and_exclusivity() -> None:
13     metrics = _json(VARIANTS / "q3_replacement" / "S3_0_6_12_18" / "metrics.json")
14     components = (
15         metrics["baseline_purchase_cost_yuan"]
16         + metrics["adjustment_cost_yuan"]
17         + metrics["emergency_cost_yuan"]
18     )
19     assert abs(metrics["total_cost_yuan"] - components) < 1e-6
20     assert metrics["max_simultaneous_charge_discharge"] < 1e-5
21     assert metrics["passed"] is True
22 def test_paired_node_decomposition_is_exact() -> None:
23     summary = _json(VARIANTS / "q3_replacement" / "summary.json")
24     chain = (
25         ("S0_0", "C1_load6", "S1_0_6"),
26         ("S1_0_6", "C2_load12", "S2_0_6_12"),
27         ("S2_0_6_12", "C3_load18", "S3_0_6_12_18"),
28     )
29     for before, control, after in chain:
30         total = summary[before]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
31         load_part = summary[before]["total_cost_yuan"] - summary[control]["total_cost_yuan"]
32         pv_part = summary[control]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
33         assert abs(total - load_part - pv_part) < 1e-6
34 def test_required_date_tables_cover_all_contract_dates() -> None:
35     generated = ROOT / "paper" / "generated"
36     for name in ("q2_required_tables.tex", "q3_required_tables.tex", "q4_required_tables.tex"):
37         text = (generated / name).read_text(encoding="utf-8")
38         for date in DATES:
39             assert date in text
40 def test_q4_submission_uses_attachment4_direct_results() -> None:
41     pairs = (
42         ("result4-2.xlsx", VARIANTS / "q4" / "q42_perfect_price" / "result4-2.xlsx"),
43         ("result4-3.xlsx", VARIANTS / "q4" / "q43_perfect_price" / "result4-3.xlsx"),
44     )
45     for name, source in pairs:
46         assert _sha256(ROOT / "submit" / name) == _sha256(source)
47 def test_frozen_submission_results_unchanged() -> None:
48     expected = {
49         "result1.xlsx": "66363557c72486358af304657b5b8ec105fc2150aed09ca9727915fe80931636",
50         "result2.xlsx": "5c12208e4a609cb06f663d37ec16280f1ecf10d97d578c2025171cafc49e6c15",
51         "result4-2.xlsx": "c542ca6ec322e7012ed4fe6f496686cc9cb527e3fb78358d9c4c6a8214fd5cf6"

```

```

52     }
53     for name, digest in expected.items():
54         assert _sha256(ROOT / "submit" / name) == digest

```

B.29 结算口径单元测试 (test_settlement.py)

Listing 29:

```

1  from __future__ import annotations
2  import unittest
3  import numpy as np
4  from ..solve.lp_core import settlement_cost, solve_adjustment_dispatch
5  from ..solve.params import DEFAULT_STORAGE
6  class ReplacementFormulaTest(unittest.TestCase):
7      def test_purchase_below_plan(self):
8          value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
          replacement")
9          self.assertAlmostEqual(value, 800.0, places=9)
10     def test_purchase_above_plan(self):
11         value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
          replacement")
12         self.assertAlmostEqual(value, 1000.0, places=9)
13     def test_unchanged(self):
14         value = settlement_cost(np.array([800.0]), np.array([800.0]), np.array([1.0]), "
          replacement")
15         self.assertAlmostEqual(value, 800.0, places=9)
16     class AdditiveFormulaTest(unittest.TestCase):
17         def test_purchase_below_plan(self):
18             value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
              additive")
19             self.assertAlmostEqual(value, 1000.0, places=9)
20         def test_purchase_above_plan(self):
21             value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
              additive")
22             self.assertAlmostEqual(value, 1000.0, places=9)
23     def _case(seed: int, t: int = 24, mask_all: bool = True):
24         rng = np.random.default_rng(seed)
25         load = 800.0 + 200.0 * rng.random(t)
26         pv = 400.0 * rng.random(t)
27         price = 0.3 + 0.7 * rng.random(t)
28         baseline = np.maximum((load - pv) * (1 / 6), 0.0)
29         mask = np.ones(t, dtype=bool) if mask_all else np.r_[np.ones(t - 6, dtype=bool), np.
          zeros(6, dtype=bool)]
30         return load, pv, price, baseline, mask
31     def _model_objective(result, price, baseline, mode: str) -> float:

```

```

32     mask = result.adjustment_mask
33     p = price[mask]
34     if mode == "replacement":
35         return float(np.dot(p, result.dispatch.grid[mask] + 0.5 * result.increase[mask] +
36                             0.5 * result.decrease[mask]))
37     return float(np.dot(p, baseline[mask]) + np.dot(p, 0.5 * result.decrease[mask] + 1.5 *
38                             result.increase[mask]))
39
40 class LinearizationTest(unittest.TestCase):
41     def _gap(self, mode: str, seed: int) -> float:
42         load, pv, price, baseline, mask = _case(seed)
43         result = solve_adjustment_dispatch(
44             load, pv, price, baseline, mask,
45             soc_initial=6000.0, soc_terminal=6000.0, settlement_mode=mode,
46         )
47         direct = settlement_cost(baseline, result.dispatch.grid, price, mode)
48         return abs(direct - _model_objective(result, price, baseline, mode))
49     def test_replacement_linearization(self):
50         for seed in range(5):
51             self.assertLess(self._gap("replacement", seed), 1e-7, f"seed={seed}")
52     def test_additive_linearization(self):
53         for seed in range(5):
54             self.assertLess(self._gap("additive", seed), 1e-7, f"seed={seed}")
55     def test_two_modes_are_reoptimized_not_revalued(self):
56         for seed in range(5):
57             load, pv, price, baseline, mask = _case(seed)
58             kwargs = dict(soc_initial=6000.0, soc_terminal=6000.0)
59             replacement = solve_adjustment_dispatch(
60                 load, pv, price, baseline, mask, settlement_mode="replacement", **kwargs
61             )
62             additive = solve_adjustment_dispatch(
63                 load, pv, price, baseline, mask, settlement_mode="additive", **kwargs
64             )
65             best_replacement = settlement_cost(baseline, replacement.dispatch.grid, price, "
66                 replacement")
67             cross_replacement = settlement_cost(baseline, additive.dispatch.grid, price, "
68                 replacement")
69             best_additive = settlement_cost(baseline, additive.dispatch.grid, price, "
70                 additive")
71             cross_additive = settlement_cost(baseline, replacement.dispatch.grid, price, "
72                 additive")
73             tol_r = 1e-6 * max(1.0, abs(best_replacement))
74             tol_a = 1e-6 * max(1.0, abs(best_additive))
75             self.assertLessEqual(best_replacement, cross_replacement + tol_r, f"seed={seed}")
76             self.assertLessEqual(best_additive, cross_additive + tol_a, f"seed={seed}")
77
78 class StorageParamTest(unittest.TestCase):
79     def test_energy_limit(self):

```

```

72         self.assertAlmostEqual(DEFAULT_STORAGE.energy_limit, 5000.0 / 6.0, places=9)
73 if __name__ == "__main__":
74     unittest.main()

```

B.30 历史配置回归测试 (test_regression.py)

Listing 30:

```

1  from __future__ import annotations
2  import json
3  import unittest
4  from pathlib import Path
5  from ..solve.io_data import read_attachment4
6  from ..solve.q1 import run_q1
7  from ..solve.q2 import run_q2
8  from ..solve.q3 import run_q3
9  FIXTURE = Path(__file__).with_name("legacy_metrics.json")
10 LEGACY = json.loads(FIXTURE.read_text(encoding="utf-8"))
11 ABS_TOL = 1e-3
12 REL_TOL = 1e-9
13 def _close(test: unittest.TestCase, label: str, new: float, old: float, abs_tol: float =
    ABS_TOL) -> None:
14     tolerance = max(abs_tol, REL_TOL * abs(old))
15     test.assertLess(
16         abs(new - old),
17         tolerance,
18         f"{label} 未复现 legacy 数值: new={new!r} legacy={old!r} diff={new - old!r}",
19     )
20 class LegacyRegressionTest(unittest.TestCase):
21     @classmethod
22     def setUpClass(cls) -> None:
23         cls.prices = read_attachment4().price
24     def test_q1_unchanged(self):
25         metrics = run_q1()["metrics"]
26         _close(self, "Q1 费用", metrics["cost_yuan"], LEGACY["q1"]["cost_yuan"])
27         _close(self, "Q1 购电量", metrics["grid_energy_kwh"], LEGACY["q1"]["grid_energy_kwh"
            ])
28         _close(self, "Q1 充电量", metrics["charge_energy_kwh"], LEGACY["q1"]["
            charge_energy_kwh"])
29     def test_q2_legacy_benchmark(self):
30         metrics = run_q2(
31             max_days=365,
32             write_outputs=False,
33             load_mode="perfect",
34             pv_risk_mode="asymmetric",
35             realtime_feedback=True,

```



```

36         price_mode="perfect",
37         terminal_policy="daily_cycle",
38         use_typical_prior=False,
39     )["metrics"]
40     _close(self, "Q2 总费用", metrics["total_cost_yuan"], LEGACY["q2"]["total_cost_yuan"
41         ])
42     _close(self, "Q2 计划费用", metrics["plan_purchase_cost_yuan"], LEGACY["q2"]["
43         plan_purchase_cost_yuan"])
44     _close(self, "Q2 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q2"]["
45         emergency_energy_kwh"])
46     _close(self, "Q2 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q2"]["
47         curtailment_energy_kwh"])
48
49     def test_q3_legacy_benchmark(self):
50         metrics = run_q3(
51             max_days=365,
52             write_outputs=False,
53             downscale="linear",
54             calibrate=True,
55             update_nodes=(0, 36, 72, 108),
56             settlement_mode="replacement",
57             load_mode="perfect",
58             price_mode="perfect",
59             realtime_feedback=True,
60             terminal_policy="daily_cycle",
61             use_typical_prior=False,
62         )["metrics"]
63         _close(self, "Q3 总费用", metrics["total_cost_yuan"], LEGACY["q3"]["total_cost_yuan"
64             ])
65         _close(self, "Q3 基准计划费用", metrics["baseline_purchase_cost_yuan"], LEGACY["q3"
66             ]["baseline_purchase_cost_yuan"])
67         _close(self, "Q3 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q3"]["
68             emergency_energy_kwh"])
69         _close(self, "Q3 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q3"]["
70             curtailment_energy_kwh"], abs_tol=1e-2)
71
72     def test_q4_legacy_benchmark(self):
73         metrics2 = run_q2(
74             max_days=365,
75             write_outputs=False,
76             price_matrix=self.prices,
77             load_mode="perfect",
78             pv_risk_mode="asymmetric",
79             realtime_feedback=True,
80             price_mode="perfect",
81             use_typical_prior=False,
82         )["metrics"]
83         _close(self, "Q4-2 总费用", metrics2["total_cost_yuan"], LEGACY["q4-2"]["
84             total_cost_yuan"])

```

```
74     metrics3 = run_q3(
75         max_days=365,
76         write_outputs=False,
77         price_matrix=self.prices,
78         update_nodes=(0, 36, 72, 108),
79         settlement_mode="replacement",
80         load_mode="perfect",
81         price_mode="perfect",
82         realtime_feedback=True,
83         use_typical_prior=False,
84     )["metrics"]
85     _close(self, "Q4-3 总费用", metrics3["total_cost_yuan"], LEGACY["q4-3"]["
        total_cost_yuan"])
86 if __name__ == "__main__":
87     unittest.main()
```